

Spec-Driven Development with OpenSpec and GitHub Copilot

A Practical Guide for Java and Angular Developers

Gregg Obst

09/22/2026

Table of contents

Preface	1
I. Part I: Why Durable Intent Matters	3
1. From Chat Sessions to Durable Intent	5
1.1. A request that looks complete	5
1.2. The same request with OpenSpec	5
1.3. What this book will teach you to do	5
1.4. The running example	6
1.5. Learning objective	6
1.6. The coordination problem	6
1.7. Jira, chat, and repository artifacts	7
1.8. When a specification earns its cost	7
1.9. Hands-on exercise	7
1.10. Verification checkpoint	7
1.11. Online references	8
2. The OpenSpec Operating Model	9
2.1. Learning objective	9
2.2. Read the tree as two kinds of truth	9
2.3. Follow one change through its artifacts	10
2.3.1. Proposal: why this change, and how far does it reach?	10
2.3.2. Delta spec: what observable behavior would change?	11
2.3.3. Design: how might the implementation satisfy the behavior?	11
2.3.4. Tasks: what work remains, and in what order?	11
2.4. Move through the loop with review points	12
2.4.1. Explore without pretending the plan is settled	12
2.4.2. Propose, then review the artifacts	12
2.4.3. Apply from the plan, but allow the plan to be wrong	12
2.4.4. Verify against the change, not only the task list	13
2.4.5. Sync or archive when the baseline should change	13
2.5. Distinguish the interface from the operation	13
2.6. Hands-on exercise	14
2.7. Verification checkpoint	14
2.8. What you learned	14
2.9. Online references	15
3. Writing Requirements an Agent Can Test	17
3.1. Learning objective	17
3.2. Start with the decisions hidden in the story	17
3.3. Write the requirement as a behavioral contract	18

3.4.	Give each scenario one job	19
3.4.1.	Cover decisions, not every imaginable input	19
3.5.	Let the agent draft without letting it decide	20
3.6.	Validate structure, then review meaning	20
3.7.	Connect scenarios to implementation evidence	21
3.8.	Maintain the requirement when behavior changes	21
3.9.	Hands-on exercise	22
3.10.	Verification checkpoint	23
3.11.	What you learned	23
3.12.	Online references	23
4.	Engineering Context for GitHub Copilot	25
4.1.	Learning objective	25
4.2.	Context engineering in an SDD workflow	25
4.3.	Treat the context window as shared space	26
4.3.1.	Start with the smallest authoritative package	26
4.4.	Choose a model with the task and window in mind	27
4.5.	Expect long conversations to be compacted	27
4.6.	Carry the working state forward in Markdown	28
4.7.	Use each Copilot client deliberately	29
4.7.1.	VS Code	30
4.7.2.	IntelliJ IDEA and other JetBrains IDEs	30
4.7.3.	GitHub Copilot CLI	30
4.8.	Understand caching without confusing it with memory	31
4.9.	Reduce context waste before it reaches the model	31
4.10.	Hands-on exercise	32
4.11.	Verification checkpoint	32
4.12.	What you learned	33
4.13.	Online references	33
II.	Part II: Establishing the Greenfield Baseline	35
5.	Preparing the Two-Repository Lab	37
5.1.	Learning objective	37
5.2.	One product, two specification histories	37
5.3.	Let the contract cross the boundary	38
5.4.	Decide where a statement belongs	38
5.5.	Start from a known baseline	39
5.6.	Backend baseline	39
5.7.	Frontend baseline	40
5.8.	VS Code and IntelliJ IDEA setup	40
5.9.	Hands-on exercise	40
5.10.	Verification checkpoint	41
5.11.	What you learned	41
5.12.	Online references	41
6.	Initialize and Configure OpenSpec	43
6.1.	Learning objective	43
6.2.	Begin at the repository root	43
6.3.	Separate installation from initialization	43

6.4. What initialization adds	44
6.5. Configure context that can affect every change	45
6.6. Configure the backend repository	45
6.7. Configure the frontend repository	46
6.8. Understand generated files and team-owned context	46
6.9. Review initialization as a code change	47
6.10. Verify the initialized repositories	47
6.11. Hands-on exercise: initialize both repositories	48
6.12. Verification checkpoint	48
6.13. What you learned	49
6.14. Online references	49
7. Exploring the Unit-Number Search	51
7.1. Learning objective	51
7.2. Treat exploration as investigation	51
7.3. Begin with the thin request	51
7.4. Establish the exploration boundary	52
7.5. Ask Copilot for evidence, not a solution	52
7.6. Separate evidence from inference	53
7.7. Build one cross-repository question set	54
7.8. Turn questions into a decision record	54
7.9. Use follow-up prompts to challenge the record	55
7.10. Compare VS Code and IntelliJ IDEA	55
7.11. Know when to stop exploring	56
7.12. Hands-on exercise	56
7.13. Verification checkpoint	56
7.14. What you learned	57
7.15. Online references	57
8. Proposing the First Full-Stack Change	59
8.1. Learning objective	59
8.2. Treat proposal as a review boundary	59
8.3. Check whether exploration is ready to become a proposal	60
8.4. Name two changes around one capability	60
8.5. Draft the backend change first	61
8.5.1. Read proposal.md for scope	62
8.5.2. Read the backend delta for behavior	63
8.5.3. Read design.md for consequential choices	63
8.6. Draft the frontend change from the contract boundary	64
8.7. Turn the handoff into ordered tasks	65
8.8. Inspect the generated changes with OpenSpec	65
8.9. Review both changes as one capability	66
8.10. Hands-on exercise	67
8.11. Verification checkpoint	67
8.12. What you learned	68
8.13. Online references	68

III. Part III: Applying and Evolving the Specification	69
9. Applying the Spring Boot Change	71
9.1. Learning objective	71
9.2. Move from artifacts to code	71
9.3. Check readiness before changing code	72
9.4. Start a bounded apply session	72
9.5. Follow a scenario through the backend	73
9.5.1. Keep validation at the API boundary	73
9.5.2. Preserve controller and error semantics	74
9.5.3. Add service and persistence behavior narrowly	74
9.6. Use the test result to control progress	74
9.7. Know when to return to the artifacts	75
9.8. Generate and inspect the backend contract	75
9.9. Resume without relying on chat memory	76
9.10. Hands-on exercise	76
9.11. Verification checkpoint	77
9.12. What you learned	77
9.13. Online references	77
10. Applying the Angular Change	79
10.1. Learning objective	79
10.2. Enter apply through the frontend change	79
10.3. Check the handoff before changing Angular code	80
10.4. Confirm that the contract is ready to consume	80
10.5. Start a bounded apply session	81
10.6. Generate the Angular client without creating a second contract	82
10.7. Follow one scenario through Angular	82
10.7.1. Bind input behavior to the scenario	83
10.7.2. Make request transitions visible	83
10.7.3. Keep transport details behind the generated boundary	83
10.8. Let focused tests control task progress	84
10.9. Return contradictions to their owners	84
10.10. Resume from artifacts and unchecked tasks	85
10.11. Hands-on exercise	85
10.12. Verification checkpoint	86
10.13. What you learned	86
10.14. Online references	86
11. Verifying, Synchronizing, and Archiving	87
11.1. Learning objective	87
11.2. Treat completion as a claim	87
11.3. Reestablish the change boundary	88
11.4. Verify the implementation against the artifacts	88
11.4.1. Build a scenario-to-evidence map	89
11.4.2. Follow the contract across repositories	90
11.5. Decide whether the change is ready to close	90
11.6. Understand synchronization during archive	91
11.7. Inspect what archival preserved	91
11.8. Respond to failed checks without hiding them	92

11.9. Run the fresh-session test	92
11.10 Hands-on exercise	93
11.11 Verification checkpoint	93
11.12 What you learned	94
11.13 Online references	94
12. Adding VIN Search to a Brownfield System	95
12.1. Learning objective	95
12.2. Why brownfield work changes the question	95
12.3. Establish the baseline before describing the delta	96
12.4. Decide whether VIN search is additive	96
12.5. Explore the extension points	97
12.6. Coordinate one product change across two repositories	98
12.7. Write deltas that expose the additive boundary	99
12.8. Design the contract handoff	100
12.9. Apply the change in dependency order	101
12.10 Verify both the addition and the preservation claim	101
12.11 Hands-on exercise	102
12.12 Verification checkpoint	102
12.13 What you learned	103
12.14 Online references	103
 IV. Part IV: Change Under Pressure	 105
13. Modifying Accepted Behavior	107
13.1. Learning objective	107
13.2. Recognize a replacement	107
13.3. Recover the accepted baseline	108
13.4. Separate the request from its unanswered questions	109
13.5. Write the proposal around the behavioral change	109
13.6. Replace the complete requirement	110
13.7. Inspect the delta before implementation	112
13.8. Trace one rule through two repositories	112
13.9. Apply in an order that preserves evidence	113
13.10 Verify the replacement before archive	113
13.11 Hands-on exercise	114
13.12 Verification checkpoint	114
13.13 What you learned	114
13.14 Online references	114
 14. Revising an In-Flight Change	 117
14.1. Learning objective	117
14.2. Treat the active change as the current plan	117
14.3. Recover the interruption point	118
14.4. Name the decision that moved	118
14.5. Decide whether to revise or split the change	119
14.6. Revise the artifacts as a set	120
14.6.1. Revise the proposal boundary	120
14.6.2. Rewrite the proposed requirement completely	121
14.6.3. Revisit design decisions	122

14.6.4. Reassess every task checkbox	123
14.7. Check coherence before resuming implementation	123
14.8. Reconcile partial implementation with the revision	124
14.9. Verify the revised behavior	124
14.10 Hands-on exercise	125
14.11 Verification checkpoint	125
14.12 What you learned	126
14.13 Online references	126
15. Coordinating Parallel Changes	127
15.1. Learning objective	127
15.2. Separate isolation from coordination	127
15.3. Establish one comparison point	128
15.4. Read each delta in two directions	128
15.5. Ask Copilot to report overlap, not resolve it	129
15.6. Recognize the forms of semantic overlap	130
15.7. Choose a coordination response	130
15.8. Coordinate the backend-owned contract	131
15.9. Use worktrees without confusing their role	131
15.10 Reconcile before synchronization or archive	132
15.11 Hands-on exercise	132
15.12 Verification checkpoint	133
15.13 What you learned	133
15.14 Online references	133
16. Making SDD a Team Practice	135
16.1. Learning objective	135
16.2. Start with the handoff problem	135
16.3. Agree on the minimum operating model	136
16.4. Give each system one job	136
16.5. Walk one change through the team	137
16.5.1. Frame the proposal together	137
16.5.2. Review behavior before design hardens	137
16.5.3. Turn tasks into handoff points	138
16.5.4. Preserve the review result	138
16.6. Match review ownership to the decision	138
16.7. Scale the workflow to the risk	139
16.8. Make Copilot work from reviewed context	139
16.9. Introduce the practice in stages	140
16.10 Capstone: establish a team workflow	140
16.11 Verification checkpoint	141
16.12 What you learned	141
16.13 Online references	141
17. Summary	143
References	145
OpenSpec and workflow	145
Petclinic and technology	145
AI-assisted development	145

Appendices	147
A. Environment Setup and Troubleshooting	147
A.1. Installing and Updating OpenSpec	147
A.1.1. Scope	147
A.1.2. Before installation	147
A.1.3. Install OpenSpec 1.13.1	147
A.1.4. Inspect the installed version	147
A.1.5. Upgrade OpenSpec	147
A.1.6. Update an existing repository	148
A.1.7. Verification checklist	148
A.1.8. Online references	148
A.2. Lab Setup	148
A.2.1. Required tools	148
A.2.2. Prepare the exercise repositories	148
A.2.3. Preflight check	149
A.3. Troubleshooting OpenSpec	149
A.3.1. A diagnostic method	149
A.3.2. Installation failures	149
A.3.3. Initialization and update failures	150
A.3.4. Command discovery failures	150
A.3.5. Artifact and validation failures	150
A.3.6. Two-repository failures	150
A.3.7. Recovery boundaries	150
A.3.8. Online references	151
B. OpenSpec Reference	153
B.1. OpenSpec CLI Command Reference	153
B.1.1. Typical workflow	153
B.1.2. Core commands	153
B.1.3. Store and workset commands	154
B.2. GitHub Copilot Commands and OpenSpec Skills	156
B.2.1. Using the matrix	157
B.3. OpenSpec schema.yaml and Template Reference	158
B.3.1. Schema fields	158
B.3.2. Template structure	160
B.4. Further Reading and Community Resources	161
B.4.1. Official documentation	162
B.4.2. Community schemas and extensions	162
C. Workshop Resources	163
C.1. Prompt Workbook	163
C.1.1. Explore unit-number search	163
C.1.2. Propose unit-number search	163
C.1.3. Add VIN search	163
C.1.4. Modify unit-number validation	163
C.1.5. Update an active filter proposal	163
C.1.6. Review generated artifacts	164
C.1.7. Verify before archive	164

C.2. Artifact Review Checklists	164
C.2.1. Proposal	164
C.2.2. Requirements and scenarios	164
C.2.3. Design	164
C.2.4. Tasks	164
C.2.5. Apply review	165
C.2.6. Archive readiness	165
C.3. Instructor Guide	165
C.3.1. Course shape	165
C.3.2. Teaching checkpoints	165
C.3.3. Expected friction	166
C.3.4. Recovery points	166
C.3.5. Closing discussion	166

Preface

AI-assisted development with tools like GitHub Copilot can be easy to start and difficult to control. A prompt can produce a plausible implementation in minutes, but plausibility is not the same as correctness. The hard questions remain: What are we building? Which parts of the existing system matter? How will we know that the result matches the intent?

This book is for software engineers, technical leads, and architects who want better answers to those questions without turning every change into a heavyweight ceremony. It introduces Spec-Driven Development (SDD) with OpenSpec as a practical way to make intent explicit, keep requirements close to implementation, and give GitHub Copilot the project context it needs. The examples move from greenfield work to brownfield systems, then through the less glamorous parts of real delivery: drift, verification, archiving, governance, and troubleshooting.

The approach has costs. Writing a useful spec takes time, and a vague or oversized spec can slow a team down instead of helping it. OpenSpec does not remove judgment from engineering, and Copilot can still misunderstand both the code and the specification. The goal is not to add paperwork for its own sake. It is to create a small, inspectable record of intent that helps people and tools make fewer expensive assumptions.

This book presents Spec-Driven Development (SDD) as a practical response to that continuity problem. It uses OpenSpec to keep requirements, proposed changes, design decisions, and implementation tasks in the repository where developers and agents can review them together.

The running example is a commercial truck search application. Its requirements change over three 90-minute mornings: the team first searches by a 15-character unit number, then adds VIN search, then shortens the unit-number limit and revises an active filter proposal. The changes are small enough to follow, but they expose the coordination problems that appear in larger systems.

This is not an argument for writing a specification before every code edit. Small, local, reversible changes may not justify the overhead. The workflow earns its cost when behavior crosses layers, requirements are likely to change, or several people and agent sessions need the same account of what the system should do.

Each part mirrors one morning of the training class. You can read the book independently or use its exercises as the class lab guide.

Part I.

Part I: Why Durable Intent Matters

1. From Chat Sessions to Durable Intent

1.1. A request that looks complete

Suppose a Jira story asks a developer to add exact search by a 15-character truck unit number across a Spring Boot backend and an Angular frontend. The developer gives the story to Copilot and asks it to implement the change.

Copilot can generate plausible code, but the request leaves basic behavior unsettled. It does not say whether matching is case-sensitive, which characters are valid, what the API returns when no unit matches, or how the interface reports invalid input. The agent may need to stop and ask questions or make choices that look reasonable. If those choices remain in the chat, the pull request may contain working code without a reviewable account of the behavior the team accepted.

A later developer inherits the code and tests, but not necessarily the reasoning behind them. A new Copilot session faces the same limit: it can inspect what the first session built, but it cannot reliably distinguish an intentional business rule from an assumption that survived implementation.

1.2. The same request with OpenSpec

In a spec-driven workflow, the team records unsettled behavior before asking Copilot to implement it. An OpenSpec proposal states the goal and scope, while requirements and scenarios describe observable behavior. Technical choices that affect the implementation may belong in a design, and a task list connects that intent to the work.

These artifacts can give reviewers a stable target before the code exists. Later Copilot sessions can draw on accepted decisions in the repository instead of the full history of a conversation. The team can then check code and tests against the specification rather than whichever interpretation an agent chose.

This approach does not make the decisions correct or remove the need for engineering judgment. It changes where the decisions live, when the team can review them, and how readily another person can recover them after the original chat session ends.

1.3. What this book will teach you to do

By working through the book, you will learn how to:

- decide when a change warrants a specification and when a direct edit is enough;
- turn an incomplete request into testable requirements and scenarios;
- place rationale, behavior, design decisions, and tasks in the appropriate OpenSpec artifacts;
- give Copilot focused repository context across sessions and repositories;
- compare code and tests with accepted behavior rather than with the latest chat response;

- modify accepted requirements, revise work already in progress, and coordinate parallel changes without discarding their history; and
- synchronize and archive completed changes so another developer can reconstruct what changed and why.

This workflow can expose ambiguity before code generation, give reviewers an explicit behavioral target, and preserve decisions beyond one Copilot session. OpenSpec does not guarantee better code. It can make intent reviewable and give the team something concrete to verify the implementation against.

1.4. The running example

The book builds a truck search capability for a fictional application developed by a fictional company. The example is small enough to keep the focus on the workflow, but broad enough to cover greenfield and brownfield development, changes to accepted specifications, and revisions to work already in progress.

- The backend uses Java 21 and Spring Boot 4.x.
- The frontend uses Angular 21 and TypeScript.
- The backend and frontend live in separate Bitbucket repositories, each with its own OpenSpec history.
- The first request asks for search by a 15-character truck unit number but leaves several behaviors unsettled.

The missing decisions make the coordination problem concrete: matching rules, invalid input, no-match behavior, response shape, and the boundary between the two repositories. Later chapters add VIN search, change accepted unit-number behavior, revise an active change, and coordinate parallel work. Each step tests whether the specification can connect the team's previous intent to its next decision.

1.5. Learning objective

By the end of this chapter, you should be able to explain the coordination problem that spec-driven development addresses and decide when a change might warrant a specification.

1.6. The coordination problem

An agent's ability to generate an implementation does not mean the team has agreed about the behavior. Tests provide evidence about the implementation, but only against the expectations they encode. A generated test may repeat the same unsupported assumption as the generated production code.

Separate artifacts can make those concerns easier to review. The specification records what the team has agreed to build. The implementation attempts to satisfy that agreement, while tests and other verification provide evidence about whether it does. With those roles distinct, a reviewer may be better able to identify whether a disagreement belongs in the requirement, the code, or the evidence.

1.7. Jira, chat, and repository artifacts

Each artifact has a different job. Jira tracks the request, ownership, priority, and delivery status. OpenSpec records proposed behavior, changes to accepted requirements, design decisions, and implementation tasks. Code implements those decisions, while tests and review produce evidence about the result.

A chat transcript can help during exploration, but it may mix questions, discarded alternatives, and accepted decisions. When the transcript becomes the specification, a later reader must reconstruct which statements still apply. Issue identifiers and links can connect the artifacts without requiring Jira API automation in this book.

1.8. When a specification earns its cost

Before choosing the workflow, consider change risk, ambiguity, coordination cost, and reversibility. A full specification may make sense when behavior crosses the backend and frontend, requirements may change, or multiple developers and agent sessions need the same account of intent. A smaller record or direct edit may be enough when the change is local, obvious, and easy to reverse.

A specification also has a cost. The team needs to review it, update it when decisions change, and keep it consistent with the implemented behavior. Without that maintenance, OpenSpec can become ceremony or stale documentation. The aim is to record enough intent for the risk and coordination demands of the change, rather than to maximize the number of artifacts.

1.9. Hands-on exercise

Review the unit-number Jira story and list every behavior GitHub Copilot would otherwise have to guess. Classify each item as one of the following:

- a business requirement;
- an implementation decision;
- a test or verification question; or
- an explicit non-goal.

1.10. Verification checkpoint

Produce a short decision record containing:

- unresolved questions that block implementation;
- assumptions the team has accepted;
- behaviors outside the first change; and
- an SDD or no-SDD decision with its reasons.

Give the record to another developer without the original Copilot conversation. Ask them to identify what remains undecided. If they cannot, revise the record.

The next chapter turns this decision record into OpenSpec artifacts.

1.11. Online references

- [OpenSpec documentation](#)
- [GitHub Copilot documentation](#)

2. The OpenSpec Operating Model

Open an OpenSpec repository during an active change and you may find two accounts of the product. One records the behavior the team accepts now. The other proposes a different state and records the reasoning and work needed to reach it.

Both accounts need to remain visible. If a draft requirement immediately replaced an accepted one, another developer or agent could mistake a proposal for shipped behavior. If the proposal lived only in chat, reviewers could lose its scope or struggle to tell which decisions survived discussion. OpenSpec keeps accepted specifications and active changes in separate directories, then brings them together when the team archives completed work, as shown in the OpenSpec quickstart¹.

This separation is the center of the operating model. The rest of the workflow helps a team answer four questions:

1. What behavior does the system currently promise?
2. What behavior would this change add, modify, remove, or rename?
3. How does the team plan to implement and verify that difference?
4. When has the proposed behavior earned a place in the accepted specifications?

2.1. Learning objective

By the end of this chapter, you should be able to read an OpenSpec repository as a map of accepted behavior and proposed changes. You should also be able to place rationale, requirements, design decisions, and tasks in the artifacts where reviewers can evaluate them.

2.2. Read the tree as two kinds of truth

Start with the two directories that carry most of the workflow:

```
openspec/  
|-- specs/  
|   |-- <capability>/  
|       |-- spec.md  
|-- changes/  
|   |-- <active-change>/  
|       |-- proposal.md  
|       |-- specs/  
|           |-- <capability>/  
|               |-- spec.md  
|       |-- design.md
```

¹<https://openspec.dev/docs/quickstart>

```
|  \-- tasks.md
\-- archive/
    \-- <completed-change>/
```

Ask a different question of each location:

- In openspec/specs/, ask, “What behavior has the team accepted as the current baseline?”
- In openspec/changes/\TU\textless{ }active-change\TU\textgreater{ }/, ask, “What difference is under consideration or implementation?”
- In openspec/changes/archive/, ask, “Which completed change explains how the baseline reached its current form?”

Suppose the accepted specifications say nothing about searching by truck unit number. A new folder named add-unit-number-search can describe that proposed capability without making it part of the baseline. During review and implementation, the main specs and the change may intentionally disagree because the change contains a delta, not a second complete copy of the product specification.

That temporary disagreement gives reviewers a useful comparison: would applying this delta produce the intended behavior? After the implementation passes review and verification, the archive operation can merge the delta into openspec/specs/ and move the change folder under openspec/changes/archive/.

i A repository is not proof

Treat the tree as a record of claims and decisions, not as proof that the software behaves as written. Tests, review, and runtime checks still need to supply evidence. OpenSpec gives that evidence a stated target.

2.3. Follow one change through its artifacts

The default spec-driven schema uses four planning artifacts. Their dependency order follows the decisions a team needs to make: proposal first; specs and design after the proposal; tasks after both; implementation after the task list. The default spec-driven schema reference² documents these roles and the conditions under which specs or design may be skipped.

```

                                +-- delta specs --+
proposal.md -----+                +-- tasks.md -- apply
                                +-- design.md -----+
```

The dependency order exposes omissions. If the task list requires an API decision that does not appear in the design, pause before coding. If a design choice changes user-visible behavior but no requirement records it, return to the delta spec. If a delta introduces a capability that the proposal never names, revisit the proposal’s scope.

2.3.1. Proposal: why this change, and how far does it reach?

Use proposal.md to frame the decision. For unit-number search, it might identify the need for exact search, name the affected capability, describe the impact on the backend and frontend, and exclude partial matching from the first change.

²<https://openspec.dev/docs/schemas/spec-driven>

A controller class, SQL predicate, or Angular component probably does not belong here. Those details pull the proposal toward implementation before reviewers have agreed on the problem and scope.

A useful proposal should let a reviewer challenge the change at low cost:

- Does this solve the right problem?
- Is the proposed capability too broad or too narrow?
- Which existing capabilities would change?
- What would remain outside the change?

2.3.2. Delta spec: what observable behavior would change?

Place behavioral differences under the change's specs/ directory, with one spec.md for each capability named by the proposal. OpenSpec delta sections distinguish ADDED, MODIFIED, REMOVED, and RENAMED requirements. Requirements and scenarios describe what a user or another system could observe, not the code structure that might produce it.

For the running example, a delta could say that a valid 15-character unit number returns exact matches. Its scenarios could cover a match, no match, and invalid input. The delta need not name a Spring repository method or an Angular form control unless that name is part of an external contract.

When a statement feels misplaced, use a replacement test: could the team replace Spring Data with another persistence mechanism while preserving the statement? If so, the statement may describe behavior and belong in the spec. If the replacement would make the statement irrelevant while behavior stayed the same, move it to the design or task list.

2.3.3. Design: how might the implementation satisfy the behavior?

Use design.md when the technical approach deserves review before coding. Cross-repository contracts, a new dependency, data-model changes, security constraints, migrations, or a consequential trade-off may justify it. A small change with an obvious implementation may not.

For unit-number search, the design might choose an endpoint shape, assign validation responsibilities between Angular and Spring Boot, or record how both repositories will keep response semantics consistent. It may also preserve rejected alternatives and their costs so that a later agent does not reopen a settled choice without new evidence.

The design may explain how to satisfy the proposed behavior, but it should not quietly create more behavior. If the design makes unit numbers case-insensitive, users or clients can observe that choice, so the delta spec needs the rule too.

2.3.4. Tasks: what work remains, and in what order?

Turn the reviewed behavior and approach into checkable work in tasks.md. The apply phase tracks Markdown checkboxes, so each task should name an outcome that an agent can complete and a reviewer can inspect.

For example:

```
## 1. Backend search contract
```

- [] 1.1 Add request validation for the accepted unit-number rules.
- [] 1.2 Implement exact-match search and response mapping.
- [] 1.3 Add tests for match, no-match, and invalid-input scenarios.

A checked box records claimed progress, not verification evidence. A test result, review, or runtime observation must support the claim.

2.4. Move through the loop with review points

The OpenSpec quickstart presents a five-step loop: explore, propose, review, apply, and archive. Optional skills may add explicit update, sync, and verify operations, but the same review points remain useful. The OpenSpec skills reference³ distinguishes the core skills from optional profile-installed skills and states what each skill may create or change.

2.4.1. Explore without pretending the plan is settled

Use exploration to inspect the codebase, find existing behavior, compare options, and surface unanswered questions. In the unit-search change, exploration might reveal that the backend already normalizes identifiers while the frontend validates only length. That finding could change the proposal, but exploration need not create artifacts or code.

Stay in exploration while an unanswered question could change scope or behavior. Move to proposal once the team can state the problem and the affected capabilities without asking the agent to fill gaps silently.

2.4.2. Propose, then review the artifacts

Proposal turns the idea into reviewable files. Read them before implementation in an order that follows their dependencies:

1. Read proposal.md for purpose, scope, and affected capabilities.
2. Read the delta specs for observable behavior and edge cases.
3. Read design.md, when present, for technical decisions and trade-offs.
4. Read tasks.md for coverage, dependency order, and stopping points.

When review finds a defect, change the artifact that owns the decision and then inspect its dependents. A revised requirement may require a design change; a revised design may require different tasks. Editing only the task list could leave Copilot with contradictory instructions.

2.4.3. Apply from the plan, but allow the plan to be wrong

During apply, the agent reads the change artifacts, implements pending tasks, and marks completed checkboxes. This supports work across chat sessions because progress and intent remain in the repository instead of one context window.

³<https://openspec.dev/docs/skills>

Do not force implementation through a contradiction. If the codebase disproves a design assumption or exposes missing behavior, stop and revise the affected artifact. An active change can evolve; its first draft does not have to survive contact with the code.

2.4.4. Verify against the change, not only the task list

Before archive, compare the delta's required behavior with the implementation and the evidence from tests or inspection. A complete task list with an unimplemented scenario should block archive. So should passing tests that encode behavior the team never accepted.

Verification may use OpenSpec's optional verification skill, project test commands, manual checks, or a combination suited to the risk. Whatever the mechanism, check that each requirement has evidence and that the implementation did not introduce behavior outside the reviewed scope.

2.4.5. Sync or archive when the baseline should change

Use sync when accepted specifications should absorb a delta while the change remains active. Use archive when the change is complete and its folder can move into completed history. Archive can synchronize outstanding deltas before moving the folder. Warnings about incomplete artifacts or tasks should prompt review rather than automatic dismissal.

After archive, a new session should find the current unit-search behavior in the main specification. The archived change should retain the reason, alternatives, and implementation path behind that behavior.

Git still governs collaboration. OpenSpec artifacts are ordinary repository files, subject to branches, pull requests, code review, and commit history. Each team must decide whether archive occurs before or after merge based on its release and review practice.

2.5. Distinguish the interface from the operation

The same workflow may appear through several interfaces. A terminal command such as `openspec\validate` runs the OpenSpec CLI. A prompt such as `/openspec-propose` invokes an agent skill in the chat host. Some hosts expose shorter aliases, and optional profiles may install more skills.

Before copying a command, ask:

- Does this text belong in the terminal or in Copilot Chat?
- Has the repository been initialized with the expected schema and profile?
- Does this host use the documented skill name or a host-specific alias?
- Will the operation write planning artifacts, application code, accepted specs, or archive history?

The last question sets a practical guardrail. Explore and verify normally inspect and report. Propose writes planning artifacts. Apply writes code and updates task progress. Sync changes accepted specs. Archive changes accepted specs when needed and relocates the completed change. Confirm the expected write boundary before approving an agent action.

2.6. Hands-on exercise

Classify each statement from the unit-number search change, then name the file that should own it:

1. Fleet coordinators need to find a truck from the identifier printed on a work order.
2. A valid unit number contains exactly 15 characters.
3. The Angular client will reject invalid length before sending a request.
4. Add controller tests for valid, invalid, match, and no-match cases.
5. The backend and frontend changes passed their focused test suites.

Some statements need revision before they fit. Statement 1 suggests rationale for `proposal.md`. Statement 2 describes observable behavior for a delta spec. Statement 3 records an implementation decision for `design.md`. Statement 4 belongs in `tasks.md`. Statement 5 supplies verification evidence, which should be recorded with the team's change or pull-request evidence rather than rewritten as a requirement.

Now move statement 2 into `design.md` and consider the consequence. A reviewer could no longer approve the behavior without reading the implementation plan, and a later design change could erase a validation rule that users still depend on. Artifact boundaries are review boundaries, not filing conventions.

2.7. Verification checkpoint

Given an unfamiliar OpenSpec tree, try to answer these questions without reading its application code:

- Which behavior represents the accepted baseline?
- Which changes remain active, and which capabilities does each touch?
- Where would you challenge scope, behavior, technical approach, and work coverage?
- Which artifact dependencies remain incomplete?
- What evidence would you request before archive?
- After archive, where would you look for current behavior and where for its history?

If the tree supports those answers, the operating model is doing useful work. If it does not, inspect whether information is missing, duplicated, or stored in an artifact that asks reviewers to make the wrong decision.

2.8. What you learned

Read `openspec/specs/` as the accepted behavioral baseline and `openspec/changes/` as proposed differences from it. Within a change, the proposal frames purpose and scope, delta specs define observable behavior, the design records consequential implementation choices, and tasks expose the work still to do.

Treat explore, review, apply, verification, sync, and archive as points where the team can inspect or correct the change. The next chapter examines the artifact that most directly constrains an agent's guesses: testable requirements and scenarios.

2.9. Online references

- [OpenSpec quickstart](#)
- [OpenSpec spec-driven schema](#)
- [OpenSpec CLI reference](#)
- [OpenSpec skills reference](#)

3. Writing Requirements an Agent Can Test

Imagine giving Copilot this Jira story:

Let users search for a truck by its 15-character unit number.

What could the agent reasonably infer? It might accept only letters and digits, trim spaces, ignore case, return one truck, return several trucks, or treat no match as an error. Each choice could lead to coherent code and green tests. None of those tests could tell the team whether the agent chose the intended behavior.

Use OpenSpec to expose those choices before they disappear into implementation. Record reviewed behavior in a requirement, make that behavior observable through scenarios, and use a delta to show how the proposal would change the accepted specification. Then ask the agent to work from that bounded target rather than from the Jira sentence alone.

3.1. Learning objective

Write and maintain OpenSpec requirements whose scenarios could guide implementation, review, and verification without prescribing REST endpoints, database queries, or Angular components.

3.2. Start with the decisions hidden in the story

Before asking an agent to draft a specification, mark every phrase that permits more than one reasonable interpretation. For the running example, try questions such as these:

- Does 15-character count leading or trailing spaces?
- Which characters may appear?
- Does exact include case?
- Could more than one truck share the same unit number?
- How should the system distinguish missing input, invalid input, and no match?
- Which result fields may the user or client observe?

Do not turn unanswered questions into confident requirements. Record an owner or a deliberate deferral instead. An agent can draft alternatives, inspect existing behavior, and find related tests, but the team still has to choose product behavior.

A small decision table can keep that boundary visible:

Question	Status	Decision or next action
Required length?	Settled	Exactly 15 characters
Match rule?	Settled	Exact match
Allowed character set?	Open	Confirm with the system owner
No-match response?	Open	Reconcile API and UI expectations

Question	Status	Decision or next action
----------	--------	-------------------------

Do not implement from this table yet. The open rows could still change validation, response handling, and tests, so preserve them as questions until someone with authority resolves them.

 A testable sentence can still encode a guess

“The system SHALL return 404 when no truck matches” can be tested, but testability does not make the choice correct. Ask where the behavior came from before asking whether a test could prove it.

3.3. Write the requirement as a behavioral contract

Once the decisions have owners and answers, write the proposed difference in the active change’s capability spec. For a change named add-unit-number-search, the file might live here:

```
openspec/changes/add-unit-number-search/
`-- specs/
    |-- vehicle-search/
        -- spec.md
```

OpenSpec delta specs group requirements under ADDED, MODIFIED, REMOVED, or RENAMED sections. Each requirement needs at least one `##\Scenario:` block for validation to accept its structure, as described by the OpenSpec CLI reference¹.

Suppose an agent converts only the two settled rows into a delta. Its first draft might pass a structural check while still exposing gaps for review:

```
## ADDED Requirements

### Requirement: Search by unit number

The system SHALL allow a user to search for trucks by an exact 15-character
unit number.

#### Scenario: Matching unit number

- **GIVEN** a truck exists with unit number `TRUCK-000001234`
- **WHEN** the user searches for `TRUCK-000001234`
- **THEN** the system returns that truck
```

Read the requirement as a contract rather than a design sketch. Test its boundary by replacing a Spring Boot controller with another delivery mechanism or changing the persistence layer. Keep the wording in the spec when those substitutions would leave the promised behavior unchanged.

Next, test the wording against the decision table. The word exact still leaves case handling unclear, and 15-character still leaves whitespace handling unclear. The missing no-match scenario correctly reflects an

¹<https://openspec.dev/docs/cli>

open decision, but it also blocks implementation of that path. Treat the draft as structurally recognizable and behaviorally incomplete.

3.4. Give each scenario one job

Treat a scenario as a small experiment against the requirement. Establish only the state needed for the behavior, perform one action, and name an observable result. Avoid turning the scenario into a transcript of controller calls, SQL execution, or component state changes.

Use each clause to answer a different question:

Clause	Ask	Avoid
GIVEN	What state or input matters before the action?	Setup unrelated to the behavior
WHEN	What does the user or system do?	A chain of implementation steps
THEN	What could a test or reviewer observe?	Private methods or internal data structures

Compare these two scenarios:

```
#### Scenario: Search executes the repository query

- **GIVEN** the controller receives a unit-number parameter
- **WHEN** the service calls `findByUnitNumber`
- **THEN** the repository executes an equality predicate


#### Scenario: Matching unit number

- **GIVEN** a truck exists with unit number `TRUCK-000001234`
- **WHEN** the user searches for `TRUCK-000001234`
- **THEN** the system returns that truck
```

Prefer the second version when a repository rename or a move away from Spring Data should leave the behavior unchanged. It describes the result rather than the route taken to produce it. Put the endpoint shape, repository strategy, and Angular component boundaries in design.md when they need review.

3.4.1. Cover decisions, not every imaginable input

Build a scenario matrix from settled rules rather than from a generic checklist. For unit-number search, the matrix might begin like this:

Decision boundary	Example	Observable result	Ready?
Exactly 15 characters	TRUCK-000001234	Search proceeds	Yes
Fewer than 15 characters	TRUCK-00001234	Input is rejected	Only if rejection behavior is settled

Decision boundary	Example	Observable result	Ready?
More than 15 characters	TRUCK-0000001234	Input is rejected	Only if rejection behavior is settled
Existing exact value	Known unit number	Matching truck is returned	Yes
Unknown exact value	Unknown unit number	Agreed empty result appears	Only after the no-match decision

Combine rows that produce the same behavior for the same reason. Give a boundary its own scenario when it changes the contract or exposes a known risk. Leave framework checks and repeated implementation details to lower-level tests rather than promoting them to product requirements.

Avoid equating scenario count with confidence. A dozen examples cannot repair an undefined term, and one carefully chosen boundary may constrain an agent better than several happy-path variations.

3.5. Let the agent draft without letting it decide

Give Copilot the source decisions, the capability boundary, and a prohibition against filling gaps. A prompt for the first draft might look like this:

```
Draft the delta spec for add-unit-number-search from the attached decision table. Use ADDED requirements and Given/When/Then scenarios. Describe only observable behavior. Do not choose answers for open rows; list them after the draft as blockers. Do not name endpoints, classes, database queries, or Angular components.
```

Then review the output by tracing every normative word back to a source. Where did SHALL, exact, invalid, one, or all come from? If the source material does not support the word, remove it or return the question to its owner.

Try asking the agent to challenge its own draft:

```
For each requirement and scenario, identify:
1. a term with more than one plausible interpretation;
2. an observable result that a test could assert;
3. an implementation detail that should move to design.md; and
4. an assumption not supported by the decision table.
Do not rewrite the spec yet.
```

Do not treat that review as a replacement for a product owner, API owner, or developer. Use it to make ambiguities easier to see, then return each decision to the responsible person.

3.6. Validate structure, then review meaning

Run OpenSpec validation before handing the change to implementation:

```
openspec validate add-unit-number-search --strict
```

The validator can catch structural faults such as a requirement without a scenario. It cannot decide whether exact\match means case-sensitive matching, whether an unknown unit number should produce an empty result, or whether the scenario reflects the Jira request. Treat a clean command as evidence that OpenSpec could parse the artifact, not as approval of its product meaning.

Use show when the parsed change needs inspection:

```
openspec show add-unit-number-search --json --deltas-only
```

Compare the parsed output with every intended requirement and scenario. If a heading level or section name prevents OpenSpec from recognizing one, fix the Markdown structure before continuing. Then return to human review for ambiguity, scope, and unsupported assumptions.

3.7. Connect scenarios to implementation evidence

Before apply begins, create a lightweight trace from each scenario to the evidence you expect. The links may live in tasks.md, a pull-request checklist, or the team’s existing test plan; avoid duplicating the requirement text in all three places.

Specification item	Expected evidence	Status
Matching unit number scenario	Automated search result test	Ready for a backend test task
No-match behavior	None until the team decides the result	Blocked
Invalid-length boundary	No scenario defines the rejection yet	Return to the delta spec

Use this trace before implementation, not as a report written after the code. Resolve the blocked and missing rows in the artifact that owns each decision. During apply, ask the agent to report evidence against scenario names rather than merely reporting that “all tests pass.” Keep any scenario with no test or other observation visible as a verification gap.

3.8. Maintain the requirement when behavior changes

Suppose a reviewed change resolves the earlier questions and the accepted specification now covers exact 15-character matching and no-match behavior. Later, another change would allow leading and trailing spaces around user input. Avoid appending a second, partly contradictory requirement. Use MODIFIED to restate the complete changed requirement and every scenario that should survive.

```
## MODIFIED Requirements
```

```
### Requirement: Search by unit number
```

```
The system SHALL ignore leading and trailing spaces around a search value and match the remaining value against an exact 15-character unit number.
```

```
#### Scenario: Matching value surrounded by spaces

- **GIVEN** a truck exists with unit number `TRUCK-000001234`
- **WHEN** the user searches for ` TRUCK-000001234 `
- **THEN** the system returns that truck

#### Scenario: Matching unit number without surrounding spaces

- **GIVEN** a truck exists with unit number `TRUCK-000001234`
- **WHEN** the user searches for `TRUCK-000001234`
- **THEN** the system returns that truck

#### Scenario: No matching unit number

- **GIVEN** no truck has unit number `TRUCK-000009999`
- **WHEN** the user searches for `TRUCK-000009999`
- **THEN** the system returns no matching trucks
```

Restate the complete block because archive needs enough information to replace the accepted requirement without dropping scenarios that should remain. Match the existing requirement header exactly, then inspect the proposed difference with:

```
openspec show trim-unit-number-search --diff
```

Read that diff as a behavioral review. Confirm that the new wording changes only whitespace handling, that earlier scenarios remain, and that no implementation choice entered the contract. Resolve any discrepancy before archive updates the accepted spec.

Once the implementation and evidence satisfy the reviewed delta, archive can merge the change into openspec/specs/. The next agent can then read the current behavior without reconstructing it from old chats, Jira comments, or test names. Keep the archived change when the team needs the proposal, design discussion, and task history that explain how the requirement changed.

3.9. Hands-on exercise

Turn the unit-number Jira story into a reviewable OpenSpec delta:

1. Mark every ambiguous term in the story.
2. Create a decision table with Settled, Open, or Deferred for each question.
3. Draft one requirement using only settled decisions.
4. Add scenarios for the agreed match, no-match, and validation boundaries.
5. Ask an agent to find unsupported assumptions and implementation details.
6. Run `openspec\validate\TU\textless{ }change-name\TU\textgreater{ }}\V-strict`.
7. Map each scenario to expected test or runtime evidence.

Swap drafts with a partner. Could each of you derive the same observable behavior without agreeing on controller names, query methods, or component structure? Where your interpretations differ, revise the requirement or return the question to the decision table.

3.10. Verification checkpoint

Before calling the requirement ready, ask:

- Can every normative term be traced to an accepted decision?
- Could a reviewer observe each scenario's result without reading private implementation details?
- Does every requirement have at least one scenario that OpenSpec recognizes?
- Do the scenarios cover the boundaries the team chose, rather than boundaries the agent invented?
- Could the planned evidence fail when the specified behavior is wrong?
- Would a future MODIFIED delta replace the requirement without losing behavior that should remain?

If one answer remains uncertain, keep the uncertainty visible. A short blocker costs less than a precise implementation of the wrong rule.

3.11. What you learned

Begin with decisions, not normative prose. Turn settled behavior into a requirement, use scenarios as observable examples, and let OpenSpec expose the proposed difference from the accepted baseline. Validate the structure, review the meaning, and trace each scenario to evidence before apply.

When behavior changes, revise the requirement through a complete delta and inspect what archive would carry forward. That maintenance step gives later agents a current contract rather than a pile of tests whose original intent must be guessed.

3.12. Online references

- [OpenSpec quickstart](#)
- [OpenSpec CLI reference](#)
- [OpenSpec spec-driven schema](#)

4. Engineering Context for GitHub Copilot

Suppose you ask Copilot to add unit-number search to a Spring Boot service and its Angular client. You could attach the Jira story, both repositories, an architecture guide, several production logs, the OpenAPI document, and every test under src. Copilot would receive plenty of information, but the relevant constraints would be harder to find.

A smaller package might work better: the active OpenSpec change, the existing search endpoint, the generated-client boundary, and the commands that verify each repository. Let the agent search outward when those sources expose a gap. This approach treats context as a budget to allocate rather than a pile to maximize.

4.1. Learning objective

Build a bounded context package that preserves intent across a long-running change without crowding the selected model's context window. Decide what belongs in repository instructions, OpenSpec artifacts, a chat session, or a Markdown handoff, then carry that package between VS Code, IntelliJ IDEA, and GitHub Copilot CLI.

4.2. Context engineering in an SDD workflow

Context engineering asks a practical question: what information should the model receive for this decision, in what form, and for how long? Prompt wording still matters, but the prompt forms only one part of the request. Copilot may also receive system instructions, repository customizations, conversation history, editor state, explicit file references, tool definitions, and tool output. VS Code documents this assembly process in its context guide¹.

That larger definition fits SDD well. Instead of asking a chat transcript to remember the change, give each kind of information an owner:

Information	Suggested owner	Expected lifetime
Java, Angular, naming, and validation conventions	Repository or path-specific instructions	Many changes
Accepted business behavior	openspec/specs/	Until a later archived change modifies it
Proposed behavior and technical choices	Active OpenSpec proposal.md, delta specs, and design.md	Life of the change, then archive
Remaining implementation work	Active OpenSpec tasks.md	Life of the change
Reusable multi-step procedure	Agent Skill	Many sessions when invoked
Current hypothesis, recent evidence, and next action	Focused chat or reviewed handoff file	Hours or days

¹<https://code.visualstudio.com/docs/agents/concepts/context>

Put facts that a future developer will need in an accepted spec, architecture record, or repository instruction. Keep implementation-specific facts in the active change or handoff. Evidence from one failed-test investigation can remain in chat unless the investigation must survive a session boundary.

This separation may feel slower than pasting everything into one prompt. It usually asks for more judgment at the beginning. In return, each new session can recover intent from named artifacts rather than from a transcript whose useful parts have become hard to distinguish from abandoned ideas.

4.3. Treat the context window as shared space

A context window limits how much information a model can process in one request. Your current prompt does not get the whole window. System instructions, custom instructions, chat history, attached files, source snippets, tool schemas, terminal output, and space reserved for the response all compete for it.

More context can therefore make an answer worse. An obsolete design note might conflict with the accepted delta spec. A large log could push an earlier decision toward compaction. A folder attachment might bury the one validation rule that the implementation must preserve. Even relevant information has a cost when the model must repeatedly process it.

Evaluate each candidate source with four questions:

1. Does it affect the next decision or the requested output?
2. Is it more authoritative than another source already present?
3. Is it current, or could it describe behavior the team has replaced?
4. Could Copilot retrieve it later with a focused search instead of receiving it now?

Attach the source when the first three answers support it and retrieval would add avoidable uncertainty. Leave it discoverable when only a small part might become relevant. Remove or label it when its authority or freshness remains unclear.

 A large context window does not resolve contradictions

A model with room for both an old design and a new specification still has to choose between them. State which source wins, remove the stale source, or revise the artifacts before implementation.

4.3.1. Start with the smallest authoritative package

For the unit-number search change, an initial implementation request might include:

- The active OpenSpec proposal, delta spec, design, and tasks.
- The current controller or route that owns nearby search behavior.
- The OpenAPI contract and the frontend service that consumes the generated client.
- The narrow backend and frontend test commands.

Avoid attaching every controller, component, test, and generated client file. Ask the agent to inspect the named starting points and search for adjacent code before editing. VS Code's semantic index can find relevant snippets without sending the entire workspace to every model request, while IntelliJ IDEA and Copilot CLI can use their own file references and agent tools to expand the working set as evidence requires.

A bounded request might read:

```
Implement the next unchecked task in the active add-unit-number-search change.  
Treat its delta spec as the behavioral authority and design.md as the technical  
boundary. Start with the existing truck search endpoint and generated Angular  
client. Search for adjacent tests before editing. Do not change generated files.  
Run the narrow backend or frontend test named in tasks.md, then report the  
scenario covered by the result.
```

Notice what this prompt does not repeat. It points to durable context instead of copying the requirements, architecture rules, and task list into chat. A correction to an artifact can then affect the next request without leaving a second version embedded in the transcript.

4.4. Choose a model with the task and window in mind

The selected model affects more than tone or coding style. Models may differ in context-window size, reasoning behavior, latency, supported inputs, and token price. VS Code's context-window control reports the capacity available for the selected model, while Copilot CLI's `/model` picker may offer a default or long-context tier for models that support one. GitHub's model comparison² changes as models enter and leave Copilot, so avoid encoding a permanent list of model names or limits in repository instructions.

A routine one-file edit rarely needs the largest available window. A faster model with focused context may finish at lower cost. An architecture review that compares both repositories, a contract, and competing designs may justify stronger reasoning and a larger window. Compare the total work required, including retries, rather than price or window size in isolation.

Before beginning a long task, select the model, context tier, reasoning effort, and tools you expect to use. Changing them later may alter available capacity, price, response behavior, or prompt-cache reuse. In VS Code and IntelliJ IDEA, use the model picker in the chat panel. In Copilot CLI, use `/model`; the picker can expose long-context options where the selected model supports them.

Keep two cautions in view:

- A larger window may postpone compaction, but it may also invite an unfocused session and cost more per token.
- A stronger reasoning model may complete a hard task with fewer retries, while the same model may waste credits on mechanical edits.

Use representative work from your repository to compare quality, retries, latency, and credit use. Recheck those results as Copilot's model catalog and pricing change.

4.5. Expect long conversations to be compacted

Conversation history cannot grow inside a fixed model window forever. As a session approaches its limit, Copilot can replace older turns with a shorter summary. That process, called context compaction, frees room for new prompts and tool results. It does not preserve every earlier sentence.

VS Code automatically compacts a conversation when its context window fills. You can inspect usage through the context-window control or run `/compact` before the limit, with guidance such as:

²<https://docs.github.com/en/copilot/reference/ai-models/model-comparison>

```
/compact Retain the accepted API contract, rejected persistence approaches,  
failing test evidence, files changed, and the next unchecked OpenSpec task.
```

Copilot CLI also compacts automatically. Its current documentation describes background compaction as beginning near 80 percent of the token limit, with a dynamic threshold when static instructions and tool definitions already occupy much of the window. Use `/context` to inspect the window, `/compact` to request a focused summary, and `/session/checkpoints` to inspect summaries created during the session. See the Copilot CLI context guidance³.

Do not treat automatic compaction as lossless memory. A summary may omit the reason an approach was rejected, the exact output of a failing test, or a small constraint mentioned many turns earlier. The chance of omission grows when the session mixes unrelated work.

Protect the state that must survive before compaction occurs:

- Start a new session when the task changes.
- Compact manually at a stable checkpoint rather than waiting for a crowded window.
- Tell the compactor what must survive.
- Move accepted facts into the artifact that owns them.
- Save a reviewed handoff before closing or switching tools.

IntelliJ IDEA exposes chat history, file references, working sets, and plan handoff features, but its controls may not match VS Code or the CLI release for release. Keep durable context in repository files rather than depending on a client-specific transcript feature.

4.6. Carry the working state forward in Markdown

A chat summary belongs to the chat system. A Markdown handoff belongs to the project and can be reviewed, diffed, attached to another session, or read from another client. Use one when an investigation or implementation will cross a session, model, developer, or IDE boundary.

Ask Copilot to draft the handoff, then verify every line against the repository and the active change. A useful file might look like this:

```
# Unit-number search handoff  
  
Updated: 2026-09-21  
Change: `add-unit-number-search`  
  
## Current objective  
  
Complete the backend search task without changing the accepted API contract.  
  
## Sources of truth  
  
- Behavior: `openspec/changes/add-unit-number-search/specs/vehicle-search/spec.md`  
- Technical choices: `openspec/changes/add-unit-number-search/design.md`  
- Remaining work: `openspec/changes/add-unit-number-search/tasks.md`
```

³<https://docs.github.com/en/copilot/how-tos/copilot-cli/use-copilot-cli/overview#context-management>

Confirmed decisions

- Unit-number matching uses the rule stated in the delta spec.
- The backend owns the OpenAPI document.
- The Angular client remains generated code.

Evidence

- `TruckSearchControllerTest` passes for the existing search path.
- The new no-match test fails because the repository query is not implemented.

Rejected or superseded approaches

- Do not edit the generated TypeScript client by hand.
- Do not introduce a second search endpoint.

Next action

Implement the repository query, run the focused backend test, and update the matching checkbox in `tasks.md` only after the test passes.

Open questions

- None for the current task.

Keep this file short. Link to specifications and source files instead of duplicating them. Record evidence with enough detail to reproduce it, and distinguish confirmed decisions from hypotheses. Delete the handoff when it no longer helps, or update it when the next developer would otherwise follow stale instructions.

A team might store the file at docs/handoffs/TU\textless{ }change-name\TU\textgreater{ }.md, in a ticket-specific notes directory, or in another reviewed location outside generated output. Agree on one convention. If a handoff contains a lasting business rule, move that rule into the OpenSpec specification rather than allowing the handoff to become a shadow source of truth.

To start the next session, reference the handoff and the active change:

Read docs/handoffs/add-unit-number-search.md and the linked OpenSpec artifacts. Verify that the handoff still matches the working tree and current test results. Then perform only the Next action. Stop and report any contradiction before editing code.

That verification clause matters. A durable file can carry context forward, but durability also lets mistakes persist.

4.7. Use each Copilot client deliberately

The same repository artifacts can support all three clients, while the attachment and inspection controls differ.

4.7.1. VS Code

VS Code can infer context from the active editor, selection, diagnostics, Git state, and workspace index. Use # mentions or **Add Context** when a file, folder, symbol, issue, pull request, terminal result, or URL must be considered. Attach a known source explicitly when omission would change the result; let workspace search discover ordinary neighboring code.

The context-window control can show current token use and the selected model's capacity. Use /compact for a focused summary, /fork when exploring an alternative that should inherit the current history, and a new session for unrelated work. Check the response's references when you need to confirm which instructions and files informed it.

Repository-wide rules may live in .github/copilot-instructions.md or AGENTS.md. Path-specific .instructions.md files can apply only to Java, TypeScript, tests, or selected folders. Keep always-on instructions short because they enter many requests. VS Code's custom-instructions guide⁴ describes the current discovery rules.

Prompt files can still encode reusable requests for VS Code's local agent, but current VS Code documentation marks them as deprecated for Agent Host sessions and recommends migration to Agent Skills. Prefer a skill for a reusable, multi-step workflow that should load only when relevant. Avoid moving change-specific requirements into a skill merely to make them reusable.

4.7.2. IntelliJ IDEA and other JetBrains IDEs

Copilot Chat in IntelliJ IDEA can use the open file or selected code as implicit context. Drag a file into the prompt, or right-click it and choose **GitHub Copilot > Reference File in Chat**, when the file must be part of the request. In edit mode, keep the working set limited to files the change may modify.

Use plan mode for a cross-cutting change, review the plan, and choose **Open in Editor** when you want the plan as Markdown. Save a reviewed version in the repository when it should outlive the conversation. The Copilot Chat guide for JetBrains IDEs⁵ covers file references, edit working sets, plan mode, and model selection.

Repository instructions can give VS Code, JetBrains IDEs, Copilot CLI, and GitHub-hosted Copilot features a shared baseline, subject to each client's current support. Inspect the response references instead of assuming an instruction file loaded. Avoid placing client-specific button sequences in repository-wide coding rules.

4.7.3. GitHub Copilot CLI

Start Copilot CLI from the narrowest directory that owns the task. Add a file with @relative/path, inspect context use with /context, and check model and token statistics with /usage. Use /new when changing topics and /compact\TU\textless{ }focus\TU\textgreater{ } when the current task should continue with less history.

The CLI can resume sessions, but a resumed transcript still carries its earlier assumptions and compacted summaries. A checked-in handoff and OpenSpec change may provide a cleaner restart than an old conversation. Use /share\file when you need a Markdown export of the session, then distill the useful facts rather than treating the full transcript as project documentation.

Use /instructions to inspect discovered instruction files. Keep @file references focused, and consider subagents for isolated exploration because each subagent has a separate context window and can return a concise result

⁴<https://code.visualstudio.com/docs/agent-customization/custom-instructions>

⁵<https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide?tool=jetbrains>

to the main session. The Copilot CLI command reference⁶ documents these commands and the current model options.

4.8. Understand caching without confusing it with memory

Prompt caching may let a model provider reuse an unchanged prefix from a previous request. That prefix can include stable system instructions, tool definitions, custom instructions, and earlier conversation content. When supported by the provider and model, reuse may lower latency and the cost assigned to cached input tokens.

Caching does not enlarge the context window. It does not guarantee that Copilot will recall an omitted fact. It also does not replace compaction or a handoff file. The same tokens may remain part of the model request even when the provider can process them at a lower cached-input rate.

GitHub's current token-based pricing converts input, output, cache-read, and, for some models, cache-write tokens into GitHub AI Credits. The rates vary by model, and one AI credit represents \$0.01 USD under that billing model. Some annual plans may still use legacy request-based billing, so check the plan that applies to your account. See Models and pricing for GitHub Copilot⁷.

To improve cache reuse without weakening the request, choose the model, reasoning effort, context tier, instructions, and tool set before the session begins. Keep stable instructions stable. Avoid repeatedly replacing large attachments or changing the tool catalog between turns without a task-related reason. Forking an existing VS Code session may preserve the earlier prompt prefix better than reconstructing it in a new chat.

Do not optimize a cache-hit percentage at the expense of correctness. A changed specification should invalidate the old context. A required MCP tool should remain available even if its schema changes the prompt prefix. VS Code's Cache Explorer⁸ can show where consecutive requests first diverge, but the documentation cautions that this comparison does not prove how the provider processed or billed the request.

4.9. Reduce context waste before it reaches the model

Several small practices can keep the useful signal easier to recover:

- Give each session one change or investigation.
- Attach a precise file or symbol before attaching a folder or codebase.
- Exclude generated output, dependencies, build directories, and noisy logs from search when they should not be discovered.
- Keep tool catalogs narrow; every tool definition and result may consume context.
- Run deterministic bulk work in a script or build tool, then return the concise result instead of many repetitive tool calls.
- Use a subagent for independent research and ask for a sourced summary.
- Ask Copilot to name the files, instructions, assumptions, and test evidence behind a recommendation.
- Correct the owning artifact when a rule changes rather than repeating the correction in later prompts.
- Start a fresh session when the accumulated history costs more than rebuilding a small context package.

⁶<https://docs.github.com/en/copilot/reference/copilot-cli-reference/cli-command-reference>

⁷<https://docs.github.com/en/copilot/reference/copilot-billing/models-and-pricing>

⁸<https://code.visualstudio.com/docs/agents/agent-troubleshooting/cache-explorer>

MCP servers can supply current data from issue trackers, documentation systems, or internal services. Add one when the task needs information that the repository cannot contain or should not duplicate. Limit enabled servers and tools to the current job; a large catalog consumes context before any tool runs and gives the agent more choices to evaluate.

The same restraint applies to custom instructions. A linter can enforce formatting without spending context tokens on a prose rule. Source code can reveal an ordinary framework convention. Reserve instructions for non-obvious constraints, commands, boundaries, and exceptions that Copilot would otherwise have to rediscover or might infer incorrectly.

4.10. Hands-on exercise

Take one oversized prompt from your work or use the unit-number search example. Break it into these layers:

1. Move stable repository facts into `.github/copilot-instructions.md` or a path-specific instruction file.
2. Move accepted and proposed behavior into the appropriate OpenSpec artifacts.
3. Keep the current objective and evidence in a focused session.
4. Attach only the files needed to begin, then let the agent search for adjacent code.
5. Ask Copilot to draft a Markdown handoff using the template in this chapter.
6. Review the handoff against the working tree, test output, and active OpenSpec change.
7. Start a fresh session in another client and reference only the handoff and its linked artifacts.

Ask the new session to state the current objective, authoritative sources, next action, and unresolved questions before editing. If it cannot recover those facts without the old transcript, revise the durable artifacts or handoff rather than pasting the transcript into the new session.

For a second pass, compare a focused and an overloaded run of the same bounded task. Record the selected model, context use, tool calls, AI credits, retries, and verification result. One run will not establish a universal rule, but it may expose noisy sources or missing instructions in your repository.

4.11. Verification checkpoint

Before handing implementation to Copilot, ask:

- Does the session have one bounded objective and an observable completion condition?
- Can every attached source affect the next decision?
- Does each rule have one named owner, with stale or conflicting copies removed?
- Could the selected model hold the required context with room for tools and output?
- Have accepted decisions moved out of chat and into a durable artifact?
- Would automatic compaction be allowed to omit anything that has not been saved?
- Can another client recover the objective from the repository without the old transcript?
- Did you inspect references, tests, and usage rather than infer that context was applied?

An uncertain answer may call for a smaller task, a better artifact, or a new session. Adding the whole repository rarely resolves that uncertainty.

4.12. What you learned

Treat context as a bounded input to each model request. Keep stable rules in repository instructions, proposed behavior in OpenSpec, reusable procedures in skills, and short-lived evidence in a focused session. Before a session boundary, move verified working state into a reviewed Markdown handoff.

Model choice changes the available window, behavior, and price. Compaction summarizes rather than preserving every detail, and caching reduces processing cost rather than creating memory. Give Copilot the smallest current, authoritative package from which it can complete and verify the next task.

4.13. Online references

- VS Code context engineering guide
- VS Code context concepts
- VS Code session and compaction management
- VS Code AI credit optimization
- VS Code prompt-cache diagnostics
- GitHub Copilot Chat in JetBrains IDEs
- GitHub Copilot CLI best practices
- GitHub Copilot CLI command reference
- GitHub Copilot model comparison
- GitHub Copilot models and pricing
- GitHub Copilot repository custom instructions
- OpenSpec supported tools
- OpenSpec skills reference

Part II.

**Part II: Establishing the Greenfield
Baseline**

5. Preparing the Two-Repository Lab

The exercises in Part II use two prepared repositories: `commercial-truck-search-api` and `commercial-truck-search-ui`. Although they support one product, each repository will keep its own `openspec/` folder, accepted specifications, active changes, and archive.

A backend change and a frontend change may support the same user capability without making the same decisions. In this lab, the backend owns API behavior and the OpenAPI contract. The frontend owns the user's interaction with that contract and consumes a generated TypeScript Angular client. This arrangement keeps each specification beside the code that can enforce and test it.

5.1. Learning objective

By the end of this chapter, you should be able to explain why each repository has a separate OpenSpec project, trace a full-stack change across the contract boundary, and establish a working baseline for both repositories. Your baseline should record the tool versions, application ports, verification commands, and clean commits needed to repeat the later exercises.

5.2. One product, two specification histories

After initialization in the next chapter, the repositories should have parallel OpenSpec structures. A simplified view might look like this:

```
commercial-truck-search-api/  
|-- openspec/  
|   |-- specs/  
|   |-- changes/  
|   |-- archive/  
|-- src/  
|-- ...  
  
commercial-truck-search-ui/  
|-- openspec/  
|   |-- specs/  
|   |-- changes/  
|   |-- archive/  
|-- src/  
|-- ...
```

The matching folder names do not imply shared contents. In the backend repository, `openspec/specs/` may describe accepted API behavior such as valid search input, response shapes, and error semantics. In the

frontend repository, the same path may describe accepted user behavior such as form validation, loading feedback, result presentation, and empty or error states.

Each openspec/changes/ directory can hold work for its repository. A backend change may include tasks for a controller, service, persistence query, tests, and OpenAPI publication. Its frontend counterpart may include tasks for a form, generated-client integration, view states, and browser tests. Neither change needs to carry the other's implementation plan.

Two change folders can drift in names, assumptions, or timing. Explicit ownership and a recorded dependency can expose that drift without copying the same requirement into both repositories.

5.3. Let the contract cross the boundary

For the later unit-number search, the work might move through the repositories in this order:

1. Explore the user capability across both repositories.
2. Create linked backend and frontend OpenSpec changes.
3. Let the backend change define the observable API behavior and OpenAPI contract.
4. Let the frontend change define the user interaction and record its contract dependency.
5. Implement and test the backend, then publish the reviewed OpenAPI document.
6. Generate the TypeScript client, then implement and test the frontend behavior.

This order leaves one owner for the wire contract while allowing both proposals to be reviewed before implementation. The frontend specification can define the user experience and record its dependency without restating endpoint fields by hand. If the frontend needs a field or error response that the contract does not provide, the team can revise the backend change before client generation.

The relationship may be easier to remember as a dependency:

```
linked OpenSpec changes
|
v
backend implementation and reviewed OpenAPI contract
|
v
generated TypeScript Angular client
|
v
frontend implementation
```

Frontend exploration may expose a missing API decision before implementation begins. The team can then resolve that decision in the backend change and regenerate the client instead of editing generated TypeScript or creating a competing frontend definition.

5.4. Decide where a statement belongs

When a requirement touches both repositories, ask which codebase can verify and enforce it. The answer may prevent a shared idea from becoming two independent rules.

Statement	OpenSpec owner	Reason
Which unit-number inputs the API accepts	Backend	The API validates the request boundary
Which status and body represent no match	Backend	The OpenAPI contract exposes this behavior
When the search control becomes available	Frontend	The UI owns the interaction
What the user sees while a request runs	Frontend	The UI renders this state
Which response fields the generated client exposes	Backend	The backend publishes the schema
How response fields appear on the page	Frontend	The UI owns presentation

Some scenarios may need evidence in both repositories. Invalid input, for example, could have a backend scenario for API enforcement and a frontend scenario for immediate feedback. Each scenario can use the agreed meaning of valid input while describing behavior that its own repository can test.

Avoid a third source of truth

A cross-repository planning note can link the two changes and record their order. It should not become another specification of request fields, response codes, or UI behavior. Put each lasting rule in the OpenSpec project that owns it, and let the OpenAPI document carry the wire contract.

5.5. Start from a known baseline

Separate histories help only when you can tell what each change introduced. Before initializing OpenSpec or asking Copilot to implement a feature, establish a baseline in both repositories. A failed build at this point may indicate a toolchain or checkout problem. The same failure after a change may indicate a regression. Without the baseline, the two explanations can be difficult to separate.

5.6. Backend baseline

Inspect the backend before changing it. Check the versions and tools in use: Java 21, Spring Boot 4.1.1, Maven, Spring Data JPA, and H2. Locate the existing test conventions, application boundaries, and OpenAPI source. Also check whether truck search has been implemented. Later chapters use its absence as the starting condition for the first feature.

Record the command that builds the service, the command that runs its tests, and the port used when it starts. Capture the current commit ID and check the working tree. These details will give the backend OpenSpec tasks concrete verification targets in later chapters.

5.7. Frontend baseline

Inspect the frontend separately. Check for Angular 21 and Node.js 24.19.0, then locate the Vitest, component-test, Playwright, and generated-client boundaries. Record the local backend and frontend ports from the prepared repositories. The two applications may not use the same defaults in every environment.

Find how the generated client enters the application and where its generated files live. Treat that location as output rather than as the source of API decisions. Record the frontend build, unit-test, and browser-test commands, along with the current commit ID and working-tree state.

Keep the two baselines together in your lab notes, but record their commits and verification results separately. A passing frontend test cannot establish that the backend contract is correct. A passing backend test cannot show that the browser renders the intended state.

5.8. VS Code and IntelliJ IDEA setup

Open both repositories in VS Code or IntelliJ IDEA. Check that GitHub Copilot Chat is available in the IDE you plan to use. A multi-root workspace may make both repositories visible at once, while separate windows may make the active repository boundary harder to miss. Either arrangement can work if you check which repository owns the current chat context, terminal, and OpenSpec command.

Record which terminal, task runner, and project view you used for each command. The repository commands and files should remain the same even when each IDE exposes them differently. Keeping that difference visible may help separate an IDE integration problem from a repository problem.

5.9. Hands-on exercise

Clone `commercial-truck-search-api` and `commercial-truck-search-ui`, then work through each repository separately:

1. Confirm the expected tool versions.
2. Inspect the source, tests, and generated-client or OpenAPI boundary.
3. Run the repository's build and existing tests.
4. Start the application and confirm its local port.
5. Record the commit ID and verify that the working tree remains clean.

Next, sketch where the future unit-number search statements from the ownership table would belong. Do not create a proposal or implementation yet. The purpose of the sketch is to check whether you can identify the backend-owned contract decisions and the frontend-owned interaction decisions before OpenSpec creates either repository structure.

You can leave OpenSpec uninitialized in this chapter. Chapter 6 uses the clean baselines to show what initialization adds to each repository.

5.10. Verification checkpoint

Before moving on, check that your notes answer these questions:

- Which repository owns the OpenAPI contract?
- Which repository will generate and consume the TypeScript client?
- Where will accepted backend behavior and accepted frontend behavior live after initialization?
- Which command builds and tests each repository?
- Which ports and tool versions did you observe?
- Which clean commit can serve as a reset point in each repository?

Both repositories should build, their existing tests should pass, and their working trees should remain clean. If one check fails, record the failure as a baseline issue before an agent changes code. That record may keep an environment problem from being misread as an OpenSpec or implementation defect.

5.11. What you learned

The two repositories need coordination, but they do not need a shared implementation specification. The backend OpenSpec project can govern the wire contract, and the frontend OpenSpec project can govern the interaction built on it. The reviewed OpenAPI document connects them.

With separate clean baselines recorded, Chapter 6 can initialize OpenSpec once for the backend's constraints and once for the frontend's.

5.12. Online references

- Spring Boot system requirements
- Angular local environment setup
- OpenAPI Generator installation

6. Initialize and Configure OpenSpec

Running `openspec\init` takes little time. The harder part is deciding what the generated setup needs to teach an agent about the repository.

Initialization places the OpenSpec project beside the code it describes and installs workflow files for the selected AI tool. Configuration supplies the repository context used by those workflows. Keeping these steps separate distinguishes generated OpenSpec guidance from rules the team owns.

6.1. Learning objective

By the end of this chapter, you will be able to initialize OpenSpec for GitHub Copilot, explain which files the CLI generates, and configure each lab repository with concise planning context. You will also review the resulting diff and decide whether the repository is ready for the exploration workflow in Chapter 7.

6.2. Begin at the repository root

Chapter 5 established a clean baseline for `commercial-truck-search-api` and `commercial-truck-search-ui`. Because initialization writes files into the current project, that baseline provides the comparison point for the changes that follow. Running the command from a parent directory or the wrong terminal risks initializing the wrong folder.

Before changing either repository, inspect the working directory and Git state:

```
pwd
git status --short
```

Does the path identify the root of one lab repository? Does an empty `git\status\--short` result match the baseline from Chapter 5? If either answer is no, stop before running initialization.

6.3. Separate installation from initialization

Install the OpenSpec CLI once on the machine, then initialize each repository separately. The global installation can support both lab repositories, but each repository needs its own OpenSpec project and Copilot integration files.

For the book's pinned version, install the package and check the executable:

```
npm install -g @fission-ai/openspec@1.13.1
openspec --version
```

The second command needs to report 1.13.1 for the exercises in this book. If another version appears, resolve the mismatch before generating files because CLI versions can install different workflow content. The installation appendix covers operating-system-specific installation and PATH checks.

i A profile is not a technology stack

The core profile selects the standard OpenSpec workflows, such as explore, propose, apply, update, sync, and archive. It does not describe Java, Spring Boot, Angular, or the test tools in a repository. Put those project facts in `openspec/config.yaml` after initialization.

6.4. What initialization adds

From a repository root, the following command selects GitHub Copilot without opening the interactive tool picker:

```
openspec init --tools github-copilot --profile core
```

OpenSpec creates an `openspec/` directory and installs Copilot workflow files under `.github/`. The workflow count depends on the selected profile and OpenSpec version, so review the command summary rather than assuming a fixed count.

A newly initialized repository resembles this simplified tree:

```
repository-root/
|-- .github/
|   |-- prompts/
|   |-- skills/
|-- openspec/
|   |-- config.yaml
|   |-- specs/
|   |-- changes/
|   |-- archive/
|-- ...
```

The directories have different jobs:

Path	Intended role	Review question
<code>openspec/config.yaml</code>	Repository context, artifact rules, and operation guidance	Does this describe this repository rather than the whole product?
<code>openspec/specs/</code>	Accepted behavior after changes have been archived or synchronized	Is it empty for this new baseline?
<code>openspec/changes/</code>	Proposed work that has not yet entered the accepted specifications	Is it empty before Chapter 8?
<code>.github/skills/</code>	OpenSpec workflows GitHub Copilot may select as skills	Did initialization install the expected workflow set?
<code>.github/prompts/</code>	Prompt files exposed by supported Copilot IDE integrations	Does the IDE discover them after a restart?

Do not use initialization to invent accepted product behavior. Empty specs/ and changes/ directories are the expected result for these lab repositories. Later chapters add content through reviewed changes.

6.5. Configure context that can affect every change

OpenSpec reads openspec/config.yaml when a workflow asks the CLI for artifact or operation instructions. The context field reaches proposal, specification, design, task, apply, and archive work. Reserve it for facts that belong in every one of those prompts.

Useful context answers questions that source inspection alone may not settle:

- Which repository boundary does this project own?
- Which contract does it publish or consume?
- Which verification commands should planned work include?
- Which generated files should an agent avoid editing by hand?
- Which architectural or product constraints should shape every change?

Avoid turning config.yaml into a second README. File locations that an agent can discover, temporary implementation details, and lengthy style guides consume context without improving a plan. Prefer a short repository-specific rule to a page of general advice.

6.6. Configure the backend repository

Use the backend OpenSpec project to describe the boundary owned by commercial-truck-search-api. Its context identifies the service stack, the OpenAPI ownership rule, and the checks required before a task is complete.

After initialization, revise openspec/config.yaml toward a compact form such as this:

```
schema: spec-driven

context: |
  Repository: commercial-truck-search-api
  Tech stack: Java 21, Spring Boot 4.1.1, Maven, Spring Data JPA, and H2.
  This repository owns API behavior and the reviewed OpenAPI contract.
  Backend changes should preserve the existing controller, service, and persistence boundaries.

rules:
  specs:
    - Describe observable API behavior without prescribing Java implementation details.
    - Include scenarios for validation, successful results, no match, and relevant errors.
  tasks:
    - Include automated tests and OpenAPI publication when a contract changes.

operations:
  apply:
    guidance:
      - Run the repository's Maven tests before marking implementation tasks complete.
```

Treat this example as a starting point, not verified repository configuration. Confirm every version, command, and ownership statement against the Chapter 5 baseline before committing it. If the repository uses a Maven wrapper, name that checked-in command rather than a machine-wide Maven executable.

Notice what the example leaves out. Frontend rendering rules do not belong here, and the future unit-number endpoint has not yet been specified. Configuration can establish planning boundaries without deciding the behavior of a change that the team has not explored.

6.7. Configure the frontend repository

The frontend project needs different context because it owns user interaction and consumes the generated TypeScript client. Copying the backend configuration would give Copilot the wrong ownership model even though both repositories support the same product.

A starting configuration could look like this:

```
schema: spec-driven

context: |
  Repository: commercial-truck-search-ui
  Tech stack: Angular 21, TypeScript, Vitest, component tests, and Playwright.
  This repository owns user interaction and consumes a generated TypeScript API client.
  Do not edit generated client files by hand or redefine the backend wire contract here.

rules:
  specs:
    - Describe user-visible behavior, including loading, empty, validation, and error states.
    - Refer to the backend contract as a dependency rather than duplicating its schema.
  tasks:
    - Include the appropriate unit, component, or browser test for each behavior change.

operations:
  apply:
    guidance:
      - Run the repository's configured frontend tests before marking tasks complete.
```

This configuration should not make the generated client a source of product decisions. The backend owns the OpenAPI contract, while the frontend can record how it uses that contract and how the browser should respond to each state. If a frontend proposal needs a response field that the contract does not provide, the missing field should return to the backend change for review.

6.8. Understand generated files and team-owned context

The workflow files under `.github/` come from the installed OpenSpec package. Review and commit them, but avoid hand-editing generated instructions. A later `openspec\update` could replace or conflict with local edits.

The repository context in `openspec/config.yaml` carries a different responsibility. OpenSpec creates the file, but the team supplies and reviews the facts inside it. Divide ownership this way:

Material	Primary source	Maintenance action
Workflow instructions and prompt files	Installed OpenSpec package	Refresh with openspec\update
Workflow selection and delivery preference	OpenSpec profile on the developer machine	Change the profile, then update each repository
Repository planning context and rules	Repository team	Edit and review openspec/config.yaml
Accepted behavior and active changes	Product and engineering review	Change through the OpenSpec workflow

This distinction prevents two common mistakes: placing repository architecture in generated prompt files and expecting a global profile to describe a particular codebase.

6.9. Review initialization as a code change

Initialization deserves the same diff review as any other repository-wide tool addition. Because plain git\diff omits untracked files, stage the generated files before reviewing their content:

```
git status --short
git add openspec '.github/skills/openspec-*' '.github/prompts/opsx-*.prompt.md'
git diff --cached -- openspec '.github/skills/openspec-*' '.github/prompts/opsx-*.prompt.md'
```

During review, ask:

1. Did OpenSpec write into the intended repository root?
2. Does openspec/config.yaml name only facts that apply to this repository?
3. Do backend rules stay out of the frontend project, and vice versa?
4. Are accepted specifications and active changes still empty at this stage?
5. Did initialization overwrite an existing team-owned Copilot file?
6. Can another developer understand which files come from OpenSpec and which context the team must maintain?

Re-running openspec\init refreshes files in an initialized project, but still review the diff. When an installed CLI or profile changes, use the direct maintenance command:

```
openspec update
```

The update command refreshes configured workflow files from the installed CLI. Review those changes before committing them, especially when the team changes the pinned OpenSpec version.

6.10. Verify the initialized repositories

Run the following checks from each repository root:

```
openspec --version
openspec doctor
```

```
openspec list --specs
openspec list
git status --short
```

At this point in the lab, expect version 1.13.1, a health report with the intended OpenSpec root, no accepted specifications, and no active changes. The Git output should list only the initialization and configuration files you intend to commit.

openspec\validate\ -V-all checks discovered changes and specifications, but an empty project has no such artifacts yet. A successful empty validation confirms only that OpenSpec found no invalid item. It does not verify the accuracy of the repository context; reviewers must do that.

Restart VS Code or IntelliJ IDEA after initialization so the IDE can discover newly installed Copilot prompt files. Then confirm that the expected OpenSpec workflows appear in Copilot Chat. If the terminal commands work but the chat entries do not appear, investigate IDE discovery before changing the OpenSpec project structure.

6.11. Hands-on exercise: initialize both repositories

Work through the same sequence once for commercial-truck-search-api and once for commercial-truck-search-ui:

1. Confirm the repository root and clean baseline.
2. Verify that the CLI reports version 1.13.1.
3. Run openspec\init\ -V-tools\github-copilot\ -V-profile\core.
4. Inspect the generated tree and command summary.
5. Edit openspec/config.yaml with the repository's own context, artifact rules, and apply guidance.
6. Stage only the generated OpenSpec files and review them with git\diff\ -V-cached.
7. Run the verification commands and restart the IDE.
8. Commit each repository's initialization separately so later feature changes have a clean boundary.

Compare the two config.yaml files before committing. Their shared schema and workflow look similar, but their ownership statements, testing guidance, and generated-code boundaries are not interchangeable.

6.12. Verification checkpoint

Before moving to exploration, could you answer each question from repository evidence?

- Which OpenSpec version generated the workflow files?
- Which GitHub Copilot skills and prompt files did initialization add?
- Which facts in config.yaml will reach every planned change?
- Which artifact rules apply only to specifications or tasks?
- Which repository owns the OpenAPI contract?
- Which repository owns user-visible behavior and consumes the generated client?
- Does openspec\doctor resolve the intended root?
- Are openspec/specs/ and openspec/changes/ still free of invented feature behavior?

If an answer remains uncertain, revise the configuration or record the open question before asking Copilot to explore the unit-number search. A workflow can follow the context it receives, but it cannot repair an ownership rule the repository never stated.

6.13. What you learned

Initialization can establish the OpenSpec directory structure and install GitHub Copilot workflow files. Configuration can then tell those workflows what the repository owns, which constraints should shape its artifacts, and how implementation work should be checked.

With separate backend and frontend contexts under version control, Chapter 7 can begin exploration without asking one repository to speak for the other.

6.14. Online references

- OpenSpec installation¹
- Set up an OpenSpec project²
- OpenSpec project configuration³
- OpenSpec profiles⁴
- OpenSpec CLI reference⁵
- OpenSpec supported tools⁶

¹<https://openspec.dev/docs/installation>

²<https://openspec.dev/docs/setup>

³<https://openspec.dev/docs/project-config>

⁴<https://openspec.dev/docs/profiles>

⁵<https://openspec.dev/docs/cli>

⁶<https://openspec.dev/docs/supported-tools>

7. Exploring the Unit-Number Search

A Jira story asks for search by a 15-character unit number. That sentence may be enough to name the feature, but would it be enough to write the first test? Does “15-character” describe the stored value or the text a user may enter? Would a valid number with no matching truck produce an empty result, a 404, or a message on the page? Which repository would own each answer?

OpenSpec exploration can keep those questions visible while the team examines the request and the two repositories. The work in this chapter stops before proposal artifacts or code, when the team can tell which behavior has evidence and which behavior still needs a decision.

7.1. Learning objective

By the end of this chapter, you should be able to explore the unit-number search without silently designing it. Your output should be a reviewable decision record that separates facts from inferences, names unresolved questions, and assigns an owner or next action to each question.

7.2. Treat exploration as investigation

What should exploration produce? Not `proposal.md`, `design.md`, or `tasks.md`. Those artifacts would ask the team to review behavior that may still be in dispute. The immediate output should be a record of what the story states, what the repositories demonstrate, and what neither source can decide.

Consider the cost of skipping that separation. A backend agent could infer exact matching and return 404 for no result, while a frontend agent could infer partial matching and expect an empty list. Each implementation might look reasonable in its own repository. The generated client and the page would still expose the disagreement.

Exploration should therefore move from the story to repository evidence, then route unanswered questions to an owner. The reviewed record, rather than the chat transcript, could then become the source for the proposal in Chapter 8.

7.3. Begin with the thin request

For this exercise, begin with the available statement:

Users need to search for a commercial truck by its 15-character unit number.

What can that sentence support on its own? It names a user goal, an entity, a search field, and a length. Everything else in the table below would need evidence or a decision.

A first pass could classify the story this way:

Statement	Classification	What it permits
Users search for a commercial truck	Stated intent	Scope a truck-search capability
Search uses a unit number	Stated intent	Investigate that field across both repositories
The unit number has 15 characters	Stated constraint	Ask where and how length is validated
Matching is exact	Unstated	Raise a question; do not write a requirement
Lowercase input is accepted	Unstated	Inspect normalization behavior, then ask if needed
No match returns 404	Unstated	Compare API conventions without selecting one

The last three rows might sound reasonable, but the story does not support them. Repository precedent could make one of them a sensible option; only review could turn that option into accepted behavior.

Keep the Jira identifier and its exact wording with the exploration record. That link could later let reviewers trace an OpenSpec proposal back to its business source without asking the issue to carry details it never contained.

7.4. Establish the exploration boundary

Before opening Copilot Chat, decide what the agent may inspect and what it should leave untouched. For the two-repository lab, the useful boundary would include:

- the backend controllers, services, persistence queries, validation, error responses, tests, and reviewed OpenAPI contract;
- the frontend search forms, generated-client usage, loading and error states, result views, and tests;
- each repository's openspec/config.yaml and relevant Copilot instructions; and
- the Jira story or a faithful excerpt supplied in the prompt.

A generated TypeScript client may reveal the current API contract, but it should not decide what a new endpoint ought to return. Old chat messages deserve the same caution unless their claims can be traced to the story, the current repository baseline, or a recorded team decision.

Warning

Exploration can expose a missing decision, but it cannot approve that decision. When repository evidence conflicts with the Jira story, keep the conflict visible and send it to the person who owns the behavior.

7.5. Ask Copilot for evidence, not a solution

Open the backend repository in the IDE, start the OpenSpec exploration workflow, and supply the story as quoted source material. A prompt such as the following could constrain the response:

```
/opsx:explore We are considering an initial commercial-truck search capability:
"Users need to search for a commercial truck by its 15-character unit number."
```

Investigate this request without creating a proposal, specification, design, tasks, or code. Inspect the current repository and report:

1. Relevant existing routes, domain fields, validation, error handling, persistence queries, tests, and OpenAPI conventions.
2. What the Jira statement explicitly requires.
3. Candidate behavior suggested by repository evidence, with file references.
4. Questions the repository cannot answer, including exact versus partial matching, normalization, malformed input, no-match behavior, response fields, and non-goals.
5. A decision-record table with columns for question, evidence, status, owner, and next action.

Label every item as Stated, Observed, Inferred, or Open. Do not convert an inference into a requirement.

Why ask for file references? Without them, “the repository validates identifiers this way” may be no more than a fluent guess. A controller, schema, test, or generated contract gives the reviewer something to inspect. When Copilot cannot cite evidence, the item should remain inferred or open.

Run a corresponding exploration in the frontend repository. Adjust the first list so the agent looks for form controls, validators, generated-client calls, visible states, and interaction tests. Keep the same source labels and decision-record columns so the two reports can be compared without first rewriting them into a common shape.

7.6. Separate evidence from inference

An exploration response may mix four kinds of material in one polished paragraph. Give each claim one of these labels before using it:

Label	Meaning	Suitable next use
Stated	The Jira story says it directly	Candidate scope or constraint
Observed	A cited repository artifact demonstrates it	Evidence for a convention or dependency
Inferred	The agent connects stated intent with observed precedent	Option for review, not accepted behavior
Open	Neither the story nor repository settles it	Owner decision or explicit deferral

Suppose the agent finds an existing identifier search that trims whitespace. The finding would be observed behavior for that endpoint. Applying the same rule to unit numbers would remain an inference until the team accepts it.

Check each cited file and ask a few direct questions:

- Does the cited code execute in the current search path, or is it unused or generated?
- Does a test demonstrate behavior, or merely arrange test data?

- Does the OpenAPI document describe the implemented contract at the pinned baseline?
- Does the evidence belong to this repository, or did the agent import a convention from the other project?
- Did the response answer a business question by choosing the easiest implementation?

If a citation fails one of those checks, change the claim from “Observed” to “Inferred” or “Open.”

7.7. Build one cross-repository question set

The backend and frontend reports will probably overlap without matching exactly. Merge them by question rather than by prose. The resulting question set might look like this:

Question	Why implementation depends on it	Likely owner
Must all 15 characters match?	Determines query semantics and UI expectations	Product owner
May input contain spaces, hyphens, or lowercase letters?	Determines normalization and validation	Product owner with backend lead
What happens for empty or malformed input?	Shapes API errors and immediate form feedback	Product owner with API and UI leads
What happens when no truck matches?	Must agree across HTTP response, client behavior, and page state	API and UI leads
Which truck fields appear in a successful result?	Defines the backend-owned response contract	Product owner with backend lead
May more than one truck match?	Determines whether the response is an object or collection	Product owner
Which searches remain out of scope?	Prevents VIN, partial, and multi-field search from entering the change	Product owner

“Likely owner” should remain a routing suggestion until the team confirms it. The agent can show who appears to own an existing contract or component, but organizational authority may not be visible in either repository.

7.8. Turn questions into a decision record

Build the decision record from the merged question set. Each row should preserve the evidence reviewed, the current status, the owner, and the next action; an open cell should not be filled with a guess merely to make the table look complete.

```
| Question | Evidence | Status | Owner | Next action |
|---|---|---|---|---|
| Exact or partial match? | Jira gives only field and length | Open | Product owner | Choose match
| Normalize input? | Existing endpoint trims input at `path:line` | Inferred option | Product owner
| Malformed input? | API uses validation response at `path:line` | Candidate convention | Backend
| No matching truck? | Backend and UI precedents differ | Open conflict | API + UI leads | Agree c
| Result fields? | Current truck summary type at `path:line` | Inferred option | Product owner | A
| VIN search? | Not named in Jira | Deferred non-goal | Product owner | Confirm exclusion from fir
```

Replace each path:line placeholder with a real repository reference during the lab. If no reference supports a row, mark the evidence as unavailable.

Notice the difference between “Open” and “Deferred non-goal.” An open question blocks some part of the proposed behavior. A deferred item has an explicit boundary and may be considered in a later change. Calling an unanswered question “out of scope” would hide a decision rather than postpone it responsibly.

7.9. Use follow-up prompts to challenge the record

One broad prompt may identify the obvious questions, while short follow-ups can test the weakest claims:

Which claims lack a direct repository citation? Reclassify each one as Inferred or Open.

Trace no-match behavior from the backend response through the generated client to the UI state. Where does the current evidence stop?

Which behaviors did you infer only because they would be conventional or easy to implement? Turn them into questions.

What is the smallest set of unresolved decisions that prevents a reviewer from writing observable scenarios for this change?

These follow-ups test the report rather than widen the scope. If the conversation has accumulated unsupported assumptions, start a fresh exploration from the verified record instead of treating earlier chat as evidence.

7.10. Compare VS Code and IntelliJ IDEA

Run the same bounded prompt in VS Code and IntelliJ IDEA after both environments can discover the OpenSpec workflow files. Exact wording need not match. Compare whether each response:

- inspected both repository evidence and OpenSpec context;
- preserved the distinction among stated, observed, inferred, and open material;
- cited files that actually support its claims;
- found the same implementation-blocking questions; and
- avoided creating change artifacts or implementation details.

Differences could point to context discovery rather than model quality. Did one IDE have the correct repository root open? Could both see the same generated prompt files and instructions? Were the same Jira text and repository baseline available? Check those conditions before judging the responses by their prose.

Combine verified findings into one decision record. Two competing records would leave Chapter 8 without a stable source, even if each IDE produced useful observations.

7.11. Know when to stop exploring

More exploration does not always produce more certainty. Repository search cannot answer whether the business wants partial matching, and repeated prompts may only produce increasingly polished guesses. Consider the exploration ready for review when:

- every behavior needed for an observable scenario has a decision, named owner, or explicit deferral;
- every “Observed” claim has a checked repository reference;
- every inference remains labeled as an option;
- backend and frontend expectations no longer contradict each other without a recorded owner; and
- non-goals define what the first change should leave alone.

The record may still contain unanswered questions, provided each one has an owner or an explicit deferral. Once repository evidence runs out, another search may add less than a short conversation with the product owner or API lead.

7.12. Hands-on exercise

Explore the unit-number search once in each repository and IDE:

1. Copy the Jira wording into a working note without expanding it.
2. Run the bounded exploration prompt in the backend repository.
3. Verify each cited route, field, validator, test, error convention, and contract artifact.
4. Run the corresponding prompt in the frontend repository.
5. Verify each cited form, client call, visible state, and interaction test.
6. Merge both reports by question and reclassify unsupported claims.
7. Ask the listed owners to decide, defer, or reject each unresolved behavior.
8. Save one agreed decision record for the proposal work in Chapter 8.

Do not create an OpenSpec change during this exercise. If an open row has neither an owner nor a next action, the exploration has located a gap but has not made it actionable.

7.13. Verification checkpoint

Before leaving exploration, could a reviewer answer these questions from the record?

- Which words came directly from the Jira story?
- Which claims came from checked repository evidence?
- Which candidate behaviors remain inferences?
- Who can decide each open product or contract question?
- Which disagreements cross the backend and frontend boundary?
- Which capabilities have been explicitly deferred?
- Which decisions would Chapter 8 use to write observable scenarios?

If those answers require rereading the chat transcript, revise the record. The durable output of exploration should be the reviewed evidence and decisions, not the conversation that happened to produce them.

7.14. What you learned

For the unit-number search, exploration should leave one cross-repository record whose claims can be traced and whose gaps have owners. Chapter 8 can use accepted rows to propose backend and frontend changes; unsupported inferences should remain outside the requirements.

7.15. Online references

- OpenSpec skills reference¹
- GitHub Copilot prompt engineering²

¹<https://openspec.dev/docs/skills>

²<https://docs.github.com/en/copilot/concepts/prompt-engineering>

8. Proposing the First Full-Stack Change

Chapter 7 ended with a decision record, not a plan. How might that record become a change that a backend developer, a frontend developer, and Copilot could inspect without replaying the original conversation? Which decisions belong in shared behavior, which belong to one repository, and which should still block implementation?

Follow the unit-number search through OpenSpec's proposal workflow. Keep the backend and frontend work in separate active changes, then connect them through one reviewed API contract and an explicit handoff. Aim for more than a set of completed Markdown files: give reviewers a cheap way to find scope drift before either repository changes application code.

8.1. Learning objective

Use this chapter to turn a reviewed exploration record into proposal, delta specification, design, and task artifacts for a full-stack capability. By the end, you should be able to explain why two repository changes are preferable here, trace a scenario across both changes, and decide whether the pair is ready for the apply workflow.

8.2. Treat proposal as a review boundary

Do not treat entry into the proposal workflow as evidence that behavior has shipped. Treat it as the point where the team places a specific difference under review while the accepted product behavior remains unchanged.

That distinction should affect how you read the generated files:

Artifact	Question for the reviewer	Warning sign
proposal.md	Why should this change exist, and where should it stop?	Classes, methods, or response details appear before the scope has been accepted
specs\TU\textless{ }capability\ TU\textgreater{ }/spec.md	What new behavior could a user or client observe?	A scenario depends on an unanswered question or names implementation machinery
design.md	How might this repository satisfy the behavior?	A technical choice quietly creates new user-visible behavior
tasks.md	What work and evidence would complete the change?	A task has no requirement, or a requirement has no implementation and verification task

Use one proposal when the affected code and review path belong to one repository. For this lab, keep one active change in each repository because the applications have separate code, tests, and ownership boundaries. Connect those changes through the contract dependency instead of hiding the boundary inside one plan.

Before generating files, trace one accepted decision through the artifacts it would affect:

```
accepted no-match decision
|
+-- backend scenario
|   +-- response-mapping design decision
|   |-- API test task
|
|-- frontend empty-state scenario
    +-- state-handling design decision
    |-- component test task
```

Use the same trace for valid input, a successful match, and each reviewed error. A broken branch would expose a missing artifact before implementation begins.

8.3. Check whether exploration is ready to become a proposal

Before invoking the proposal workflow, reopen the Chapter 7 decision record. Ask whether a reviewer could write an observable scenario without supplying another assumption.

Use questions such as these:

- Has the product owner selected exact or partial matching?
- Has the team agreed on valid input, normalization, and no-match behavior?
- Has someone approved the fields returned for a match?
- Have related searches, such as VIN or multi-field search, been accepted or deferred?
- Does each unresolved item have an owner and a reason it does not block this change?

Do not mistake an assigned owner for permission to ignore a blocking question. If the answer could change a response status, request constraint, result shape, or visible UI state, pause the proposal and get the decision. If the answer affects only a later capability, record the deferral and keep it outside the current scope.

Do not let the example choose the product behavior

The artifact excerpts below illustrate structure. Replace bracketed text with the reviewed decisions from your Chapter 7 record. Do not treat exact matching, an empty result, or any example response shape as accepted merely because it appears in this chapter.

8.4. Name two changes around one capability

Choose names that describe outcomes rather than layers or ticket numbers. For the lab, the two change directories might look like this:

```

commercial-truck-search-api/
`-- openspec/changes/add-unit-number-search-api/
    |-- proposal.md
    |-- specs/unit-number-search/spec.md
    |-- design.md
    `-- tasks.md

commercial-truck-search-ui/
`-- openspec/changes/add-unit-number-search-ui/
    |-- proposal.md
    |-- specs/unit-number-search/spec.md
    |-- design.md
    `-- tasks.md

```

Use repository-qualified names when they make cross-repository review notes and build output easier to read. The suffix matters less than using the same capability term, unit-number-search, in both delta specifications.

Record the ownership boundary before asking Copilot to draft either change:

Concern	Backend change	Frontend change
Accepted search rule	Express API-observable behavior	Refer to the same rule through user-visible interaction
Wire contract	Own and publish the reviewed OpenAPI document	Consume a generated client from that document
Validation	Enforce the API boundary	Offer immediate feedback without weakening or redefining the API rule
Outcomes	Return the reviewed success, no-match, and error responses	Render loading, result, empty, and error states
Verification	Test service and HTTP behavior; inspect the contract	Test interaction and rendering; exercise the generated client boundary

Keep request and response types under backend ownership. If the frontend needs a field or error that the backend change does not provide, return that need to the backend proposal instead of editing generated types or inventing a second contract.

8.5. Draft the backend change first

Start in the backend repository because its reviewed contract constrains the frontend handoff. In Copilot Chat, adapt the following prompt to the decisions you actually recorded:

```

/opsx:propose add-unit-number-search-api

Using the attached Chapter 7 decision record, draft an OpenSpec change for the backend-owned unit-number search capability.

```

Please:

- treat only Accepted decisions as requirements;
- preserve Open and Deferred items as dependencies or non-goals;
- keep Java, Spring, persistence, and controller choices out of behavioral requirements;
- include scenarios for each accepted valid-input, match, no-match, and invalid-input outcome;
- record OpenAPI publication and backend verification in the design and tasks; and
- stop and report any decision needed to write an observable scenario.

Do not modify application code or accepted specs.

After generation, inspect the artifact order rather than accepting the folder as one indivisible answer. The proposal should constrain the delta and design; the delta and design should constrain the task list.

8.5.1. Read `proposal.md` for scope

A compact backend proposal might use this shape:

```
# Change: Add unit-number search API
```

```
## Why
```

Fleet coordinators need to locate a commercial truck from the unit number supplied in their workflow.

```
## What Changes
```

- Add API behavior for searching by a [reviewed valid unit-number rule].
- Return the [reviewed match and no-match outcomes].
- Publish the request, response, and error behavior in OpenAPI.

```
## Non-Goals
```

- VIN search
- Partial or multi-field search
- Frontend rendering behavior

```
## Dependencies
```

- The frontend change may generate its client only after review of the published OpenAPI document.

Review the proposal before reading its dependent artifacts:

- Does Why justify the work without restating the Jira title?
- Does What\Changes name observable scope without selecting endpoint classes or query methods?
- Do the non-goals prevent plausible but unwanted expansion?

Revise the proposal first if any answer is no.

8.5.2. Read the backend delta for behavior

Insert the approved input, result, and error decisions into the delta. Until then, keep explicit placeholders rather than completing the requirement from repository precedent or convention:

```
## ADDED Requirements

### Requirement: Search by unit number

The API SHALL accept a unit number that satisfies [the reviewed input rule] and return [the reviewed result shape].

#### Scenario: Matching truck exists

- **WHEN** a client searches with a valid unit number that matches a truck
- **THEN** the API returns the reviewed truck representation

#### Scenario: No matching truck exists

- **WHEN** a client searches with a valid unit number that matches no truck
- **THEN** the API returns the reviewed no-match response

#### Scenario: Unit number is invalid

- **WHEN** a client searches with a unit number that violates the reviewed input rule
- **THEN** the API returns the reviewed validation response
```

Check whether each scenario could become an HTTP-level test without knowledge of a Java class. Keep invalid input distinct from valid input with no match, and map every placeholder to an accepted row in the decision record. Leave a missing answer visible rather than replacing it with whatever behavior seems conventional.

8.5.3. Read design.md for consequential choices

Now ask how the backend might satisfy those scenarios. A useful design would usually discuss the request boundary, service and persistence path, error mapping, OpenAPI publication, and test levels. It might record a decision in this form:

```
## Decision: Publish the backend contract before client generation

Generate the OpenAPI document from the implemented backend boundary and review it against the delta scenarios before the frontend generates a client.

### Rationale

This order keeps request constraints, result fields, and error responses under backend ownership.

### Consequence
```

The frontend implementation task remains blocked until the reviewed contract artifact is available.

Apply a replacement test to each statement. If swapping Spring Data for another persistence mechanism would preserve the statement, keep the mechanism in the design. If changing case handling would alter what a client observes, place that rule in the delta rather than hiding it in a mapper or query decision.

8.6. Draft the frontend change from the contract boundary

Create the frontend proposal only after the backend scope and observable contract have survived review. The OpenAPI document need not exist yet, but the dependency and ownership rule should.

```
/opsx:propose add-unit-number-search-ui
```

Using the reviewed backend change and the attached Chapter 7 decision record, draft an OpenSpec change for the Angular unit-number search interaction.

Please:

- describe user-visible input, loading, result, empty, validation, and error behavior;
- refer to the backend-owned OpenAPI contract instead of duplicating request or response schemas;
- make client generation depend on the reviewed backend contract;
- keep Angular component and service choices in design.md; and
- include tests and a cross-application verification task.

Do not edit generated client files or application code.

Write the frontend scenarios from the user's interaction rather than restating HTTP mechanics. A representative excerpt might read:

```
## ADDED Requirements
```

```
### Requirement: Search for a truck by unit number
```

The interface SHALL let a user submit a unit number that satisfies [the reviewed input rule].

```
#### Scenario: Search is in progress
```

- ****WHEN**** the user submits a valid unit number and the request has not completed
- ****THEN**** the interface shows the reviewed loading state

```
#### Scenario: No truck is found
```

- ****WHEN**** the backend returns the reviewed no-match response
- ****THEN**** the interface shows the reviewed empty state

```
#### Scenario: Search fails
```

- ****WHEN**** the request returns a reviewed error response

- ****THEN**** the interface shows the corresponding error state

Apply two checks to this delta. First, confirm that it would still make sense if the generated TypeScript method received a different name. Second, replace broad wording such as “corresponding error state” with the accepted message or state when the decision record provides one.

8.7. Turn the handoff into ordered tasks

Use the full-stack task lists to expose waiting points instead of implying that both agents can work independently from the first minute. Keep repository-local work in its own tasks.md, then repeat the contract handoff in both lists.

The backend sequence might include:

```
## 1. API behavior

- [ ] 1.1 Implement the reviewed input and search behavior.
- [ ] 1.2 Map match, no-match, and invalid-input outcomes to the reviewed responses.
- [ ] 1.3 Add focused tests for every backend scenario.

## 2. Contract handoff

- [ ] 2.1 Generate the OpenAPI document from the implemented API.
- [ ] 2.2 Compare its constraints, schemas, and responses with the delta specification.
- [ ] 2.3 Publish the reviewed document for the frontend change.
```

The frontend sequence might include:

```
## 1. Contract intake

- [ ] 1.1 Obtain the reviewed OpenAPI document from `add-unit-number-search-api`.
- [ ] 1.2 Generate the TypeScript client without hand-editing generated files.

## 2. Search interaction

- [ ] 2.1 Implement the reviewed input and submission behavior.
- [ ] 2.2 Render loading, result, empty, and error states from the generated client response.
- [ ] 2.3 Add focused tests for every frontend scenario.
- [ ] 2.4 Run the cross-application unit-number search path.
```

Block client generation until the backend publishes a reviewed contract. For every task, name the expected test, generated artifact, or review evidence so that a changed file alone cannot satisfy the checkbox.

8.8. Inspect the generated changes with OpenSpec

Use Copilot Chat to draft and revise the artifacts, then use the terminal to inspect OpenSpec’s view of each change:

```

openspec status --change add-unit-number-search-api
openspec validate add-unit-number-search-api
openspec show add-unit-number-search-api

```

Run the corresponding commands in the frontend repository with `add-unit-number-search-ui`. A successful validation could confirm structure, but it could not confirm that the team chose the right behavior. Read the Markdown and compare it with the source decision record.

When review changes a requirement, update its dependents in the same pass. Recheck error mapping, frontend state handling, tests, and contract generation after a change to the no-match rule. Do not treat the change as coherent until its dependent artifacts agree.

8.9. Review both changes as one capability

Open the backend and frontend changes side by side. Follow behavior from the user’s action to the API response and back to the rendered state:

Review path	Backend evidence	Frontend evidence	Question
Valid submission	Request constraint and success scenario	Input and submission scenario	Do both use the same accepted input rule?
Match	Response scenario and schema decision	Result-state scenario	Can the generated contract supply every displayed field?
No match	Reviewed API response	Empty-state scenario	Does the UI interpret the response without redefining it?
Invalid input	Validation response	Immediate feedback and response handling	Could the UI permit input that the API rejects?
Failure	Error contract	Error-state scenario	Which failures receive distinct user-visible treatment?

Look for drift that valid Markdown would not catch:

- Does one change say “exact” while the other implies partial matching?
- Does the frontend expect one result while the backend returns a collection, or the reverse?
- Does either design introduce normalization absent from the behavioral delta?
- Does a task add VIN search, pagination, filtering, or another deferred capability?
- Does every scenario lead to implementation and verification work?

Ask Copilot for a review rather than another rewrite when the drafts are complete:

```
Review these two OpenSpec changes as one full-stack capability without editing them.
```

```
Trace every accepted decision from the Chapter 7 record through the backend and frontend proposals, delta specifications, designs, and tasks. Report contradictions, unsupported assumptions, duplicated contract definitions, missing scenarios, broken dependency order, and
```

tasks with no requirement. Cite the artifact and heading for each finding.

Still give the agent's report a human review. In particular, verify whether a cited "contradiction" represents a real behavioral mismatch or an intentional difference between an API response and its user-facing presentation.

8.10. Hands-on exercise

Create the two unit-number search changes in their respective repositories:

1. Reopen the Chapter 7 decision record and separate Accepted, Open, and Deferred rows.
2. Draft add-unit-number-search-api from accepted decisions only.
3. Review its proposal, delta, design, and tasks in dependency order.
4. Validate the backend change and record any structural errors.
5. Draft add-unit-number-search-ui with an explicit dependency on the reviewed backend contract.
6. Review and validate the frontend change.
7. Trace each accepted scenario across both repositories with the review table above.
8. Revise the artifact that owns each defect, then recheck its dependents.

Run the workflow once from VS Code and once from IntelliJ IDEA if both environments are available. Compare the resulting artifacts and validation commands even if prompt discovery or chat entry points differ. Record any difference that changes which repository instructions or OpenSpec skills the agent can see.

Do not apply either change during this exercise. The stopping point should be a reviewed pair of plans whose remaining dependencies are explicit.

8.11. Verification checkpoint

Before moving to Chapter 9, ask:

- Can every implementation-blocking question from exploration be found as an accepted decision, named dependency, or explicit deferral?
- Does every requirement describe observable behavior through scenarios?
- Can each scenario be traced to a design decision when one is needed and to at least one task?
- Does the backend alone own the OpenAPI wire contract?
- Must contract review and publication occur before frontend client generation?
- Do frontend states correspond to the reviewed backend outcomes?
- Do both openspec\validate commands succeed?
- Has a human reviewer checked behavior and scope beyond structural validation?

Let a "no" send the change back to the artifact that owns the missing information. Repair a missing behavior in the delta, a technical omission in the design, and missing work in the task list. Avoid compensating for an incomplete requirement with a more detailed implementation task.

8.12. What you learned

Use a full-stack proposal to let a reviewer start with an accepted decision, follow it through both repositories, and see the contract handoff that orders the work. Keep the two OpenSpec changes linked without merging separate ownership boundaries or asking the frontend to duplicate the API contract.

Begin Chapter 9 only after that path survives review. Use the backend change's current artifacts as its first input, not the original Jira sentence and not a remembered chat response.

8.13. Online references

- [OpenSpec quickstart](#)
- [OpenSpec spec-driven schema](#)
- [OpenSpec CLI reference](#)
- [OpenAPI Specification](#)

Part III.

**Part III: Applying and Evolving the
Specification**

9. Applying the Spring Boot Change

What happens when a reviewed OpenSpec plan meets an existing Spring Boot codebase? Use the apply workflow to find out one task at a time. Let Copilot inspect the active change and nearby code, then require a focused test before it checks off the task. If the repository contradicts the plan, pause and repair the artifact that owns the decision.

9.1. Learning objective

Use this chapter to apply the backend unit-number change without reducing OpenSpec to a prompt template. By the end, you should be able to start or resume `/opsx:apply`, connect each backend scenario to code and a meaningful assertion, recognize when implementation should stop, and prepare a reviewed OpenAPI document for the Angular handoff.

9.2. Move from artifacts to code

The proposal workflow answered, “What change should the team review?” Apply asks a different question: “What code and evidence would satisfy the reviewed change in this repository?”

Keep the four planning artifacts in distinct roles while answering it:

Artifact	Question during apply	Reason to pause
proposal.md	Does this edit remain inside the accepted purpose and scope?	The implementation introduces a deferred capability or affects an unnamed system
Delta spec.md	Which observable result must this task produce?	The scenario cannot determine a request, response, or error outcome
design.md	Which technical boundary and trade-off should guide the edit?	The codebase disproves an assumption or requires a consequential new choice
tasks.md	What remains, and what evidence would justify checking it off?	The task has no matching requirement, test, or reviewable output

Do not ask Copilot to implement from tasks.md alone. A task such as “add the search endpoint” cannot settle the accepted matching rule, response shape, or no-match behavior. Require the workflow to read the proposal, delta specification, and design before treating the checklist as executable work.

i Apply does not approve the plan

Treat a reviewed plan as the current implementation target, not as an infallible prediction. If the code exposes a missing behavior or an impossible design assumption, return to the owning artifact and review the correction before continuing.

9.3. Check readiness before changing code

Start in the backend repository and inspect the active change from the terminal:

```
openspec status --change add-unit-number-search-api
openspec validate add-unit-number-search-api
openspec show add-unit-number-search-api
```

Look for the expected change name, complete planning artifacts, structurally valid requirements, and unchecked implementation tasks. Then read the files themselves; structural validation cannot decide whether the team chose the right behavior.

Before invoking apply, answer these questions from the artifacts:

- Which unit-number inputs should the API accept or reject?
- What should a client observe for a match and for no match?
- Which error responses belong to this change?
- Which existing controller, service, and persistence boundaries should remain in place?
- Which test command and OpenAPI generation path does the repository use?
- Which capabilities, such as VIN or partial search, remain outside this change?

If any answer requires a guess, stop there. A more detailed prompt should not substitute for a missing product decision, and an implementation convention should not quietly become new API behavior.

Also check the worktree before delegating edits. Note which files already contain user changes and locate the nearest comparable search path. Ask Copilot to preserve unrelated work and inspect the nearest implementation and test before creating another abstraction.

9.4. Start a bounded apply session

Name the change when more than one active change could be selected:

```
/opsx:apply add-unit-number-search-api
```

The installed workflow should read the change context, find pending tasks in tasks.md, implement them, and mark completed checkboxes as work progresses. Strengthen that invocation with a bounded request when repository details or review expectations deserve emphasis:

```
/opsx:apply add-unit-number-search-api
```

Apply the next unchecked backend task from the active change.

Before editing:

- read proposal.md, the unit-number-search delta specification, design.md, and tasks.md;
- inspect the existing controller, service, repository, and nearest tests; and
- state which scenario the task should satisfy and which focused check could disprove the edit.

During implementation:

- preserve the existing Spring Boot boundaries described by the design;
- keep VIN, partial search, and frontend behavior outside this change;
- do not invent a response or validation rule when the artifacts are silent; and
- preserve unrelated worktree changes.

After editing, run the narrow test named by the repository or task. Mark the task complete only after the code, assertion, and test output agree. Pause and report contradictions or missing decisions instead of choosing behavior on the team's behalf.

Start with one unchecked task so that a focused test can reject the implementation before the same mistake spreads through controller, service, repository, contract, and test code. A tightly coupled pair of edits may need to land together; ask Copilot to explain that dependency and run the narrowest useful check afterward.

The editor need not change the source of truth. In VS Code, invoke the installed OPSX prompt from Copilot Chat and attach only the files needed to resolve the current task. In IntelliJ IDEA, use the corresponding installed workflow and working-set controls. In either client, judge the result against the same repository artifacts and commands.

9.5. Follow a scenario through the backend

Choose one scenario before choosing files. Trace a matching unit number through the application:



Use this path as an inspection guide, not as permission to add every layer. If the repository already keeps validation in a request type or delegates search directly from a controller to a service, follow that local design unless design.md explicitly changes it.

9.5.1. Keep validation at the API boundary

Start with the accepted input rule, then find the Spring boundary that already validates comparable request values. Reuse its annotation, validator, or error-mapping convention when it can express the reviewed behavior without changing it.

Then write or adapt a focused assertion. Could the test distinguish invalid input from a valid unit number that has no match? Could it catch an implementation that trims, folds case, or accepts partial input when the specification does not permit that transformation?

Avoid deriving the rule from a database column or a frontend control. Those details might suggest a useful implementation, but the backend delta should remain the authority for what a client may send and observe.

9.5.2. Preserve controller and error semantics

Next, inspect the controller or route that owns adjacent search behavior. Follow its response- mapping pattern for successful results, no match, and invalid requests only where that pattern agrees with the reviewed scenarios.

Make assertions at the HTTP boundary for status, body shape, and relevant headers. A service test could prove that no entity was returned while still missing a controller that maps the absence to the wrong response. Conversely, a controller test with a fully stubbed service might not prove the repository query.

9.5.3. Add service and persistence behavior narrowly

Let the design decide whether Spring Data JPA and H2 belong in this path. Search for the nearest repository method and naming convention before adding a query, and avoid broad cleanup while the change remains focused on unit-number search.

Choose test boundaries by the defect each test could expose:

Boundary	Useful question	Possible evidence
Validator or request model	Does invalid input fail under the reviewed rule?	Focused validation test
Service	Does the use case preserve match and no-match meaning?	Unit test with a controlled repository result
Repository	Does persisted data produce the reviewed match behavior?	JPA slice or repository test where needed
HTTP API	Can a client observe the specified status and body?	Spring Boot integration test

Do not add every test type mechanically. Choose the narrowest test that could fail for the defect under review, then add an integration test where several Spring boundaries must cooperate to prove the scenario.

9.6. Use the test result to control progress

After the first implementation edit, run the focused check named in tasks.md or the repository instructions. Ask Copilot to report what the result established and what it left untested instead of presenting a green command as complete proof.

A useful progress report might use this form:

```
Task: 1.2 Map match and no-match responses
Scenario exercised: valid unit number with no matching truck
Implementation boundary: controller response mapping
Command run: <repository-configured focused test command>
Result: pass
Remaining gap: invalid-input response belongs to task 1.3
Task checkbox updated: yes
```

Keep the checkbox open when the command fails, when the assertion does not observe the specified result, or when the generated output still disagrees with the change. If a task contains several independent outcomes, consider splitting it in tasks.md so that partial progress remains visible without overstating completion.

Compare each passing assertion with the delta scenario before accepting it. Otherwise, a test may repeat the implementation's mistake. The requirement should supply the expected behavior; the production method should not supply the expected value back to its own test.

9.7. Know when to return to the artifacts

Implementation often reveals facts that planning missed. Use the kind of mismatch to choose the file that needs review:

Discovery during apply	Return to	Do not hide it by
The requested outcome exceeds the proposal scope	proposal.md	Adding an untracked side feature
Match or no-match behavior remains ambiguous	Delta spec.md	Selecting a convenient status code
The planned persistence path conflicts with the codebase	design.md	Creating a parallel architecture
A scenario has no implementation or test work	tasks.md	Relying on a broad final test task

After revising an artifact, inspect its dependents again. A changed no-match requirement may alter controller mapping, OpenAPI responses, test assertions, or the frontend handoff. Update the affected design and tasks before resuming /opsx:apply.

 Do not make the implementation retroactively correct

If the code behaves differently from the reviewed requirement, do not rewrite the requirement merely to make the test pass. Ask whether the requirement was wrong, whether the implementation was wrong, or whether the team has made a new decision. Record and review that decision before continuing.

9.8. Generate and inspect the backend contract

Once the HTTP scenarios pass, generate the OpenAPI document through the repository's configured Spring Boot path. Do not recreate the schema by hand for the frontend. The generated document should carry the implemented request constraints, response schemas, and error responses across the repository boundary.

Inspect the document as another implementation output:

- Does the unit-number operation use the reviewed path and request location?
- Do required fields and constraints match the delta scenarios?
- Can the success schema supply every field promised by the backend change?
- Can a client distinguish no match from invalid input and other reviewed errors?
- Does each documented status correspond to tested controller behavior?

If generation succeeds but the document omits a validation constraint, leave the contract task open. Repair the owning annotation, schema customization, or controller description. A successful generator run would show only that a document could be produced, not that it describes the accepted API.

Use a short traceability table during review:

Backend scenario	Implementation owner	Assertion	Contract evidence
Valid input with match	Request boundary through repository	Reviewed success status and body	Operation and success schema
Valid input with no match	Service or controller mapping	Reviewed no-match response	Documented no-match response
Invalid input	Validation and error mapping	Reviewed validation response	Request constraint and error schema
Reviewed backend failure	Error boundary	Reviewed error status and body	Documented error response

Replace the generic row labels with the actual scenarios from the active delta. If the delta has another scenario, add a row rather than assuming one of these examples covers it.

9.9. Resume without relying on chat memory

If the session ends after task 1.2, start a fresh session, name the same change, and let the unchecked boxes identify the remaining work:

```
/opsx:apply add-unit-number-search-api
```

Before accepting the resumed task, ask the agent to read the current artifacts and inspect the latest test evidence. The task list can record claimed progress, while a short reviewed handoff may record a failing command or temporary blocker. Neither should require the old conversation to reconstruct accepted behavior.

9.10. Hands-on exercise

Apply the backend unit-number change in the prepared Spring Boot repository.

1. Run `openspec\status`, `openspec\validate`, and `openspec\show` for `add-unit-number-search-api`, then read the four artifact types in dependency order.
2. Identify the first unchecked task, its governing scenario, the existing implementation boundary, and one focused check that could reject the proposed edit.

3. Run `/opsx:apply\add-unit-number-search-api` in VS Code. Ask Copilot to stop after the task's focused check and explain the evidence before updating the checkbox.
4. Continue through validation, controller mapping, service behavior, persistence, and error handling only where the reviewed design and task list call for them.
5. Add focused tests for every backend scenario, using a Spring Boot integration test when the application, persistence, and HTTP boundaries must work together.
6. Generate the OpenAPI document through the configured repository path and compare it with each backend scenario.
7. Resume the same change in a fresh session or in IntelliJ IDEA. Confirm that the artifacts and unchecked tasks provide enough context without replaying the earlier chat.
8. Leave any unsupported task open, record the blocker in the owning artifact, and rerun the affected checks after review.

9.11. Verification checkpoint

Before handing the contract to the Angular repository, ask:

- Does every backend scenario map to an implementation boundary and an observable assertion?
- Did the focused and integration checks run through the repository's configured commands?
- Do checked tasks correspond to passing evidence rather than edited files?
- Did implementation remain inside the proposal scope and reviewed design?
- Were contradictions returned to the owning OpenSpec artifact?
- Does the generated OpenAPI document match the tested constraints, response types, and errors?
- Could a fresh Copilot session identify the next task from the active change alone?

Let any “no” keep the backend handoff open. Begin Chapter 10 with a reviewed contract rather than an Angular workaround for behavior the backend has not settled.

9.12. What you learned

Apply should now mean more than “ask Copilot to write the code.” Move from one reviewed task to one bounded edit and one discriminating check. When a check exposes a missing decision, return to the artifact that owns it. Hand the frontend only the behavior supported by the backend code, tests, and generated OpenAPI document.

9.13. Online references

- OpenSpec command reference¹
- OpenSpec skills reference²
- Spring Boot testing³
- Spring Data JPA reference⁴
- H2 Database documentation⁵

¹<https://github.com/Fission-AI/OpenSpec/blob/main/docs/commands.md>

²<https://openspec.dev/docs/skills>

³<https://docs.spring.io/spring-boot/reference/testing/index.html>

⁴<https://docs.spring.io/spring-data/jpa/reference/>

⁵<https://h2database.com/html/main.html>

10. Applying the Angular Change

Begin with the active OpenSpec change, accept the reviewed OpenAPI document as the backend handoff, and apply one frontend scenario at a time. After each edit, identify the requirement the code satisfies and the focused check that could prove it wrong. This sequence keeps the component from becoming an accidental source of API or product behavior.

Do not ask Copilot to “build the search page” from a screenshot, a chat summary, or the generated client alone. None of those sources records the complete loading state, the distinction between no match and failure, or behavior deliberately left outside the change. Read those decisions from the frontend delta specification, use `design.md` to constrain the Angular boundaries, and use `tasks.md` to expose unfinished work.

10.1. Learning objective

Use this chapter to apply `add-unit-number-search-ui` as an OpenSpec change rather than as an unbounded Angular prompt. Practice checking readiness, starting or resuming `/opsx:apply`, generating the client from the reviewed backend contract, tracing each visible state to a focused test, and pausing when code or generated output contradicts the accepted artifacts.

10.2. Enter `apply` through the frontend change

Let the four reviewed artifacts control the first edit in the Angular repository. Keep each one in its assigned role:

Artifact	Question to ask during frontend <code>apply</code>	Reason to pause
<code>proposal.md</code>	Does this UI edit remain inside the accepted purpose and scope?	The edit adds another search mode, field, or workflow
<code>Delta spec.md</code>	Which user-visible state should this task produce?	The scenario leaves the input, result, empty, or error behavior open
<code>design.md</code>	Which Angular boundary should own the behavior?	The repository contradicts the planned component, service, or generated-client boundary
<code>tasks.md</code>	What work remains, and what evidence would justify checking it off?	The task names files but no requirement, interaction, or check

Read the artifacts together before editing application code. Do not let a task such as “implement search component” silently decide when submission becomes available, what happens to an old result during a second request, or how the interface distinguishes no match from failure.

i Apply the reviewed change, not the previous chat

Start a fresh Copilot session if useful. Name the change, attach only the relevant repository context, and require the session to recover accepted behavior from OpenSpec artifacts. Treat any detail available only in the earlier conversation as missing project context.

10.3. Check the handoff before changing Angular code

Begin in the frontend repository and inspect the active change:

```
openspec status --change add-unit-number-search-ui
openspec validate add-unit-number-search-ui
openspec show add-unit-number-search-ui
```

Then read proposal.md, the frontend delta, design.md, and tasks.md in dependency order. Ask whether the artifacts can answer all of these questions:

- Which unit-number input may the user submit?
- When should the control show validation feedback?
- What should remain visible while a request is pending?
- Which fields should appear after a match?
- How should no match differ from a request failure?
- May the user retry, revise the input, or start another search?
- Which generated-client method and types should carry the request and response?
- Which task remains blocked until the backend OpenAPI document passes review?

If an answer would change observable behavior, return to the frontend delta rather than asking Copilot to choose. If an answer concerns component boundaries, cancellation strategy, or test placement, check design.md and nearby Angular code. If the answer concerns a request schema, response field, or status code, reopen the backend contract and its owning change.

Also inspect the worktree, repository instructions, package scripts, Angular version, generated client configuration, and nearest comparable interaction. Find the local pattern that already handles a form submission, an asynchronous request, or a generated service. Prefer that pattern unless the reviewed design calls for a different boundary.

10.4. Confirm that the contract is ready to consume

Treat the OpenAPI document as a reviewed implementation output from Chapter 9. Record the backend revision or handoff identifier that supplied it, then compare the document with the frontend scenarios before generation.

Use a short intake check:

Frontend need	Contract question	Stop when
Valid submission	Does the operation accept the reviewed unit-number input?	The request location or constraint differs

Frontend need	Contract question	Stop when
Matching result	Can the success schema supply every displayed field?	The UI would need an invented or duplicate field
No match	Can the generated client preserve the reviewed no-match meaning?	No match becomes indistinguishable from failure
Invalid input	Does the contract describe the reviewed validation response?	Frontend and backend rules disagree
Failure	Can reviewed failures reach distinct UI handling where required?	The generated method erases information needed by the scenario

Do not repair a contract mismatch in a hand-written TypeScript interface. Return a missing request constraint, response property, or error distinction to `add-unit-number-search-api`; review the backend correction; regenerate the document; and only then resume the frontend task.

Before the final contract arrives, limit parallel UI work to boundaries that the accepted frontend behavior already settles, such as presentation structure or pure state transitions. Keep generated-client integration blocked until the reviewed document becomes available, and do not let a temporary mock harden into a second contract.

10.5. Start a bounded apply session

Name the frontend change explicitly:

```
/opsx:apply add-unit-number-search-ui
```

When the first unchecked task still spans several concerns, narrow the request around one scenario and one check:

```
/opsx:apply add-unit-number-search-ui
```

Apply the next unchecked frontend task from the active change.

Before editing:

- read `proposal.md`, the `unit-number-search` delta specification, `design.md`, and `tasks.md`;
- inspect the reviewed OpenAPI handoff, generated-client configuration, nearest Angular implementation, and nearest focused tests;
- identify the frontend scenario this task should satisfy; and
- name one focused check that could reject the proposed edit.

During implementation:

- use the generated request, response, and service types;
- keep transport details out of the component where the repository design already provides a service or facade boundary;
- preserve unrelated worktree changes;
- keep VIN, partial search, and other deferred behavior outside this change; and
- pause rather than invent behavior when an artifact or contract remains silent.

After the first edit, run the focused check. Mark the task complete only when the implementation, assertion, and command output agree with the governing scenario. Report any contradiction and name the artifact that should receive the correction.

Stop after one task so that the first failing check can expose a wrong assumption before it spreads through generated types, a service, component state, template markup, and browser tests. When two edits must land together to compile, ask Copilot to name that dependency and run the narrowest test that crosses it.

10.6. Generate the Angular client without creating a second contract

Use the repository's configured generation command and version. Check the package script, generator configuration, output directory, naming options, and local backend URL before running anything. Avoid borrowing a one-off command from another project when the prepared repository already records those choices.

After generation, inspect the diff before writing component code. Ask:

- Did the generator create the expected operation and models?
- Do required fields and constraints match the reviewed document?
- Can the generated return type represent match and no-match behavior correctly?
- Can application code observe the errors distinguished by the frontend scenarios?
- Did generation touch only the configured output and supporting metadata?
- Could a clean checkout reproduce the same output with the recorded command?

Keep generated files unedited. Put mapping, presentation, and interaction decisions in hand-written Angular code. Regenerate after a reviewed backend contract change or generator configuration change; do not preserve a local patch inside generated output.

 Do not check off contract intake after generation alone

Ask more than “Did the command exit successfully?” Compare the generated operation, types, and error surface with the reviewed scenarios. Leave the task open when generated code compiles but cannot represent the accepted behavior.

10.7. Follow one scenario through Angular

Choose a scenario before choosing files. For a matching unit number, trace the path from the delta to the rendered result:

```
frontend delta scenario
  |
  v
input state -> submit event -> hand-written boundary -> generated client
                        |
                        v
rendered result <- view state <- response mapping <- backend contract
|
```

Use the trace to locate existing owners, not to require a new layer at every arrow. If the repository already calls generated services through a facade, extend that facade. If the local design keeps a small interaction in the component, avoid introducing a store merely because the diagram contains a state step.

10.7.1. Bind input behavior to the scenario

Implement only client-side rules approved by the frontend delta. Keep immediate feedback useful, but do not present it as final API enforcement. A stale browser, a direct API caller, or a changed contract can bypass the Angular validator, so keep backend validation authoritative.

Test the distinction between these paths:

- input that the accepted rule rejects before submission;
- valid input that produces no match;
- valid input that produces a matching result; and
- a request that fails after valid submission.

Ask whether the component prevents an invalid request, displays feedback at the specified time, and clears or retains prior state exactly as the scenario requires. Avoid deriving new rules from an HTML attribute or generated model comment when the delta remains silent.

10.7.2. Make request transitions visible

Model only states that a user or test needs to distinguish. Check the initial, loading, result, empty, and error paths against the actual delta rather than treating this list as a product decision.

For each transition, settle what should happen to prior output. Record whether a second submission clears an old result, whether the UI ignores a stale response after a newer request finishes, and whether an error preserves the entered unit number for correction or retry. Put accepted answers in the frontend delta or design before encoding them in RxJS operators, signals, or component fields.

Use the Angular mechanism already chosen by the repository and reviewed design. Whether the code uses signals, observables, reactive forms, or another established pattern, require the same observable outcomes and test them at the interaction boundary.

10.7.3. Keep transport details behind the generated boundary

Route requests through the generated client and the repository's hand-written service or facade when one exists. Do not duplicate URL paths, serialize a parallel request body, or cast an unrecognized response until TypeScript stops objecting.

When the generated API feels awkward, inspect the OpenAPI document and generator configuration first. Wrap a transport-shaped method behind a small application boundary when the contract remains correct; return to the backend owner when the contract itself fails to express the reviewed behavior.

10.8. Let focused tests control task progress

Choose each test for the defect it could expose:

Boundary	Question the test should answer	Possible evidence
Pure validation or state logic	Does this transition follow the reviewed rule?	Focused Vitest test where the repository isolates pure logic
Angular component	Does user interaction produce the specified visible state?	Component test with controlled service results
Hand-written service or facade	Does application code map generated responses without changing their meaning?	Focused service test
Browser-to-backend path	Do both applications satisfy the accepted scenario together?	Playwright assertion against visible output

Stub the generated client at the narrowest repository-approved boundary. Make each stub represent a reviewed contract outcome, and compare it with the generated type. Do not let an untyped object give the component fields the backend never promised.

Assert what the user can observe: disabled or rejected submission, loading indication, rendered result, empty state, error state, and recovery behavior where specified. Also assert the request boundary when absence matters; for example, prove that invalid input did not call the API rather than merely checking that an error message appeared.

After every bounded edit, report progress in a form such as this:

```
Task: 2.2 Render the reviewed no-match state
Governing scenario: valid unit number with no matching truck
Implementation boundary: component view state
Command run: <repository-configured focused test command>
Result: pass
Remaining gap: browser-to-backend verification belongs to task 2.4
Task checkbox updated: yes
```

Keep the checkbox open when the test fails, when the assertion observes an implementation detail instead of the scenario, or when the generated client no longer matches the reviewed contract. Split a task in tasks.md when independent outcomes otherwise invite a misleading partial checkoff.

10.9. Return contradictions to their owners

When apply exposes a mismatch, ask which artifact owns the decision:

Discovery during apply	Return to	Avoid hiding it with
The requested interaction exceeds accepted scope	proposal.md	An extra control or search mode
A visible state or transition remains ambiguous	Frontend delta spec.md	A convenient component default

Discovery during apply	Return to	Avoid hiding it with
The planned Angular boundary conflicts with the repository	Frontend design.md	A parallel architecture
A scenario has no implementation or test work	Frontend tasks.md	A broad final verification task
A request, response, or error cannot support the UI	Backend change and OpenAPI document	A duplicate TypeScript model or cast

After repairing an artifact, inspect its dependents before resuming apply. A revised no-match decision may affect response mapping, generated code, component state, assertions, and the cross-application path. Update only the affected artifacts and tasks, review them, then rerun the focused check.

 Do not rewrite the scenario to excuse the component

When code and requirement disagree, ask whether the requirement was wrong, the implementation was wrong, or the team made a new decision. Record and review the answer before changing the delta or checking off the task.

10.10. Resume from artifacts and unchecked tasks

After a pause or a new session, invoke the named change again:

```
/opsx:apply add-unit-number-search-ui
```

Ask Copilot to read the current artifacts, identify the next unchecked task, inspect the latest test evidence, and name the scenario before editing. Do not ask it to reconstruct the plan from a chat transcript. Test the handoff by asking another developer or a fresh session to resume from the repository alone. If that attempt fails, improve the artifact that failed to carry the decision.

10.11. Hands-on exercise

Apply the Angular unit-number change in the prepared frontend repository:

1. Run `openspec\status`, `openspec\validate`, and `openspec\show` for `add-unit-number-search-ui`, then read the four artifact types in dependency order.
2. Confirm the reviewed backend revision and compare its OpenAPI document with every frontend scenario.
3. Identify the first unchecked task, its governing scenario, the nearest Angular boundary, and one focused check that could reject the edit.
4. Run `/opsx:apply\add-unit-number-search-ui` in VS Code, and stop after the task's focused check and evidence report.
5. Generate the TypeScript Angular client through the repository command, inspect its diff, and leave generated files unedited.
6. Continue through input, loading, result, empty, error, and recovery behavior only where the reviewed delta and task list require them.

7. Add focused tests for every frontend scenario, then run the configured Playwright path against the backend from Chapter 9.
8. Resume the same change in a fresh session or in IntelliJ IDEA, and judge whether the artifacts and unchecked tasks provide enough context without the previous conversation.
9. Leave unsupported work open, repair the owning artifact or contract, and rerun the affected checks after review.

10.12. Verification checkpoint

Before moving to cross-repository verification, ask:

- Does each checked frontend task point to a scenario and passing evidence?
- Did client generation use the reviewed backend document and repository configuration?
- Do hand-written files import generated request, response, and service types?
- Do focused tests distinguish invalid input, loading, match, no match, and reviewed failures?
- Does the UI avoid stale results or overlapping states according to the accepted behavior?
- Did contradictions return to the artifact or repository that owns them?
- Does the Playwright path exercise the configured Angular and Spring Boot applications together?
- Could a fresh Copilot session identify the next task without replaying the earlier chat?

Let any unanswered question keep the change open. Carry only reviewed artifacts, reproducible generated code, checked tasks with evidence, and a working browser path into Chapter 11.

10.13. What you learned

Apply each pending task from its reviewed frontend scenario, then let a focused test challenge the edit before checking off the task. Generate transport code from the backend-owned contract and keep interaction decisions in hand-written Angular code. When those sources disagree, return the decision to its owning artifact rather than hiding the mismatch in the component.

10.14. Online references

- Angular testing guide¹
- OpenAPI Generator TypeScript Angular generator²
- Vitest guide³
- Playwright documentation⁴

¹<https://angular.dev/guide/testing>

²<https://openapi-generator.tech/docs/generators/typescript-angular>

³<https://vitest.dev/guide/>

⁴<https://playwright.dev/docs/intro>

11. Verifying, Synchronizing, and Archiving

The backend and frontend implementations now exist in separate repositories. Before closing either change, test a stronger claim than “the code passes.” The artifacts must be valid, the code must match the accepted scenarios, the two applications must agree on the contract, and the main specifications must receive the behavior that shipped.

OpenSpec separates that work into distinct checks. Use artifact validation to examine the change’s structure. Use implementation verification to compare code and tests with the accepted artifacts. Review synchronization when archival merges the delta into the main specs. Then inspect the archive as the retained history of the decision. None of these checks substitutes for the others.

11.1. Learning objective

By the end of this chapter, you should be able to use `openspec\validate` and `/opsx:verify` for their different purposes, review the main-spec update produced during archival, and preserve the reason and identity of a change after active work ends.

11.2. Treat completion as a claim

Break “the change is done” into claims that repository evidence can support:

Completion claim	Evidence	Limit
The artifacts follow OpenSpec’s structure	<code>openspec\validate\TU\textless{ }change\TU\textgreater{ }</code> output	Does not test application behavior
The implementation matches the accepted change	<code>/opsx:verify</code> , code review, and scenario-focused tests	Does not prove that two running applications work together
The integrated path preserves the contract	A browser-to-backend test and its output	Does not update the main specs
The main specs contain the accepted delta	The archive preview and reviewed spec diff	Does not prove that tests passed
The decision remains recoverable	Archived artifacts, identifiers, and a fresh-session check	Does not revisit whether the original decision was correct

Run these checks in order. Failed artifact validation leaves implementation review without a stable target. A missing implementation scenario makes the proposed main-spec update premature. An unexpected spec diff should stop archival before it records the wrong behavior.

Capture the result in the pull request, issue, or team review record:

```
Change: add-unit-number-search-api
Artifact validation: pass, <command or CI run>
Scenario verification: pass, <focused test commands>
Cross-repository path: pass, <browser test or recorded run>
Main-spec update: reviewed, <commit or diff>
Archive path: openspec/changes/archive/<date>-add-unit-number-search-api/
Related change: add-unit-number-search-ui
Work item: <Jira issue>
```

Keep each entry traceable to repository evidence. Do not create another OpenSpec artifact merely to repeat information already recorded by the team's review process.

11.3. Reestablish the change boundary

Start by recovering the accepted scope from each repository. In this lab, the backend change is add-unit-number-search-api, and the frontend change is add-unit-number-search-ui. A shared Jira issue or cross-repository identifier connects them as one product change without erasing repository ownership.

Run the inspection commands from the root of each repository:

```
openspec status --change add-unit-number-search-api
openspec show add-unit-number-search-api
openspec validate add-unit-number-search-api --strict

openspec status --change add-unit-number-search-ui
openspec show add-unit-number-search-ui
openspec validate add-unit-number-search-ui --strict
```

Use status to inspect artifact completion, show to read the change and its deltas, and validate to find structural problems such as a malformed delta or a requirement without a scenario. These commands do not prove that Angular sent the expected request or that Spring Boot returned the accepted response. Implementation verification covers that question.

Read the artifacts after the commands finish. Do not let a successful exit code replace checking that the CLI resolved the intended OpenSpec root, selected the intended change, and examined the files just reviewed. In a repository with several active changes, validating the wrong change provides reassuring but irrelevant output.

i Validation checks artifacts, not application behavior

openspec\validate checks structures and relationships covered by the OpenSpec rules. It does not run the application's scenario path or decide whether a requirement expresses the team's intent. Keep its output as one part of the completion evidence.

11.4. Verify the implementation against the artifacts

Open each repository in turn, select the named change, and run OpenSpec's verification skill:

Review its findings under completeness, correctness, and coherence. Check that every task is complete, each requirement has corresponding code and tests, and the implementation still follows the reviewed design. The skill reports findings but does not block archival, so decide how to resolve each warning before proceeding.

If the frontend report finds all tasks checked but no focused test for the no-match scenario, keep the change open and add the missing evidence. A user-visible state in the accepted delta deserves a stronger response than an optional naming cleanup.

Use the report to guide inspection, then confirm its claims in the repository. An agent can miss a test hidden behind an unfamiliar convention, infer correspondence from similar names, or accept a checked task without reproducing its command. Connect every accepted scenario to code and to a check that fails when the behavior breaks.

11.4.1. Build a scenario-to-evidence map

Map the unit-number scenarios before running a broad final suite:

Accepted scenario	Backend evidence	Frontend evidence	Cross-repository evidence
Invalid unit number	Request validation test	Submission and feedback test	Browser confirms that invalid input does not produce a search result
Matching unit	Controller or service test for the reviewed response	Component test for the result state	Browser displays data returned by the running backend
No matching unit	API test for the reviewed no-match contract	Component test for the empty state	Browser distinguishes no match from request failure
Request failure	Error-contract test where specified	Component test for the error and retry path	Integrated test or controlled failure evidence where practical

Include only scenarios from the accepted delta specifications. If the change does not define retry behavior, omit that assertion or return to the change for a reviewed decision before implementing it.

Run focused checks before broad suites. Use each repository's package, build, and test configuration rather than inventing generic commands. Preserve the exact commands and results in the review record:

```
Backend scenario: matching unit
Command: <repository-configured focused Spring Boot test>
Result: pass

Frontend scenario: no matching unit
Command: <repository-configured focused Angular test>
Result: pass

Integrated scenario: matching unit from browser through API
```

```
Command: <repository-configured Playwright test>
Result: pass
```

11.4.2. Follow the contract across repositories

Review the cross-repository path because each repository can pass in isolation while the product path fails. Start with the backend-owned OpenAPI document and confirm that it generated the Angular client under test. Then trace one accepted example through the browser, generated client, HTTP boundary, backend implementation, and rendered result.

For a matching unit, check these handoffs:

1. Confirm that the browser submits the input accepted by the frontend delta.
2. Confirm that the generated client sends the request described by the reviewed OpenAPI operation.
3. Compare the backend status and response shape with the API document.
4. Trace the response through the frontend without introducing another contract.
5. Compare the rendered fields and state with the frontend scenario.

Repeat the path for distinctions that often disappear at a repository boundary, especially no match versus request failure. Backend tests can distinguish their status codes, and component tests can distinguish views built from mocked values. Use the integrated path to show that the generated client carries the distinction between them.

When a mismatch appears, repair the boundary that owns it. A backend response that contradicts the reviewed OpenAPI document belongs in the backend change. A correct generated response that produces the wrong visible state belongs in the frontend implementation or delta. Do not change the main specification merely to match whichever behavior happened to ship.

11.5. Decide whether the change is ready to close

Make the completion decision against explicit conditions:

- both named changes pass strict artifact validation;
- every accepted scenario maps to implementation and appropriate test evidence;
- the reviewed OpenAPI document matches the client used by the frontend;
- the configured backend, frontend, and cross-repository checks pass;
- checked tasks correspond to completed work rather than planned or partial work; and
- the team has repaired unresolved warnings or recorded them as accepted limits.

Each OpenSpec root can archive its own change. For one user-facing feature, keep both changes active until the product path passes. If the repositories must close at different times, record that decision and link their completion states.

 A checked task is not test evidence

A checkbox records workflow state, not test evidence. Keep the command and result available where the team reviews completion.

11.6. Understand synchronization during archive

In the current OpenSpec CLI workflow, archiving normally performs the specification update. The command validates the change, previews its effect on `openspec/specs/`, merges the delta after confirmation, and moves the change directory under `openspec/changes/archive/`.

Here, synchronization means reviewing that transfer from active delta to main specification. Keep code generation, deployment, and cross-repository contract exchange in their own workflows. Never rewrite a requirement during archival simply to match the implementation.

Before running archive, inspect the relevant main spec and active delta side by side:

- identify every requirement to be added, modified, removed, or renamed;
- match modified requirements to their identities in the main spec;
- check that no condition or scenario disappears during the merge;
- remove behavior that was not accepted for this change; and
- compare the resulting main-spec behavior with the implementation just verified.

Then run archive interactively so that its proposed spec updates remain visible:

```
openspec archive add-unit-number-search-api
```

Review the preview before confirming it. After the command completes, inspect the repository diff:

```
git diff -- openspec/specs openspec/changes
```

Repeat the process in the frontend repository with its own change:

```
openspec archive add-unit-number-search-ui  
git diff -- openspec/specs openspec/changes
```

Expect the diff to show the accepted requirements in the main specs and the dated change directory under the archive. Stop before commit if it contains unrelated specification edits, missing scenarios, or an unexpected capability name.

For agents or CI, `-V=yes` can answer archive confirmations non-interactively:

```
openspec archive add-unit-number-search-api --yes
```

`-V=yes` removes the prompt, not the review obligation. Inspect the proposed update beforehand or review the resulting diff before accepting the commit.

Reserve `-V-skip-specs` for changes that do not update main specifications, such as some tooling or documentation changes. Do not use it for the unit-number feature: that would archive the delta while leaving the main source of truth behind. Likewise, do not use `-V=no-validate` to bypass a malformed change.

11.7. Inspect what archival preserved

An archive earns its place by making the decision recoverable. Open the dated directory and check for the material a future reviewer needs:

- the proposal explaining the reason and scope;
- delta specifications containing the reviewed scenarios;
- the design decisions and boundaries used during implementation;
- the completed task history;
- the Jira issue and cross-repository identifier; and
- any repository convention that points to test, pull request, or release evidence.

Keep current behavior in the main specs and change history in the archive. Packing discarded options into the main spec makes current behavior harder to read. Treating the archive as a moved folder makes past decisions harder to recover.

After both archive operations, compare their identifiers and resulting main specs. Keep API behavior in the backend spec and user-visible interaction in the frontend spec. Use the shared identifier to connect them without duplicating either specification.

11.8. Respond to failed checks without hiding them

Expect verification to reopen work that appeared finished. Repair it while the change remains active and its intent remains nearby.

Finding	Likely next step	Reason not to archive yet
openspec\validate reports a malformed scenario	Repair the delta structure and rerun validation	The implementation target cannot be validated
/opsx:verify finds an accepted scenario with no code	Implement or revise the accepted scope through review	The change claims behavior the code lacks
Focused test fails	Repair the owning implementation boundary and rerun the same check	The evidence rejects the completion claim
Integrated path collapses no match into failure	Inspect the OpenAPI and generated-client boundary	Isolated tests did not preserve the contract distinction
Archive preview changes an unrelated requirement	Cancel, inspect requirement identity and delta content, then retry	The proposed main-spec update exceeds the accepted delta
One repository remains incomplete	Keep both changes active or record a reviewed split-close decision	The shared product change lacks agreed completion evidence

Do not edit checkboxes, main specs, or archive contents merely to silence a warning. Change the artifact or implementation that owns the mismatch, review the correction, and repeat the smallest check capable of rejecting it. Run broader validation after the focused check passes.

11.9. Run the fresh-session test

After archival, start a Copilot session without the transcript from Chapters 8 through 10. Point it to the repository artifacts:

From the current main specs and archived OpenSpec changes:

1. What unit-number search behavior is currently accepted?
2. Which archived change introduced it, and why?
3. Which scenarios distinguish invalid input, no match, and request failure?
4. Which related change belongs to the other repository?
5. What evidence would you inspect before modifying this behavior?

For each answer, name the artifact that supports it. Report anything the repository does not say.

Evaluate the answer against the sources. Look for current behavior in the main specs, the reason in the archive, and the relationship between backend and frontend changes in their shared identifier. A missing answer exposes a decision left in chat history, personal memory, or an unlinked issue.

Do not repair every weak answer by packing more prose into the main specification. Put change-specific reasoning in the archive, implementation detail in the code or design, and execution evidence in the team's verification record. Require the session to cite the right source and admit what the repository does not say.

11.10. Hands-on exercise

Close the unit-number search across the prepared backend and frontend repositories:

1. Record the two change names, their shared Jira or cross-repository identifier, and the main specs each delta would affect.
2. Run `openspec\status`, `openspec\show`, and `strict openspec\validate` for each named change.
3. Run `/opsx:verify` in each repository, inspect its completeness, correctness, and coherence findings, and resolve any warning that weakens an accepted scenario.
4. Build a scenario-to-evidence map for invalid input, match, no match, and request failure where those scenarios appear in the accepted deltas.
5. Run the configured focused backend and frontend tests, followed by the browser-to-backend path.
6. Inspect each active delta beside its target main spec, then run `openspec\archive\TU\textless{ }change\ TU\textgreater{ } interactively and review the proposed updates before confirmation.`
7. Inspect `git\diff\ -V -l openspec/specs\openspec/changes` in both repositories. Confirm that the main specs received only the accepted behavior and that each dated archive retained its artifacts and identifiers.
8. Start a fresh Copilot session and ask it to recover the current behavior, original reason, related repository change, and remaining limits from repository artifacts alone.

11.11. Verification checkpoint

Accept the archive commits only when:

- both changes pass strict artifact validation;
- each accepted scenario points to implementation and reproducible evidence;
- the generated frontend client reflects the reviewed backend contract;
- the cross-repository path preserves the distinctions required by both deltas;
- the updated main specs describe the behavior that passed verification;
- the archives preserve the proposal, decisions, tasks, and linking identifiers; and
- a fresh session can recover current behavior and change history without the old transcript.

Keep any unmet condition visible as an open item rather than hiding it behind an archive date.

11.12. What you learned

Use `openspec\validate` to check artifact structure and `/opsx:verify` to compare the implementation with the accepted change. After the code and cross-repository path pass, archive the change and review the delta merged into the main specs. The work is ready to close when a fresh session can recover both the current behavior and the reason it changed from repository evidence.

11.13. Online references

- OpenSpec CLI reference¹
- OpenSpec skills reference²
- OpenSpec concepts³

¹<https://openspec.dev/docs/cli>

²<https://openspec.dev/docs/skills>

³<https://openspec.dev/docs/concepts>

12. Adding VIN Search to a Brownfield System

Suppose the unit-number search from the earlier chapters has shipped. The backend endpoint, OpenAPI document, generated Angular client, form validation, and result states now work together. Then a product owner asks for one more way to find a truck: “Could users search by VIN too?”

The request sounds small, but it leaves product decisions hidden inside the word “too.” Will VIN use the current operation or a separate one? How will a user choose the search mode? Which unit-number behaviors must survive unchanged? If a coding agent answers from nearby code, its implementation may turn local precedent into product policy before anyone reviews the choice.

How might OpenSpec keep those decisions visible? Describe the requested difference instead of reconstructing every choice behind the application. The resulting delta can define VIN behavior, while the proposal and verification plan can protect the accepted unit-number path.

12.1. Learning objective

By the end of this chapter, you should be able to propose an additive VIN-search change in a brownfield system without pretending that the whole system was specified from the beginning. You should also be able to trace that change through OpenSpec artifacts, the backend-owned API contract, the generated Angular client, and regression evidence for the behavior already in use.

12.2. Why brownfield work changes the question

Greenfield design often begins with “What should this system do?” Brownfield work tends to begin with a less comfortable question: “What does this system already do that users or other software might depend on?”

The answer may be scattered across several sources:

- accepted OpenSpec requirements;
- controller and service behavior;
- an OpenAPI document consumed by another repository;
- generated client code;
- frontend validation and visible states; and
- tests that preserve cases no one thought to document elsewhere.

Do not give those sources equal authority by default. Start with accepted specifications and reviewed contracts. Use code and tests to find dependencies, contradictions, and behavior that may need a decision. Then let the delta describe the requested change without claiming to explain the full history of truck search.

⚠ Existing code is evidence, not automatic intent

A current behavior may be intentional, accidental, or obsolete. Treat repository exploration as evidence for review rather than silently converting every code path into a requirement.

12.3. Establish the baseline before describing the delta

Because unit-number search is already accepted, begin there. Read the current main specification, then trace one successful search and one failure or no-match path through each repository. A short baseline table may keep the investigation bounded:

Question	Likely source	What to record
How does a user choose and submit unit-number search?	Angular specification, component, and tests	Current control, validation, and visible states
What request does the client send?	Generated client and OpenAPI document	Operation, parameter shape, and response types
How does the backend distinguish invalid input, no match, and failure?	API specification and focused tests	Observable statuses and response bodies
Which behavior has already been accepted?	Main OpenSpec specifications	Requirement names and scenarios that should survive

Keep the table small enough to support the VIN decision. When exploration reaches unrelated persistence rules or deployment topology, ask whether the detail could change observable VIN behavior. Leave it outside the change when the answer is no.

The baseline may uncover contradictions. Perhaps the main specification describes one no-match response while the implementation returns another. Would the VIN proposal be the right place to resolve that discrepancy? Only if the team deliberately expands the scope. Otherwise, record the mismatch as separate work and avoid copying it into the new VIN requirement.

12.4. Decide whether VIN search is additive

An ADDED delta would fit when VIN introduces another search path while the accepted unit-number path remains valid. A MODIFIED delta would fit if the request changes the meaning of an existing requirement, such as replacing a unit-number-only input with a single identifier field whose rules now vary by mode.

Do not infer the delta operation from the screen layout. One form could expose two independent capabilities, while two controls could call a single revised API operation. Choose ADDED or MODIFIED from the observable contract and the identity of the accepted requirement.

For this chapter, assume the reviewed direction is additive:

- a VIN search accepts a full 17-character VIN;
- a user can choose unit-number or VIN mode;
- the existing unit-number path keeps its accepted behavior; and
- result and error behavior may be shared only where the accepted specifications say so.

Those assumptions should remain visible in the proposal. If the product owner has not decided case normalization, whitespace handling, disallowed VIN characters, no-match behavior, or the fields shown in a result, use open questions rather than letting Copilot infer answers.

12.5. Explore the extension points

Once the baseline is clear, exploration can focus on the places where VIN behavior might join the system. In the backend repository, inspect the accepted search requirement, HTTP boundary, search service, persistence query, OpenAPI generation, and focused tests. In the frontend repository, inspect the mode control, form state, generated client boundary, result rendering, and component or browser tests.

Frame the exploration prompt around decisions and boundaries:

```
/opsx:explore
```

```
We need to add VIN search while preserving the accepted unit-number search.
```

```
Please trace the current unit-number path through this repository and report:
```

- the accepted requirements and scenarios that constrain the change;
- the nearest extension points for a VIN path;
- assumptions in code or tests that only permit unit-number search;
- generated files or external contracts that should not be edited by hand; and
- unresolved product decisions that would change observable behavior.

```
Do not modify files or convert current implementation details into requirements.
```

A shared opening prompt may work in both repositories. Follow it with backend questions about the API contract and query behavior, then frontend questions about mode selection, validation, generated types, and visible states.

The result should be a decision record, not a dump of matching filenames. For example:

Topic	Status	Decision or question
VIN length	Accepted for this exercise	Require exactly 17 characters
Search mode	Accepted for this exercise	Let the user choose unit number or VIN
Unit-number behavior	Preservation boundary	Keep accepted scenarios unchanged
VIN normalization	Open	Decide whether case or surrounding whitespace is normalized
VIN no-match response	Open until reviewed	Reuse existing behavior only if the contract permits it
Client generation	Existing constraint	Regenerate from the reviewed backend OpenAPI document

12.6. Coordinate one product change across two repositories

The earlier lab chapters established a separate OpenSpec root in each repository. Continue that model here with one active backend change and one active frontend change. Give the backend change ownership of VIN API behavior and contract publication; give the frontend change ownership of mode selection, input feedback, and rendering.

OpenSpec now also documents beta stores for planning one change across several repositories.¹ A team using that model could keep the shared artifacts in a store and expose them to both code repositories. The ownership and contract questions in this chapter would still apply, even though the artifact layout would differ.

Possible change names might be:

```
commercial-truck-search-api/
`-- openspec/changes/add-vin-search-api/

commercial-truck-search-ui/
`-- openspec/changes/add-vin-search-ui/
```

A shared work-item identifier could connect the changes. Their proposals should also name the handoff directly: the frontend client generation waits for a reviewed backend OpenAPI document. That dependency may slow parallel work, but it reduces the chance that the repositories invent different VIN contracts.

The backend proposal could define this boundary:

```
# Change: Add VIN search API

## Why

Users need to locate a truck when a VIN is available instead of a unit number.

## What Changes

- Add API behavior for search by a full 17-character VIN.
- Publish the reviewed VIN request and response behavior in OpenAPI.
- Preserve accepted unit-number search behavior.

## Non-Goals

- Combined VIN and unit-number queries
- Partial VIN search
- Changes to accepted unit-number validation or responses

## Dependencies

- The frontend change may regenerate its client after review of the updated OpenAPI document.
```

The frontend proposal could remain narrower:

¹ See the OpenSpec Stores User Guide at <https://github.com/Fission-AI/OpenSpec/blob/main/docs/stores-beta/user-guide.md>.

```
# Change: Add VIN search mode
```

```
## Why
```

Users need to choose VIN when that is the identifier available for a truck.

```
## What Changes
```

- Add a control for choosing unit-number or VIN search.
- Apply the reviewed validation rule for the selected mode.
- Send VIN searches through the generated API client.
- Preserve the accepted unit-number interaction.

```
## Non-Goals
```

- Handwritten API request or response types
- Simultaneous search by both identifiers
- Changes to accepted result fields unless separately reviewed

Notice what these proposals do not decide. Neither chooses an Angular component structure, Java method name, database index, or generated-client class. Those choices may belong in design.md after the team agrees on behavior.

12.7. Write deltas that expose the additive boundary

The backend delta should make VIN behavior testable without copying the complete unit-number specification. With unresolved details still visible, it might begin like this:

```
## ADDED Requirements
```

```
### Requirement: Search by VIN
```

The API SHALL accept a full 17-character VIN and return the reviewed search outcome.

```
#### Scenario: VIN has the required length
```

- ****WHEN**** a client searches with a VIN containing exactly 17 characters
- ****THEN**** the API evaluates the VIN according to the reviewed search rules

```
#### Scenario: VIN has the wrong length
```

- ****WHEN**** a client searches with a VIN containing fewer or more than 17 characters
- ****THEN**** the API returns the reviewed validation response

```
#### Scenario: No truck matches the VIN
```

- ****WHEN**** a client searches with a valid VIN that matches no truck

- ****THEN**** the API returns [the reviewed no-match response]

Do not add a second copy of the accepted unit-number requirement merely to say that it remains in force. Keep preservation visible in the proposal's non-goals, implementation tasks, and regression evidence. If VIN work changes the accepted unit-number contract after all, revise the proposal and write a complete MODIFIED requirement for that behavior.

The frontend delta might separate mode selection from mode-specific validation:

```
## ADDED Requirements
```

```
### Requirement: Select a search mode
```

The interface SHALL let the user choose unit-number or VIN search mode.

```
#### Scenario: User selects VIN mode
```

- ****WHEN**** the user selects VIN mode
- ****THEN**** the interface applies the reviewed VIN input and validation behavior

```
#### Scenario: User returns to unit-number mode
```

- ****WHEN**** the user selects unit-number mode
- ****THEN**** the interface applies the accepted unit-number behavior

```
### Requirement: Validate a VIN search
```

The interface SHALL accept a full 17-character VIN for submission.

```
#### Scenario: VIN is not 17 characters
```

- ****WHEN**** the user attempts to submit a VIN with fewer or more than 17 characters
- ****THEN**** the interface prevents submission and shows the reviewed validation feedback

Would switching modes clear the input, retain it, or revalidate it? Would the visible label and accessible name change with the selected mode? Would a VIN result show the same fields as a unit-number result? Each answer could affect a user-visible scenario, so unanswered cases should remain open rather than falling into component code by accident.

12.8. Design the contract handoff

The backend should continue to own the wire contract. After implementing the reviewed VIN API behavior, generate or update the OpenAPI document and compare it with the delta before publishing it to the frontend repository.

At minimum, inspect:

- how the operation distinguishes VIN from unit-number search;
- whether the VIN constraint appears in the request schema;

- which success, no-match, validation, and failure responses are documented; and
- whether existing unit-number schemas or operations changed unexpectedly.

Only then should the frontend regenerate its TypeScript client. Editing generated code by hand might make a local build pass, but the next generation would erase the fix and conceal disagreement with the backend contract.

This sequence need not block all frontend work. The team could implement the mode control and local validation against the reviewed frontend delta while waiting for the contract. Integration through the generated client, however, should wait until the backend-owned document has passed review.

12.9. Apply the change in dependency order

A task list can make that partial parallelism visible:

1. Add backend scenarios for valid VIN, invalid length, match, and the reviewed no-match behavior.
2. Implement the VIN path without changing accepted unit-number behavior.
3. Generate and inspect the OpenAPI document, including a diff of existing operations.
4. Add the frontend mode control and mode-specific validation.
5. Regenerate the Angular client from the reviewed OpenAPI document.
6. Connect VIN submission and result handling through the generated client.
7. Run focused backend, frontend, and browser checks for VIN and unit-number paths.

At each step, compare the implementation with the relevant requirement. Treat a checked task as a work record, not proof that the resulting behavior matches the delta.

Copilot may suggest consolidating both search modes behind one helper or component state model. Before accepting the refactor, check whether the abstraction preserves each mode's validation and error rules. Two explicit paths may be easier to verify when the only gain would be less repeated code.

12.10. Verify both the addition and the preservation claim

How would the team know that the change remained additive? Pair each new-path check with evidence from the accepted path at the boundary most likely to regress.

Claim	Focused evidence
A full 17-character VIN can be submitted	Backend boundary test and frontend validation test
A VIN with the wrong length is rejected	API validation test and component feedback test
A matching VIN reaches the expected result state	Backend search test and browser-to-backend test
A valid VIN with no match follows the reviewed contract	API and frontend empty-state tests
A user can change modes without stale validation	Component interaction test
Existing unit-number behavior remains available	Existing backend, component, and integrated regression tests
The repositories share one wire contract	Reviewed OpenAPI diff and regenerated-client diff

Run the narrow tests first. If a VIN validation test fails, repair that boundary and rerun the same test before broadening the suite. Once the focused checks pass, run the repository-configured test suites and the integrated path.

Also inspect the diff, especially generated artifacts and existing unit-number files. An unexpected change to a unit-number response type may reveal scope drift even when mocks allow both repositories to pass independently.

i Preservation needs named evidence

“Unit-number search still works” would be difficult to review without scenarios and test output. Name the accepted paths that matter, then retain the commands or CI runs that exercised them.

12.11. Hands-on exercise

Use the two lab repositories to propose, apply, verify, and archive VIN search. Keep a compact change record that lets another reviewer follow the work without replaying the chat:

```
Product change: add-vin-search
Backend change: add-vin-search-api
Frontend change: add-vin-search-ui
Preserved capability: unit-number-search
Contract handoff: <OpenAPI artifact or commit>
Focused VIN evidence: <commands and results>
Unit-number regression evidence: <commands and results>
Cross-repository evidence: <browser test or recorded run>
Open questions: <owners and blocking status>
```

Before applying either change, ask another reviewer to trace one VIN scenario from proposal to task and one unit-number scenario from accepted specification to regression evidence. Any broken trace would suggest that the change is not ready for implementation.

12.12. Verification checkpoint

The pair of changes may be ready to archive when every reviewed VIN scenario has implementation and test evidence, both OpenSpec changes pass strict validation, and the integrated path uses the generated client from the reviewed contract. The OpenAPI diff should contain the intended VIN extension without unexplained changes to existing unit-number behavior.

If verification exposes an unanswered product question, keep the changes open. Revise the delta when the intended behavior was incomplete, or repair the implementation when it departed from the reviewed behavior. Archiving should wait until both repositories describe the same feature and the preservation claim has evidence.

12.13. What you learned

Specify the VIN difference without rewriting the application's history. Keep unchanged unit-number requirements in the accepted specification, then protect them through explicit scope, contract review, and regression evidence.

12.14. Online references

- [OpenSpec overview](#)²
- [OpenSpec guidance for existing projects](#)³
- [OpenAPI Specification](#)⁴
- [Playwright test assertions](#)⁵

²<https://github.com/Fission-AI/OpenSpec/blob/main/docs/overview.md>

³<https://github.com/Fission-AI/OpenSpec/blob/main/docs/existing-projects.md>

⁴<https://spec.openapis.org/oas/latest.html>

⁵<https://playwright.dev/docs/test-assertions>

Part IV.

Part IV: Change Under Pressure

13. Modifying Accepted Behavior

The unit-number search from the earlier chapters has already passed review, reached both repositories, and entered the accepted OpenSpec specification. Now suppose a product owner asks for a smaller limit: “Could unit numbers have a maximum of 10 characters instead of requiring 15?”

That request may look like a one-line validation edit. Adding a second requirement, however, would leave two current rules for the same input: one requiring 15 characters and another allowing no more than 10. Developers, reviewers, and later Copilot sessions would have to guess which one wins.

OpenSpec treats this situation as a modification. The change would identify the accepted requirement, provide its complete replacement, and preserve every scenario that should remain part of that requirement. After archive, the main specification should describe one current rule, without asking readers to reconstruct precedence from change dates.

13.1. Learning objective

By the end of this chapter, you should be able to decide when a request modifies accepted behavior rather than adding a separate capability. You should also be able to write a complete MODIFIED requirement, trace its consequences through two repositories, and verify that archive would leave one coherent source of truth.

13.2. Recognize a replacement

Chapter 12 added VIN search without changing the identity of unit-number search. VIN introduced a new way to find a truck, so its requirement could sit beside the accepted unit-number requirement. The request in this chapter behaves differently. A unit number cannot remain both exactly 15 characters and at most 10 characters under the same conditions.

Before choosing a delta operation, complete this sentence:

After this change, the accepted requirement named **Search by unit number** should ...

If the answer begins “still apply, plus users can do something separate,” ADDED may fit. If the answer begins “mean something different,” MODIFIED deserves closer attention. Removing the behavior entirely may call for REMOVED, while changing only its identity may call for RENAMED.

Question	Likely operation
Will a separate requirement coexist with the accepted one?	ADDED
Will the accepted requirement keep its identity but change meaning?	MODIFIED
Will the accepted requirement cease to apply?	REMOVED

Question	Likely operation
Will its identity change while its intended behavior remains?	RENAMED

Implementation files cannot settle this choice on their own. Two validators might share a helper even though they implement separate requirements, while one requirement might span an API, generated client, form, and several tests. The operation follows the relationship between accepted and proposed behavior, not the number or shape of files involved.

 A newer sentence does not cancel an older requirement

Appending “Unit numbers SHALL contain no more than 10 characters” under ADDED\Requirements would leave the accepted 15-character rule in force. Neither recency nor proximity supplies replacement semantics. Use MODIFIED when review intends one requirement to supersede its accepted version.

13.3. Recover the accepted baseline

Before drafting a replacement, inspect the requirement as it exists under openspec/specs/. Record its heading exactly, along with every scenario nested beneath it. OpenSpec uses that requirement identity when applying the delta, and those scenarios reveal behavior that a short change request might otherwise erase.

For the lab, the baseline investigation might ask:

- Does the requirement say exactly 15 characters, or no more than 15?
- Does it describe trimming or another form of normalization?
- Which cases cover matching, no match, empty input, and invalid input?
- Do the backend and frontend specifications express the same boundary?
- Which tests provide evidence for each accepted scenario?

A small baseline table can keep evidence separate from decisions:

Topic	Accepted source	Implementation evidence	Decision needed?
Length rule	Main unit-number specification	Backend and frontend boundary tests	Yes, replace 15 with the reviewed rule
Normalization	Accepted scenarios, if present	Controller, form, and tests	Only if the request leaves it unclear
Empty input	Accepted scenarios, if present	Validation tests	Only if the new meaning could differ
Match and no match	Accepted scenarios	Service, component, and browser tests	Usually preserve unless scope says otherwise

If the specification, code, and tests disagree, record the mismatch as a review finding. Do not choose whichever behavior is easiest to edit. The team may resolve the mismatch inside this change when it affects the revised rule, or record separate work when it does not. Either choice should remain visible in the proposal.

13.4. Separate the request from its unanswered questions

“Maximum of 10 characters” answers less than it first appears to. Before proposal, the team still needs to decide:

- whether values shorter than 10 characters qualify;
- whether normalization occurs before the length check;
- how the system handles empty and oversized values; and
- whether case handling, matching, and no-match behavior change with the boundary.

An exploration prompt could make those gaps visible before proposal begins:

```
/opsx:explore
```

```
We are considering changing the accepted unit-number rule from 15 characters to a maximum of 10.
```

```
Please identify:
```

- ```
- the accepted requirement and all of its scenarios;
- every backend and frontend boundary that represents the current length rule;
- generated files and contracts affected by the rule;
- behaviors the request does not settle, including normalization and empty input; and
- tests that provide evidence for behavior likely to remain unchanged.
```

```
Do not modify files or infer product decisions from the current implementation.
```

Organize the resulting notes by what changes, what remains, and what still needs a decision:

| Category             | Example                                                         | Treatment in the change                             |
|----------------------|-----------------------------------------------------------------|-----------------------------------------------------|
| Requested difference | Replace the 15-character rule with a maximum of 10              | State in the proposal and delta                     |
| Preserved behavior   | A valid value with no match keeps the accepted no-match outcome | Restate its scenario in the replacement requirement |
| Open decision        | Whether trimming occurs before the length check                 | Resolve before apply or mark as blocking            |

This separation adds a review step before coding. It also exposes disagreements before the backend and Angular repositories implement different interpretations.

## 13.5. Write the proposal around the behavioral change

Once review settles the open questions, the proposal can state both the intended difference and the preservation boundary. For this worked example, suppose the team chooses these exercise assumptions:

- leading and trailing spaces are removed before validation;
- one through 10 remaining characters are valid;
- an empty value remains invalid;
- 11 or more remaining characters are invalid;
- matching remains exact; and

- a valid value with no match returns no matching trucks.

Those are example product decisions, not consequences of the word MODIFIED. Another team might choose different answers and still use the same OpenSpec operation.

A compact proposal could read:

```
Change: Reduce the unit-number length limit
```

```
Why
```

```
The business now assigns unit numbers containing between one and 10 characters. The search still requires exact matching after removing leading and trailing spaces.
```

```
What Changes
```

- Remove leading and trailing spaces before validating the unit number.
- Accept between one and 10 remaining characters.
- Reject empty values and values longer than 10 characters.
- Preserve exact matching and the existing no-match result.
- Publish the revised constraint through the backend-owned OpenAPI contract.

```
Non-Goals
```

- Changes to VIN search
- Partial or fuzzy unit-number matching
- New result fields or response states

Naming unchanged behavior limits the change. A length edit can invite nearby cleanup or a new matching strategy, but the non-goals and preservation claims give reviewers a boundary for those edits without copying unrelated specifications into the delta.

## 13.6. Replace the complete requirement

A MODIFIED section should contain the whole requirement as it ought to read after archive. It should not contain a patch fragment such as “change 15 to 10,” because that fragment would depend on a reader finding and mentally merging the accepted version.

Using the exercise assumptions, a backend delta might look like this:

```
MODIFIED Requirements
```

```
Requirement: Search by unit number
```

```
The API SHALL remove leading and trailing spaces from a unit-number search value. The remaining value SHALL contain between one and 10 characters, and the API SHALL match it exactly against unit numbers.
```

```
Scenario: Matching unit number is at the maximum length
```

- **\*\*GIVEN\*\*** a truck has unit number `TRUCK-1234`
- **\*\*WHEN\*\*** a client searches for `TRUCK-1234`
- **\*\*THEN\*\*** the API returns that truck

#### Scenario: Matching unit number is shorter than the maximum

- **\*\*GIVEN\*\*** a truck has unit number `TRK-123`
- **\*\*WHEN\*\*** a client searches for `TRK-123`
- **\*\*THEN\*\*** the API returns that truck

#### Scenario: Matching unit number has surrounding spaces

- **\*\*GIVEN\*\*** a truck has unit number `TRUCK-1234`
- **\*\*WHEN\*\*** a client searches for ` TRUCK-1234 `
- **\*\*THEN\*\*** the API returns that truck

#### Scenario: Unit number exceeds the maximum

- **\*\*WHEN\*\*** a client searches for `TRUCK-12345`
- **\*\*THEN\*\*** the API rejects the search value as invalid

#### Scenario: Unit number is empty

- **\*\*WHEN\*\*** a client searches with a value that is empty after removing surrounding spaces
- **\*\*THEN\*\*** the API rejects the search value as invalid

#### Scenario: No truck matches

- **\*\*GIVEN\*\*** no truck has unit number `TRUCK-9999`
- **\*\*WHEN\*\*** a client searches for `TRUCK-9999`
- **\*\*THEN\*\*** the API returns no matching trucks

This example is complete only under the stated exercise assumptions. Compare it with the actual lab requirement before using it. If that requirement contains another match, failure, or response scenario, carry the scenario forward unless the proposal deliberately changes or removes it.

The requirement heading deserves special care. If the accepted heading reads Search\by\unit\number, changing it casually to Validate\by\unit\number may prevent a clean identity match or misrepresent the requirement's scope. Copy the accepted heading, then revise the body and scenarios beneath it.

#### **i** Complete does not mean duplicated

Completeness applies to the requirement being replaced. The delta need not copy every requirement in the capability, nor should it repeat unrelated VIN behavior. Include the revised requirement and all scenarios that should survive within that requirement.

## 13.7. Inspect the delta before implementation

OpenSpec validation can catch malformed delta structure, while a behavioral diff can expose an accidental deletion or an unchanged 15-character clause. Run the repository’s configured checks, then inspect the proposed difference:

```
openspec validate reduce-unit-number-limit --strict
openspec show reduce-unit-number-limit --diff
```

Review the diff against these checks:

- Match the modified heading to the accepted requirement.
- Confirm that archive would replace the 15-character wording rather than retain it beside the new rule.
- Account for every scenario named by the proposal.
- Remove implementation choices, such as Java annotations or Angular validators, from the behavioral specification.
- Confirm that each boundary scenario could fail for the intended reason before implementation.

A passing structural check cannot answer all of those questions. Human review still needs to judge whether the replacement captures the business decision and whether its preservation claims are credible.

## 13.8. Trace one rule through two repositories

The modified requirement may be small, but the old constraint may have crossed several ownership boundaries. Search for the old value and for the concepts that express it. A literal search for 15 could find noise and miss a named constant, a schema facet, or prose that says “fifteen,” so combine value searches with searches for requirement names, validation messages, and test cases.

| Boundary                   | What may need review                             | What would count as drift?                             |
|----------------------------|--------------------------------------------------|--------------------------------------------------------|
| Backend request validation | Length annotation, validator, or request parsing | API still accepts 11 through 15 characters             |
| Search behavior            | Service and persistence tests                    | Matching changes even though the proposal preserves it |
| OpenAPI document           | maxLength, descriptions, and responses           | Published schema still advertises 15                   |
| Generated Angular client   | Regenerated request types or metadata            | Hand edit disappears on regeneration                   |
| Angular form               | Validators, feedback, and submission state       | UI rejects a valid 10-character value or accepts 11    |
| Visible text               | Labels, hints, and errors                        | User sees the obsolete limit                           |
| Automated tests            | Boundary and preserved-behavior cases            | Tests prove only the new happy path                    |

The backend should continue to own the wire contract. After its validation and OpenAPI output reflect the reviewed delta, inspect the contract diff before regenerating the Angular client. Form tests and UI changes may proceed against the reviewed frontend delta, but client integration should wait for the backend-owned contract if generation is the established handoff.

Avoid editing generated client files as though they owned the rule. A manual fix could make the current branch compile, yet regeneration would restore whatever the OpenAPI document still says. The durable correction belongs in the source contract and generation workflow.

## 13.9. Apply in an order that preserves evidence

A task list might keep the dependency visible without prescribing every class or function:

1. Add or revise backend tests for 10-character, shorter valid, 11-character, and empty values.
2. Update backend validation while preserving accepted match, no-match, and error behavior.
3. Generate and review the OpenAPI document for the revised constraint and unrelated drift.
4. Regenerate the Angular client from the reviewed contract.
5. Update Angular validation, visible feedback, and focused component tests.
6. Exercise the integrated path and rerun preserved-behavior tests in both repositories.

Begin with boundary evidence. A test for exactly 10 characters and another for 11 can show whether the implementation changed the intended edge rather than merely replacing one constant. Tests for matching and no match then check the proposal's claim that the rest of the search behavior survived.

During apply, ask Copilot to work from scenario names and report the evidence for each one. A message such as "updated max length everywhere" would reveal little about normalization, generated artifacts, or regression coverage. A scenario-to-test trace gives the reviewer something more specific to inspect.

## 13.10. Verify the replacement before archive

Verification has two jobs: show that the implementation adopted the new boundary and show that the change preserved everything it claimed not to alter.

| Claim                                          | Focused evidence                                                |
|------------------------------------------------|-----------------------------------------------------------------|
| A normalized 10-character value is accepted    | Backend boundary test and Angular form test                     |
| A normalized value shorter than 10 is accepted | Backend and frontend validation tests                           |
| An 11-character value is rejected              | API validation test and visible-feedback test                   |
| Empty input follows the reviewed rule          | Backend and Angular empty-input tests                           |
| Match and no-match behavior remain intact      | Existing service, component, and integrated tests               |
| Both repositories share the revised contract   | Reviewed OpenAPI diff and generated-client diff                 |
| The old rule no longer survives                | Search across specs, code, generated output, tests, and UI text |

Run focused checks first, then the broader repository suites and integrated path. If an 11-character value fails in the UI but reaches the backend through another path, the replacement has not reached every boundary. If all tests pass while the generated schema still reports 15, the repositories still disagree about their contract.

Before archive, inspect the source-of-truth result that synchronization would produce. Confirm that the main specification would contain one complete unit-number requirement with the 10-character maximum and every preserved scenario. If understanding the current rule still requires an older archived change, the active delta remains incomplete.

After archive, repeat the obsolete-assumption search against the main specification and generated artifacts. The dated change record may retain the old rule as history; the active source of truth should not present it as current behavior.

## 13.11. Hands-on exercise

Use the two lab repositories to propose, apply, verify, and archive the revised unit-number rule. Keep a compact record that another reviewer could follow without replaying the Copilot chat:

```
Product change: reduce-unit-number-limit
Accepted requirement: Search by unit number
Old rule: <exact accepted wording>
Reviewed replacement: <exact revised wording>
Preserved scenarios: <scenario names>
Open questions: <owners and blocking status>
Backend evidence: <commands and results>
Contract evidence: <OpenAPI and generated-client diffs>
Frontend evidence: <commands and results>
Post-archive search: <obsolete assumptions found or none>
```

Pause when exploration exposes an unanswered product decision. Revise the delta after that decision receives review, then resume implementation from the updated scenarios. A code-only choice or a corrective chat message would leave the next agent with the same ambiguity.

## 13.12. Verification checkpoint

The change may be ready to close when the modified heading matches the accepted requirement, the delta supplies its complete replacement, and strict validation passes. Focused tests should cover both sides of the new boundary, while regression evidence should cover the scenarios promised as unchanged. The OpenAPI and generated-client diffs should describe the same maximum, and a search should find no current 15-character assumption outside historical records.

If one of those checks fails, keep the change open. Update the artifact that owns the discrepancy, repeat the narrow check, and inspect the would-be source of truth again before archive.

## 13.13. What you learned

MODIFIED means supplying the exact requirement that should exist after archive, not editing only the sentence that changed. Archive can preserve only the scenarios present in that replacement, so an omitted scenario changes the source of truth whether the omission was deliberate or not.

## 13.14. Online references

- OpenSpec overview<sup>1</sup>

---

<sup>1</sup><https://github.com/Fission-AI/OpenSpec/blob/main/docs/overview.md>

- [OpenSpec editing changes guide](#)<sup>2</sup>
- [Jakarta Bean Validation specification](#)<sup>3</sup>
- [Angular forms guide](#)<sup>4</sup>

---

<sup>2</sup><https://github.com/Fission-AI/OpenSpec/blob/main/docs/editing-changes.md>

<sup>3</sup><https://jakarta.ee/specifications/bean-validation/>

<sup>4</sup><https://angular.dev/guide/forms>



## 14. Revising an In-Flight Change

Implementation has started on a Fleet Origin State filter. The backend already handles part of the request when the product owner asks for another filter: Active Status, with values of Active and Decommissioned.

Continuing from that conversation would leave the code ahead of the active proposal. Opening a second change could split one result-filtering rule across two plans. Discarding the first change would lose work that may still fit. Instead, pause implementation and revise the active OpenSpec change until its proposal, delta specifications, design, and tasks describe the new outcome.

### 14.1. Learning objective

By the end of this chapter, you should be able to revise an active OpenSpec change without confusing proposed behavior with accepted behavior. You should also be able to decide whether to revise or split the change, reassess partial implementation, and determine when /opsx:apply can resume.

### 14.2. Treat the active change as the current plan

An OpenSpec change under `openspec/changes/TU\textless{ }change-name\TU\textgreater{ }/` describes behavior that has not yet joined the accepted specifications. Some tasks may already be checked, but archive has not made the delta part of `openspec/specs/`.

That distinction matters when the request changes:

| State                               | Where current behavior lives                                               | How a new decision would usually enter                                     |
|-------------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Accepted behavior                   | <code>openspec/specs/</code>                                               | A new change with an ADDED, MODIFIED, REMOVED, or RENAMED delta            |
| Proposed behavior                   | <code>openspec/changes/TU\textless{ }change-name\TU\textgreater{ }/</code> | A revision to that active change, when the scope still belongs together    |
| Implemented but unarchived behavior | Code and tests, plus the active change                                     | A revision to the artifacts followed by reassessment of the implementation |

Suppose the active delta contains an ADDED requirement named `Filter\vehicles\by\fleet\origin\state`. Adding Active Status does not turn that block into a MODIFIED requirement. The team is editing a proposed addition, not replacing an accepted requirement. Chapter 13 used MODIFIED because the target requirement already lived under `openspec/specs/`.

⚠ Do not archive the old plan to make revision easier

Do not archive the old plan merely to create a new baseline. That would move unverified behavior toward the accepted specifications. Keep the change active while its scope and evidence remain unsettled.

### 14.3. Recover the interruption point

Before editing, recover the state of the named change:

For a change named add-vehicle-filters, start with:

```
openspec status --change add-vehicle-filters
openspec show add-vehicle-filters --diff
openspec validate add-vehicle-filters --strict
```

Use status to find the available artifacts and checked tasks. Use show\--V-diff to inspect the proposal and requirement deltas. Use validate to find structural problems. A clean validation result cannot show whether the artifacts agree with the revised request.

Read the files as a set after running those commands. For each artifact, ask a different question:

| Artifact         | Question after the interruption                                               |
|------------------|-------------------------------------------------------------------------------|
| proposal.md      | Does the reason and scope still describe the outcome the product owner wants? |
| specs/**/spec.md | Could every requirement and scenario still be accepted as written?            |
| design.md        | Can the chosen API, query, and UI design represent the revised behavior?      |
| tasks.md         | Which checked tasks still produce evidence for the revised scenarios?         |

Inspect the implementation in the same pass without treating it as the source of the new business rule. Existing query logic may reveal that AND is easy to implement or that an empty parameter already has a meaning. Record the product decision in the change instead of inheriting whichever behavior the code happens to contain.

### 14.4. Name the decision that moved

“Add Active Status” sounds specific until it meets the existing filter. Several decisions may still be open:

- Should Active Status narrow the same result set as Fleet Origin State?
- Should selecting both filters use AND or OR?
- Should one control be optional, or must the user choose both values?
- Should no selection omit a query parameter, send an empty value, or mean “all”?
- Should the result list distinguish no matches from a failed request?
- Should Active Status use a fixed set of values or accept arbitrary text?

Record answers before asking an agent to continue implementation. For this exercise, use the following reviewed decisions:

| Topic                 | Revised decision                                         |
|-----------------------|----------------------------------------------------------|
| Available filters     | Fleet Origin State and Active Status                     |
| Status values         | Active and Decommissioned                                |
| Combination rule      | When both filters have values, a vehicle must match both |
| One empty selection   | Apply only the populated filter                          |
| Both selections empty | Return the unfiltered result set                         |
| No matching vehicles  | Show the existing empty-results state                    |
| Request failure       | Preserve the existing error state                        |

These choices belong to the worked example, not to OpenSpec. Another product could require at least one filter or treat an empty selection as invalid. What matters here is that implementation resumes from a reviewed meaning.

An exploration prompt can help expose omissions without changing either artifacts or code:

```
/opsx:explore add-vehicle-filters
```

```
The active change currently adds Fleet Origin State filtering. The product owner now wants an optional Active Status filter with values Active and Decommissioned.
```

```
Compare the proposal, delta specifications, design, tasks, implementation, and tests. Identify:
```

- assumptions that describe only the original one-filter behavior;
- unresolved combination and empty-selection rules;
- completed work that may still satisfy the revised behavior; and
- generated or cross-repository surfaces that may become stale.

```
Do not edit artifacts or code. Do not infer product decisions from the implementation.
```

Explore mode can expose the questions, but its conversation does not revise the source of truth. Move the reviewed answers into the active artifacts.

## 14.5. Decide whether to revise or split the change

Judge scope from the business outcome rather than the files touched:

| Revise the active change when...                              | Start a separate change when...                          |
|---------------------------------------------------------------|----------------------------------------------------------|
| The new decision completes or corrects proposed behavior      | The new request has value on its own                     |
| Reviewers would evaluate both parts as one outcome            | Either part could be released or withdrawn independently |
| One set of scenarios must describe how the behaviors interact | Combining them would hide different risks or owners      |
| The original behavior no longer satisfies the request         | The active change still meets its reviewed purpose       |

Shared files alone should not force a merge. Two independent changes may both touch a search component, while one coherent filter behavior may span a controller, generated client, Angular form, and test suites in two repositories.

For the exercise, keep Active Status in add-vehicle-filters. The two controls define one result set, and neither filter can explain the combination or empty-selection rules alone. Preserve that reasoning in the proposal:

```
Revision note
```

```
Active Status was added while implementation was in progress. It remains in this change because the status and fleet-origin controls jointly determine one vehicle-search result set. The revised requirements replace the earlier one-filter proposal rather than adding an independent release.
```

Keep the note short. A reviewer needs the scope decision, not a diary of the conversation.

## 14.6. Revise the artifacts as a set

Use OpenSpec's update skill to revise existing planning artifacts without changing application code:

```
/opsx:update add-vehicle-filters
```

```
Revise this active change to add an optional Active Status filter with values Active and Decommissioned. When both filters have values, results must match both. When one filter is empty, apply only the populated filter. When both are empty, return the unfiltered result set. Preserve the existing empty-results and request-error states.
```

```
Show each proposed artifact revision before writing it. Do not modify application code.
```

Review each artifact revision before accepting it. A proposal may have the right scope while its delta omits the both-empty scenario. A correct delta may still conflict with a design that sends empty strings for both parameters.

### 14.6.1. Revise the proposal boundary

Revise the proposal's outcome and boundaries. Its What\Changes section might move from this:

```
What Changes
```

- Add a Fleet Origin State control to vehicle search.
- Send the selected state to the search API.
- Show vehicles from the selected fleet origin state.

to this:

```
What Changes
```

- Add Fleet Origin State and Active Status controls to vehicle search.
- Support Active and Decommissioned status values.

- Apply both populated filters with AND semantics.
- Omit an unselected filter from the request and preserve the unfiltered result when neither is selected.
- Preserve the existing empty-results and request-error states.

## ## Non-Goals

- Multi-select values within either filter
- Changes to vehicle status definitions
- New sorting or pagination behavior

Replace the stale one-filter wording. Do not leave reviewers to decide whether the first or last version wins.

### 14.6.2. Rewrite the proposed requirement completely

Suppose the original delta adds a requirement for Fleet Origin State filtering. Revise that proposed requirement into a complete statement of the combined behavior:

## ## ADDED Requirements

### ### Requirement: Filter vehicle search results

The search UI SHALL allow a user to filter vehicles by Fleet Origin State and Active Status. The available Active Status values SHALL be `Active` and `Decommissioned`. When both filters have values, a returned vehicle SHALL match both values.

#### #### Scenario: Both filters are selected

- **\*\*GIVEN\*\*** Fleet Origin State is `OH`
- **\*\*AND\*\*** Active Status is `Active`
- **\*\*WHEN\*\*** the user applies the filters
- **\*\*THEN\*\*** every returned vehicle has Fleet Origin State `OH`
- **\*\*AND\*\*** every returned vehicle has Active Status `Active`

#### #### Scenario: Only Fleet Origin State is selected

- **\*\*GIVEN\*\*** Fleet Origin State is `OH`
- **\*\*AND\*\*** Active Status has no selection
- **\*\*WHEN\*\*** the user applies the filters
- **\*\*THEN\*\*** every returned vehicle has Fleet Origin State `OH`

#### #### Scenario: Only Active Status is selected

- **\*\*GIVEN\*\*** Fleet Origin State has no selection
- **\*\*AND\*\*** Active Status is `Decommissioned`
- **\*\*WHEN\*\*** the user applies the filters
- **\*\*THEN\*\*** every returned vehicle has Active Status `Decommissioned`

```

Scenario: Neither filter is selected

- **GIVEN** neither filter has a selection
- **WHEN** the user applies the filters
- **THEN** the search returns the unfiltered result set

Scenario: No vehicle matches the selected filters

- **GIVEN** no vehicle matches the selected filter values
- **WHEN** the user applies the filters
- **THEN** the UI shows the existing empty-results state

```

Why keep ADDED here? The filter requirement has not entered the accepted specification, so the team is revising the proposed addition itself. Replacing ADDED with MODIFIED merely because the draft changed would confuse artifact history with delta semantics.

Carry forward any earlier scenarios for resetting filters, preserving pagination, or handling a request failure unless the revision deliberately removes them.

### 14.6.3. Revisit design decisions

Requirements should describe observable behavior, while the design can explain how the two repositories will represent it. A revised design might include decisions such as:

- the backend accepts optional fleetOriginState and activeStatus query parameters;
- the query layer adds a predicate only for a populated parameter;
- two predicates combine with AND;
- the backend-owned OpenAPI document defines the status enum;
- the Angular client is regenerated from that document; and
- the UI leaves an unselected control absent from the request rather than sending an empty string.

Keep framework details in the design rather than repeating them in every scenario. Also record ownership: if Angular invents its own status union before the backend contract changes, both repositories can compile while disagreeing on capitalization or allowed values.

An impact table can make stale assumptions easier to find:

| Surface            | Original assumption      | Revision to review                                     |
|--------------------|--------------------------|--------------------------------------------------------|
| API parameters     | One optional state value | Two optional values with defined omission behavior     |
| Query construction | One state predicate      | Zero, one, or two predicates; two combine with AND     |
| OpenAPI document   | State parameter only     | Status enum and optional status parameter              |
| Generated client   | One filter argument      | Regenerated method or request type for both filters    |
| Angular form       | One select control       | Two controls with independent empty selections         |
| Result states      | State-filtered results   | Combined results plus preserved empty and error states |

#### 14.6.4. Reassess every task checkbox

A checked task records work against the earlier plan. Keep it checked only when its output still supports the revised artifacts.

The task list might change from:

- [x] 1.1 Add the fleet-origin query parameter to the API contract
- [x] 1.2 Implement the fleet-origin query predicate
- [ ] 2.1 Regenerate the Angular client
- [ ] 2.2 Add the Fleet Origin State control
- [ ] 3.1 Add integrated filter tests

to:

- [ ] 1.1 Revise the API contract for optional fleet-origin and active-status parameters
- [ ] 1.2 Implement zero, one, and two-predicate query behavior
- [ ] 1.3 Add backend tests for each filter combination
- [ ] 2.1 Regenerate the Angular client from the revised contract
- [ ] 2.2 Add Fleet Origin State and Active Status controls
- [ ] 2.3 Preserve empty-results and request-error states
- [ ] 3.1 Add integrated tests for empty, single-filter, and combined-filter requests

Reopen both backend tasks if the checked contract and query cannot represent Active Status. Keep a completed request-object task checked if its implementation still satisfies the revised design. Judge the evidence, not the effort already spent.

#### **i** Update changes artifacts, not application code

/opsx:update should revise the existing plan and stop. Resume /opsx:apply only after reviewers accept the updated proposal, requirements, design, and tasks. Keeping those steps separate makes it easier to see whether a code edit came from the reviewed revision or from an agent's inference.

### 14.7. Check coherence before resuming implementation

After editing the artifacts, rerun the three opening checks:

```
openspec status --change add-vehicle-filters
openspec show add-vehicle-filters --diff
openspec validate add-vehicle-filters --strict
```

Ask the update skill for a coherence review without supplying another revision:

```
/opsx:update add-vehicle-filters
```

Look for contradictions among artifacts:

- Does the proposal promise both filters while a scenario still names only one available control?
- Does the design omit empty parameters while a task asks the client to send empty strings?

- Does the status enum have the same spelling in the delta, OpenAPI design, and UI task?
- Does each combination rule have a scenario and corresponding test task?
- Do any checked tasks rely on the superseded one-filter contract?

openspec\validate can reject malformed requirement structures. It cannot decide whether AND reflects the product owner’s request or detect every disagreement between the design and tasks. Compare those artifacts with the recorded decision before resuming implementation.

## 14.8. Reconcile partial implementation with the revision

Once the revised artifacts pass review, classify the existing work before writing more code:

| Finding                | Example                                                   | Likely treatment                                  |
|------------------------|-----------------------------------------------------------|---------------------------------------------------|
| Reusable               | The request model already supports optional filter fields | Keep it and retain supporting tests               |
| Incomplete             | The API accepts fleet origin but not active status        | Extend it under the revised task                  |
| Contradictory          | The query treats an empty value as no results             | Change it before claiming the both-empty scenario |
| Stale generated output | The Angular client exposes only fleet origin              | Regenerate after the backend contract is reviewed |

Do not discard all partial work or treat every completed line as settled. Code may survive unchanged while its test needs another case because it proves only one branch of the revised rule.

Resume implementation by naming the change:

```
/opsx:apply add-vehicle-filters
```

Let the apply workflow read the revised artifacts and work through the pending tasks. Mark a task complete only after its implementation and focused checks pass. If coding exposes another unanswered business question, pause apply and return to the artifact that owns the decision.

Suppose the generated client represents an unselected status as undefined, while the backend distinguishes a missing parameter from an empty string. Treat that mismatch as a contract question. Resolve the intended request behavior, revise the delta or design, and then continue the client work.

## 14.9. Verify the revised behavior

Test the revised scenarios alongside the behavior the change intends to preserve. A one-filter test suite would provide weak evidence after both filters become part of the request.

Build a small scenario-to-evidence map before closure:

| Revised scenario       | Backend evidence                      | Frontend evidence           | Integrated evidence                |
|------------------------|---------------------------------------|-----------------------------|------------------------------------|
| Both filters populated | Query or API test proves AND behavior | Component sends both values | Browser result matches both values |

| Revised scenario   | Backend evidence                             | Frontend evidence              | Integrated evidence                         |
|--------------------|----------------------------------------------|--------------------------------|---------------------------------------------|
| Fleet origin only  | Missing status adds no status predicate      | Empty status is omitted        | Request and results use only fleet origin   |
| Active status only | Missing fleet origin adds no state predicate | Empty fleet origin is omitted  | Request and results use only status         |
| Neither populated  | No filter predicate changes the result set   | Both controls can remain empty | Browser shows the unfiltered result set     |
| No matches         | API returns the reviewed no-match response   | Existing empty state renders   | Browser distinguishes no match from failure |

Run `/opsx:verify` after the focused repository tests:

```
/opsx:verify add-vehicle-filters
```

Use its completeness, correctness, and coherence findings to locate missing evidence or drift. Inspect the cited code and reproduce the relevant commands before deciding that the change is ready to archive.

Search for remnants of the original plan as a final check. Useful terms may include the old requirement heading, the original API parameter, UI labels that mention only Fleet Origin State, handwritten client types, and tests whose names still imply one-filter behavior.

## 14.10. Hands-on exercise

Revise the active filter change in the prepared repositories:

1. Run status, show\ -V-diff, and strict validation for add-vehicle-filters.
2. Record the original one-filter behavior and identify which tasks and implementation already exist.
3. Apply the exercise decisions for Active Status, AND behavior, and empty selections.
4. Decide whether to revise or split the change, then record the reason in the proposal.
5. Use `/opsx:update` to revise the proposal, delta specifications, design, and tasks without changing application code.
6. Review each proposed artifact edit and run the coherence and validation checks again.
7. Classify partial implementation as reusable, incomplete, contradictory, or stale.
8. Resume `/opsx:apply`, run focused tests, and use `/opsx:verify` before closure.

Keep the change active if an artifact still describes only Fleet Origin State, the contract cannot represent empty selections consistently, or the tests never exercise both filters together.

## 14.11. Verification checkpoint

Before closing the change, ask whether a developer with no chat history could recover:

- why Active Status joined the existing change;
- which status values the contract accepts;
- how two populated filters combine;
- what each empty-selection case means;
- which earlier work remained valid and which tasks reopened;

- which tests provide evidence for every revised scenario.

Leave the change active if any answer depends on an old conversation or accidental implementation behavior. Archive only after the artifacts provide those answers and focused tests support the revised scenarios.

## 14.12. What you learned

Revise an in-flight change when the new decision still belongs to the same business outcome. Update its full planning set before coding resumes, and reopen any task whose evidence no longer applies. If the next developer still needs the original chat to understand the change, the revision is not finished.

## 14.13. Online references

- OpenSpec skills reference<sup>1</sup>
- OpenSpec CLI reference<sup>2</sup>

---

<sup>1</sup><https://openspec.dev/docs/skills>

<sup>2</sup><https://openspec.dev/docs/cli>

## 15. Coordinating Parallel Changes

Consider two developers working from the same accepted truck-search specification. One opens add-license-plate-search; the other opens revise-vin-search. Each change has its own proposal, delta specifications, design, tasks, branch, and worktree. Assume that both pass their focused tests.

Reviewed alone, either change could look complete. Read together, they might assign different meanings to the same request, generated client method, or Angular control. The License Plate change could replace a single VIN field with a search-mode control, while the VIN change could revise the accepted VIN length rule around the original field. Git would not have to report a conflict for those designs to be incompatible.

OpenSpec lets each proposal develop without immediately rewriting the accepted specification. That isolation supports parallel work; compatibility still requires a comparison of the proposed behavior. The team would need to settle shared decisions and recheck any proposal that outlives the baseline from which it began.

### 15.1. Learning objective

By the end of this chapter, you should be able to coordinate concurrent OpenSpec changes without treating separate folders, branches, or worktrees as evidence of compatibility. You should also be able to find semantic overlap, record a resolution in the active artifacts, and choose a synchronization or archive order that could preserve both business goals.

### 15.2. Separate isolation from coordination

Three mechanisms may appear in a parallel workflow, but each answers a different question:

| Mechanism              | Question it can answer                                             | Question it cannot answer                                      |
|------------------------|--------------------------------------------------------------------|----------------------------------------------------------------|
| OpenSpec change folder | What behavior does this proposal intend to add, modify, or remove? | Would another active proposal make the behavior contradictory? |
| Git branch             | Which commits belong to this line of work?                         | Do two branches express compatible product decisions?          |
| Git worktree           | Can each branch have its own working directory?                    | Which shared contract should become accepted behavior?         |

Suppose the branches edit different files. License Plate search could add a controller method, while the VIN change could revise a shared request type. Git might merge both edits cleanly even though the controller sends a request the revised type can no longer represent. Suppose instead that both branches edit the same OpenAPI file. Git might report a textual conflict, but resolving the lines would not decide whether one endpoint should accept mutually exclusive parameters or separate operations.

Begin the coordination review above the file level: **Can both sets of requirements be true in one accepted system?**

⚠ A clean merge is weak evidence

A clean Git merge could show only that the edited lines did not collide. Passing tests on each branch could show only that each implementation satisfies the assumptions available on that branch. Neither result would compare the two proposed meanings.

### 15.3. Establish one comparison point

Parallel changes become harder to reason about when each participant uses a different baseline. Before comparing proposals, name the accepted commit and main specifications from which both changes will be reviewed. That snapshot need not freeze the repository, but it would give reviewers one answer to “accepted before these changes.”

For the exercise, use these two active changes:

| Change                   | Business goal                                             | Intended delta                                                          |
|--------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------|
| add-license-plate-search | Let a user find a truck by license plate and jurisdiction | Add another search mode while preserving unit-number and VIN search     |
| revise-vin-search        | Change the accepted VIN input rule                        | Modify VIN validation and request behavior without changing other modes |

The business goals could coexist. Their designs might not. Both changes may touch the search-mode selector, shared request shape, OpenAPI operation, generated Angular client, validation messages, and regression suite. Different goals do not provide enough evidence to treat the changes as independent.

Start by checking that both artifacts are structurally readable:

```
openspec status --change add-license-plate-search
openspec show add-license-plate-search --diff
openspec validate add-license-plate-search --strict

openspec status --change revise-vin-search
openspec show revise-vin-search --diff
openspec validate revise-vin-search --strict
```

Strict validation could reject malformed deltas. It would not decide whether the two valid deltas contradict each other, so the next review should compare meaning rather than repeat validation.

### 15.4. Read each delta in two directions

Avoid reading either proposal as a self-contained story. Read it first against the accepted specification and then sideways against every other active change that touches the same capability.

The baseline comparison could ask:

- Which accepted requirement does this change assume?
- Which behavior would it add, modify, remove, or rename?
- Which existing scenarios must remain true?
- Which contract, generated artifact, UI state, or test carries that behavior?

The sideways comparison could then ask:

- Do both changes alter the same requirement or observable outcome?
- Does one proposal depend on a request or response shape that the other would revise?
- Do the proposals give different meanings to the same empty, invalid, or no-match state?
- Would a scenario from one change fail after the other became the baseline?

A compact worksheet may keep this review grounded:

| Surface             | Accepted baseline          | License Plate delta               | VIN delta                    | Decision needed?                                             |
|---------------------|----------------------------|-----------------------------------|------------------------------|--------------------------------------------------------------|
| Search mode         | Unit Number or VIN         | Add License Plate                 | Preserve existing modes      | Could the mode model represent all three?                    |
| Request contract    | One identifier plus a mode | Add plate and jurisdiction inputs | Revise VIN input constraints | Could invalid parameter combinations be expressed?           |
| Generated client    | Current search request     | Regenerate for plate fields       | Regenerate for VIN rule      | Which contract version should generate the client?           |
| Validation          | Rules vary by current mode | Add plate and jurisdiction rules  | Replace VIN rule             | Could validation switch by mode without leaking rules?       |
| Regression behavior | Unit Number and VIN pass   | Preserve both modes               | Preserve Unit Number         | Which VIN scenarios would be replaced rather than preserved? |

The coordination work would sit in the last column. “Both touch SearchComponent” would describe file overlap. “License Plate assumes fixed-length VIN while the VIN delta replaces that rule” would identify a behavioral decision that reviewers could settle.

## 15.5. Ask Copilot to report overlap, not resolve it

An exploration pass could help assemble the comparison without granting an agent authority to pick the winning behavior. A prompt such as this would keep the task bounded:

```
/opsx:explore
```

```
Compare the active changes `add-license-plate-search` and `revise-vin-search` with the same accepted truck-search specifications.
```

For each change, report:

- requirements and scenarios added, modified, removed, or assumed unchanged;
- shared API, generated-client, UI, validation, and test surfaces;
- assumptions that would become stale if the other change were accepted first; and
- product or contract decisions that the artifacts do not resolve.

Do not edit artifacts or application code. Do not choose a winner from implementation order. Dismiss textual overlap from behavioral overlap.

The resulting report should point to the exact requirements and scenarios in disagreement. If it only lists files, the prompt has not yet reached the decision boundary. A follow-up could ask what observable behavior would differ after both deltas were applied.

## 15.6. Recognize the forms of semantic overlap

Some conflicts would announce themselves through the same requirement name. Others might hide behind different language or different repositories. The following patterns deserve a closer comparison:

- Both deltas modify the identity or meaning of one accepted requirement.
- One delta adds behavior under a condition that the other delta removes.
- Two deltas assign different validation rules to the same input or mode.
- One proposal assumes a response shape, enum, or status code that the other would replace.
- The backend proposal changes a contract while the frontend proposal invents a local substitute.
- A preservation claim in one proposal conflicts with a modification in the other.
- Both changes pass alone, but the scenarios require mutually exclusive UI or API states.

Consider the word “preserve.” The License Plate proposal might say that VIN behavior remains unchanged, while revise-vin-search exists specifically to change it. The two business goals need not conflict, but the preservation boundary has become stale. Revising that sentence could be as important as reconciling the request type because it changes what License Plate regression tests should prove.

## 15.7. Choose a coordination response

Finding overlap does not automatically require one large change. Choose the response that keeps the business decision easiest to review:

| Response                   | When it may fit                                                                | Artifact consequence                                                               |
|----------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Keep both changes separate | The goals remain independently releasable after one shared decision            | Revise each affected proposal, delta, design, and task list to record the decision |
| Establish a prerequisite   | One change must define a contract or behavior before the other can be coherent | Record the dependency and review the dependent change against the new baseline     |
| Combine the changes        | Their scenarios cannot describe a complete outcome independently               | Replace the two partial plans with one reviewed scope before more implementation   |

| Response         | When it may fit                                       | Artifact consequence                                                                  |
|------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------|
| Pause one change | Product owners have not settled incompatible behavior | Keep the unresolved change active rather than encoding a decision through merge order |

For this exercise, suppose reviewers choose a prerequisite rather than combining the changes. The VIN rule should become accepted first because the License Plate proposal promises to preserve VIN behavior and must test against the revised rule. License Plate remains a separate business capability, but its preservation boundary and generated-client tasks now depend on the VIN change.

That choice should appear in both active artifacts. The VIN design could identify the shared request contract it establishes. The License Plate proposal could replace its stale “VIN remains unchanged” assumption with the specific revised VIN behavior it must preserve. Its tasks could also defer final client generation until the reviewed backend contract contains both the accepted VIN rule and proposed plate fields.

#### **i** Record the decision before moving code

A new reviewer should be able to recover the dependency from proposal.md, delta specifications, design.md, and tasks.md alone. If the answer depends on the originating chat transcript, the coordination decision has not reached the durable artifacts.

## 15.8. Coordinate the backend-owned contract

In this book’s two-repository lab, the backend owns the OpenAPI contract and the Angular client is generated from it. That ownership suggests a coordination point even when implementation proceeds in parallel.

Frontend work could begin against reviewed scenarios and a proposed contract. Tests for the new mode control, empty states, or validation transitions need not wait for every backend task. Final client integration, however, should use a client generated from the contract the backend actually publishes. A hand-written TypeScript request type could make both branches compile while hiding the unresolved contract.

Before either change closes, reviewers could ask:

- Can one request represent Unit Number, VIN, and License Plate without ambiguous combinations?
- Which layer rejects a plate without jurisdiction or a VIN that violates the revised rule?
- Does the OpenAPI document express the same modes and constraints as the backend behavior?
- Has the Angular client been regenerated from that reviewed document?
- Do frontend states distinguish invalid input, no match, and request failure as specified?

Answers shared by both changes may belong in the accepted search specification or a recorded contract decision. Answers unique to License Plate should remain in its delta. This division could keep one proposal from silently redefining every search mode.

## 15.9. Use worktrees without confusing their role

Separate worktrees may let a developer or agent keep both branches available at once:

```
git worktree add ../truck-search-license-plate -b feature/license-plate
git worktree add ../truck-search-vin -b feature/revise-vin
git worktree list
```

Each directory could have its own application code and OpenSpec change. Before creating them, check the repository's branch names and destination paths; the example names may not fit an existing checkout.

Keep the accepted-baseline commit, change names, contract owner, dependency decision, and verification evidence in a place the team can recover outside those directories. Without that shared coordination record, two worktrees could merely let incompatible assumptions develop faster.

## 15.10. Reconcile before synchronization or archive

Archive order may matter when the first archived change updates the main specifications that the second change originally assumed. Finishing first would be a weak ordering rule; a dependency or shared contract would offer a stronger reason.

Before accepting the VIN prerequisite, compare its delta with the main specification and inspect the proposed result. After synchronization or archive updates the accepted baseline, return to the License Plate change before applying or closing it:

1. Re-read its proposal and delta specifications against the updated main specs.
2. Replace preservation claims or scenarios that still describe the old VIN rule.
3. Revisit design decisions that depend on the shared request or response contract.
4. Reopen any completed task whose evidence came from the earlier baseline.
5. Run strict validation and the affected VIN regression tests again.
6. Apply, verify, and archive License Plate only after its artifacts describe the combined system.

If the team uses `/opsx:sync` to merge a reviewed delta into the main specifications while keeping the change active, the same comparison would still apply. Synchronization changes the baseline used by neighboring work; it does not prove that neighboring deltas remain coherent. Before accepting the sync, inspect the main-spec diff and keep the change active until its implementation and evidence are ready.

Silently rewriting License Plate code to accommodate the new VIN rule would leave its active artifacts stale. Another reviewer could not tell whether the behavior was intentional or merely the result of whichever branch merged last. Revising the proposal, scenarios, design, and tasks would turn that implementation adjustment back into a reviewable decision.

## 15.11. Hands-on exercise

Create two active changes in the prepared truck-search repositories: one for License Plate search and one for a revision to VIN behavior.

1. Give each change a distinct business goal, scope, non-goals, and preservation boundary.
2. Record the accepted commit and main specifications used as their common baseline.
3. Validate both changes, then compare each delta with the baseline and with the other delta.
4. Complete the overlap worksheet for requirements, API contract, generated client, UI, validation, and regression tests.
5. Identify one semantic conflict that a clean Git merge would not settle.

6. Choose whether to keep, order, combine, or pause the changes, and record the reason in the affected artifacts.
7. Accept the prerequisite change, then revise the remaining change against the new baseline.
8. Apply and verify the remaining change without losing either reviewed business goal.

At each step, ask which artifact would let a reviewer recover the decision. A conflict discovered only after the first archive may indicate that the surviving change has become stale; it need not mean that parallel work itself was a mistake.

## 15.12. Verification checkpoint

Check whether the final accepted specifications describe all search modes without contradictory input, validation, or result rules. Verify the artifacts and implementation from that question outward:

```
openspec validate --all
git diff -- openspec/specs openspec/changes
```

Then run the focused backend, contract-generation, frontend, and regression checks configured for the prepared repositories. The evidence should support these conclusions:

- Each change retained a distinct, reviewed business purpose.
- Shared requirements now have one interpretation.
- The accepted specifications contain the reconciled VIN and License Plate behavior.
- The generated Angular client came from the reviewed backend contract.
- Unit Number, VIN, and License Plate scenarios pass at the affected boundaries.
- No active artifact still assumes the pre-coordination baseline.

If the final item fails, keep that change active. Updating the stale artifact and repeating the focused checks would cost less than archiving a contradiction that the next change must untangle.

## 15.13. What you learned

Separate change folders, branches, and worktrees can isolate activity. A common baseline and a sideways delta review can then expose incompatible intent. Once overlap appears, record the shared decision, order changes by dependency, and recheck the surviving proposal after the baseline moves.

Choose that coordination point before merge repair. Once Git reports the textual conflict, it may be too late to recover the unrecorded product decision from the changed lines alone.

## 15.14. Online references

- OpenSpec documentation
- Git worktree documentation



## 16. Making SDD a Team Practice

A developer finishes an OpenSpec change, closes the Copilot session, and opens a pull request. The tests pass, the code looks reasonable, and every task has a checkmark. Yet the reviewer still has to ask what behavior changed, who approved it, and whether the Angular client came from the same contract as the Spring Boot implementation.

What separated this work from a private OpenSpec exercise? Not the number of artifact files. Ask whether another developer could enter after the original session disappeared and make the next correct decision. That person would need the business intent, accepted behavior, technical constraints, and verification evidence without depending on the author's memory.

Treat that handoff as the test for team practice. Then agree on which changes deserve a spec, where decisions belong, who reviews them, and what evidence permits the work to move forward. Installing OpenSpec in every repository without that agreement may only create more places to disagree.

### 16.1. Learning objective

By the end of this chapter, you should be able to introduce a proportionate OpenSpec workflow to a team, define its decision and review boundaries, and test whether the resulting record can support a developer who did not participate in the original Copilot session.

### 16.2. Start with the handoff problem

The exercises used one person, two repositories, and a controlled sequence of changes. Now picture the commercial truck search moving through a delivery team. A product owner requests the search mode in Jira. One developer shapes the backend contract, another regenerates the Angular client, and a third reviews validation behavior. Interruptions and parallel work separate those steps.

Where should the next person look if “commercial truck” means something different in the ticket, delta specification, API schema, and UI test? Let Jira establish priority and ownership. Let the OpenSpec change record the proposed behavior and technical decisions. Let the repository preserve the reviewed result. Keep chat as working space rather than the only place where a decision exists.

#### **i** A practical test for shared context

Ask a reviewer to explain what should change, what should remain unchanged, and how the team will know the work is complete using repository artifacts alone. A “no” points to a handoff problem, even when the implementation passes its tests.

## 16.3. Agree on the minimum operating model

Before asking everyone to create artifacts, settle a few working rules. Which work requires an OpenSpec change? Who may accept behavior? When may implementation begin? What would count as enough verification to synchronize or archive?

Keep the first agreement short enough to scan during a change. A longer process document may be harder to apply and easier to ignore. Adapt the following example to the repository's paths and commands:

### ## OpenSpec working agreement

- Create a full change for new behavior, contract changes, cross-repository work, and changes to accepted requirements.
- Link the change name from the Jira issue and pull request.
- Obtain behavior review before implementation when the proposal changes a user-visible outcome.
- Obtain technical review before implementation when the delta changes an API, persistence model, security boundary, or generated client.
- Keep decisions in OpenSpec artifacts; use chat and meetings to develop them.
- Record focused test commands and results before synchronization or archive.
- Recheck every active change after its accepted baseline moves.

Would every team choose those exact rules? Probably not. Match the approval and evidence burden to the product's constraints. Keep the boundary explicit so people and agents can see where a lightweight record ends and a reviewed change begins.

## 16.4. Give each system one job

Teams often lose intent because the same decision appears in several tools with slightly different wording. Copying the full requirement into Jira, a proposal, a pull request, and a chat prompt may look thorough, but which copy would be corrected after review?

A clearer division might look like this:

| System or artifact   | Question it should answer                                         | What it should avoid becoming                  |
|----------------------|-------------------------------------------------------------------|------------------------------------------------|
| Jira issue           | Why is the work scheduled, who owns it, and what is its status?   | The only source of detailed behavior           |
| proposal.md          | What outcome is proposed, what is in scope, and what is excluded? | A duplicate task board                         |
| Delta specifications | Which observable behavior would be added, modified, or removed?   | An implementation diary                        |
| design.md            | Which technical choices and constraints shape the implementation? | A restatement of every requirement             |
| tasks.md             | In what order could the team implement and verify the change?     | Evidence that unchecked behavior is acceptable |
| Pull request         | What code and artifact revisions are ready for review?            | The first place a product decision appears     |

|                               |                                               |                                                       |
|-------------------------------|-----------------------------------------------|-------------------------------------------------------|
| Tests and verification record | What evidence supports the reviewed behavior? | A substitute for deciding what the behavior should be |
|-------------------------------|-----------------------------------------------|-------------------------------------------------------|

---

Do not eliminate links between tools; make them directional. Point from the Jira issue to the OpenSpec change for behavior. Point from the pull request to both, then record the verification commands there or in the change. Reviewers can follow that chain without comparing several competing copies of the requirement.

## 16.5. Walk one change through the team

Test the shared workflow against one change. Suppose the product owner requests commercial truck search across the Spring Boot and Angular repositories. How might that request move without turning OpenSpec into a sequence of gates for their own sake?

### 16.5.1. Frame the proposal together

Begin with the business goal, affected behavior, non-goals, and known repository boundaries. Use an exploration session to uncover current routes, generated client code, and existing search scenarios. Do not ask the agent to decide what counts as a commercial truck; assign that definition to an accountable human owner.

Focus the early review on meaning rather than prose quality:

- Would two reviewers interpret “commercial truck” the same way?
- Which current search modes must remain unchanged?
- Does the request alter a public contract or only add a client-side filter?
- Which repository owns the contract from which the client is generated?
- What outcome should occur for invalid input, no match, and service failure?

Resolve those questions while changing the proposal remains cheap. Waiting until pull-request review may turn the same disagreement into code rework across both repositories.

### 16.5.2. Review behavior before design hardens

Once the proposal and delta scenarios express the requested outcome, ask the product or domain reviewer to accept the behavior boundary. Leave class names, endpoint paths, and component structure to technical review unless those choices affect the product contract.

Next, examine design.md and the cross-repository implications with the technical reviewer. Would the backend publish the OpenAPI change before the frontend regenerates its client? Could old and new clients coexist during deployment? Which validation belongs in the API, and which feedback belongs in the UI? Let those answers shape the implementation plan without reopening an already settled business outcome.

### 16.5.3. Turn tasks into handoff points

A private checklist may follow whatever order helps its author. For a team task list, show where work passes to someone else and what must be true at that point.

For the commercial truck change, a useful sequence could include:

1. Review the behavior and non-goals with the named business owner.
2. Review the proposed request and response contract with backend and frontend owners.
3. Implement and test the backend behavior against the reviewed scenarios.
4. Publish the contract artifact and regenerate the Angular client from it.
5. Implement the UI states and run focused frontend tests.
6. Compare the implementation, generated artifacts, and accepted scenarios before archive.

Give each step a verification point rather than a bare completion claim. “Regenerate the client” says what to do. “Regenerate from the reviewed backend contract and confirm no hand-written request type remains” gives the next reviewer something to inspect.

### 16.5.4. Preserve the review result

Approval in a meeting or chat may unblock the author, but would it help the developer who takes over next week? Record the result in the artifact that governs the behavior or design. Skip the meeting transcript; preserve enough context to prevent the same question from returning during implementation.

Carry the OpenSpec change name into commits and pull requests. Name the commands run and any checks that remain outside automation. After synchronization or archive, ask whether the accepted specifications describe the behavior without requiring the active change folder.

## 16.6. Match review ownership to the decision

Would every artifact need a separate approver? That rule could add delay without improving the decision. Review ownership may work better when it follows the consequence under review.

| Decision                                    | Likely reviewer                  | Review question                                            |
|---------------------------------------------|----------------------------------|------------------------------------------------------------|
| User-visible behavior and non-goals         | Product or domain owner          | Is this the outcome the system should promise?             |
| API, data, security, or deployment boundary | Technical owner                  | Can the design preserve the required constraints?          |
| Code and automated tests                    | Implementation reviewer          | Does the implementation satisfy the reviewed scenarios?    |
| Synchronization and archive readiness       | Change owner or release reviewer | Do accepted specs, implementation, and evidence now agree? |

One person may cover several rows on a small team. Still, name the decision behind each approval. A product owner accepting endpoint naming does not replace technical review, and code approval does not settle an ambiguous business rule.

Define escalation points for consequences that need a specific owner, such as a new public API, a removed scenario, a security boundary, or disagreement between active changes. Keep routine edits with the normal reviewer. Let consequence, rather than title or artifact count, determine the review path.

## 16.7. Scale the workflow to the risk

Requiring the same artifact set for a typo and a cross-repository contract change would teach the team to work around the process. How much specification would help before its maintenance cost exceeds the uncertainty it removes?

Use a simple classification to discuss that choice:

| Change shape                                                | Possible record                                                                      | Typical review                                |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------|
| Existing behavior, low risk, easy to reverse                | Issue or pull-request note tied to accepted specs                                    | Implementation review                         |
| Local behavior with bounded impact                          | Proposal, delta scenarios, and focused tasks                                         | Behavior and implementation review            |
| Contract, security, persistence, or cross-repository change | Full proposal, deltas, design, tasks, and verification record                        | Named behavior and technical review           |
| Urgent production repair                                    | Small initial record with owner, risk, and verification, followed by artifact repair | Immediate peer review and scheduled follow-up |

Do not let the matrix make the decision automatically. Use it to expose uncertainty, reversibility, blast radius, contract impact, and the cost of discovering a mistake late. When a smaller artifact set seems sufficient, record the exception so the reviewer can challenge the reason rather than wonder whether files were forgotten.

### Ceremony can hide missing thought

A complete folder may still contain vague requirements, copied design text, and tasks that prove only that files changed. Review the decisions and scenarios, not the presence of filenames. A smaller record with a precise boundary could serve the team better than a full set of unexamined artifacts.

## 16.8. Make Copilot work from reviewed context

Once a team shares artifacts, start Copilot prompts from reviewed material rather than each developer's reconstruction of the ticket. This only works while the repository context remains current.

Ask Copilot to compare implementation with accepted scenarios, identify a contract boundary, or draft tests for review. Point to the relevant proposal, delta, design, and repository instructions. State where the agent must stop and request a human decision.

Read the active ``add-commercial-truck-search`` artifacts and the accepted truck-search specs.

Report any mismatch between the proposed scenarios, backend contract, generated Angular client, and current tests. Do not choose new product behavior or edit accepted specifications. List any

decision that requires the product owner or API owner.

Would another developer find the same governing requirements and unresolved decisions from the same repository state? If the answer depends on instructions held in one person's prompt history, move those instructions into maintained repository context.

Review that context after changes to OpenSpec, Copilot customization files, generated-client commands, or test tooling. Run the documented commands, remove contradictory instructions, and revise examples whose paths or behavior changed.

## 16.9. Introduce the practice in stages

A team may resist SDD for reasonable reasons. The artifacts take time, reviewers can become a bottleneck, and poorly scoped specs can freeze choices that implementation still needs to explore. Starting with every repository and every change could make those costs arrive before the team sees the handoff benefit.

Gather evidence with a limited pilot:

1. Choose one capability with a real handoff or cross-repository boundary.
2. Agree on the minimum rules and name the reviewers before opening the change.
3. Run two or three changes through proposal, implementation, verification, and archive.
4. Ask a developer outside the work to reconstruct one completed change from repository artifacts.
5. Revise the agreement where the handoff failed or where an artifact added no useful decision.

Measure friction in concrete terms. Which questions returned during code review? Which decisions had no owner? Which artifact became stale first? Where did a reviewer still need the author or chat transcript? Those observations say more than a count of created spec files.

Teams might also watch for signs that the practice is becoming performative:

- Proposals appear after the code and merely describe it.
- Reviewers approve artifact folders without reading scenarios.
- Jira and OpenSpec contain competing versions of the same behavior.
- tasks.md records activity but no verification.
- Active changes remain based on specifications that have since moved.
- Archive becomes a cleanup chore disconnected from acceptance.

Any one occurrence may have a reasonable explanation. When a pattern repeats, revise the working agreement, training, or scope.

## 16.10. Capstone: establish a team workflow

Use a proposed commercial truck search change to test the operating model across the Spring Boot and Angular repositories. The goal is not the largest possible artifact set. Could the team defend the amount of specification, transfer the work between people, and recover the final decision later?

1. Name the business owner, technical owner, change author, and implementation reviewer.
2. Record the business goal, affected behavior, non-goals, and accepted baseline.
3. Classify the change by uncertainty, reversibility, blast radius, and contract impact.
4. Choose the OpenSpec artifacts required by that classification and record any exception.

5. Review the delta scenarios before implementation begins.
6. Record the backend-owned contract boundary and the generated-client handoff.
7. Divide tasks.md at points where responsibility or evidence passes between team members.
8. Link the Jira issue, OpenSpec change, commits, pull requests, and verification results.
9. Complete the verification checkpoint before synchronization or archive.

## 16.11. Verification checkpoint

Give the completed change to a reviewer who did not participate in its Copilot sessions. Without asking the original author, could that reviewer answer the following questions?

- What user-visible behavior changed, and what remained outside scope?
- Who accepted the behavior and the technical boundary?
- Which accepted requirements and delta scenarios governed the implementation?
- Which repository owned the contract, and how was the Angular client produced?
- What verification evidence supports synchronization and archive?
- Which unresolved risks or follow-up items remain?

Run the structural checks available in the repository as supporting evidence:

```
openspec status --change add-commercial-truck-search
openspec validate add-commercial-truck-search --strict
openspec show add-commercial-truck-search --diff
```

Then run the focused backend, contract-generation, frontend, and regression checks named in the change. Structural validation could show that OpenSpec can parse the artifacts; it could not show that the right people made the missing product decision.

If the reviewer cannot reconstruct the work, repair the smallest broken handoff. The missing piece may be an owner, a scenario, a contract dependency, a verification result, or a link. If the reviewer can reconstruct it but the process felt heavier than the risk justified, revise the working agreement before applying it to the next class of change.

## 16.12. What you learned

Could a developer who missed the meeting and never saw the chat continue the work without guessing? That modest result would make SDD a team practice. When the test fails, repair the specific handoff rather than produce more paperwork.

## 16.13. Online references

- OpenSpec documentation<sup>1</sup>
- GitHub Copilot customization<sup>2</sup>
- Atlassian Jira documentation<sup>3</sup>

---

<sup>1</sup><https://openspec.dev/docs>

<sup>2</sup><https://docs.github.com/en/copilot/customizing-copilot>

<sup>3</sup><https://support.atlassian.com/jira-software-cloud/>



## 17. Summary

OpenSpec gives an AI-assisted team a durable place to keep intent, but the files alone do not create alignment. The team must still ask precise questions, review generated artifacts, test the implementation, and reconcile overlapping decisions.

The truck-search sequence demonstrates the core patterns:

- A greenfield change establishes the first accepted behavior through explore, propose, apply, and archive.
- An additive delta introduces VIN search without replacing unit-number search.
- A modified delta replaces the old unit-number constraint with a maximum of 10 characters.
- An active proposal can be revised before implementation when the business changes direction.
- Parallel changes require both artifact isolation and source-control isolation, followed by deliberate synchronization.

The practical test is whether a developer or agent can enter the repository in a later session and recover what the system should do, what is changing, and how completion will be verified. When the specs, code, and tests answer those questions consistently, context has become shared engineering material rather than chat history.



# References

The links below are the primary sources used for current commands, repository behavior, and technology details. Version-sensitive claims should be checked against the source at the time a team applies them.

## OpenSpec and workflow

- OpenSpec repository<sup>1</sup>
- OpenSpec documentation<sup>2</sup>
- OPSX workflow<sup>3</sup>
- Installation<sup>4</sup>
- Supported tools and aliases<sup>5</sup>
- Awesome-OpenSpec<sup>6</sup>

## Petclinic and technology

- Spring Petclinic Angular<sup>7</sup>
- Spring Petclinic REST<sup>8</sup>
- Spring Boot reference<sup>9</sup>
- Spring Framework reference<sup>10</sup>
- Angular documentation<sup>11</sup>
- Java SE 21 documentation<sup>12</sup>
- TypeScript documentation<sup>13</sup>

## AI-assisted development

- GitHub Copilot in VS Code<sup>14</sup>

---

<sup>1</sup><https://github.com/Fission-AI/OpenSpec>

<sup>2</sup><https://github.com/Fission-AI/OpenSpec/tree/main/docs>

<sup>3</sup><https://github.com/Fission-AI/OpenSpec/blob/main/docs/opsx.md>

<sup>4</sup><https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md>

<sup>5</sup><https://github.com/Fission-AI/OpenSpec/blob/main/docs/supported-tools.md>

<sup>6</sup><https://github.com/speclib/awesome-openspec>

<sup>7</sup><https://github.com/spring-petclinic/spring-petclinic-angular>

<sup>8</sup><https://github.com/spring-petclinic/spring-petclinic-rest>

<sup>9</sup><https://docs.spring.io/spring-boot/reference/>

<sup>10</sup><https://docs.spring.io/spring-framework/reference/>

<sup>11</sup><https://angular.dev/>

<sup>12</sup><https://docs.oracle.com/en/java/javase/21/>

<sup>13</sup><https://www.typescriptlang.org/docs/>

<sup>14</sup><https://code.visualstudio.com/docs/copilot/overview>

- GitHub Copilot CLI<sup>15</sup>

---

<sup>15</sup><https://docs.github.com/en/copilot/concepts/agents/about-copilot-cli>

# **A. Environment Setup and Troubleshooting**

## **A.1. Installing and Updating OpenSpec**

### **A.1.1. Scope**

This appendix is intended to cover tested OpenSpec 1.13.1 installation and maintenance procedures for Windows 11 and macOS. The operating-system differences should remain visible rather than being folded into one generic command sequence.

### **A.1.2. Before installation**

- Readers should first confirm that Node.js 24.19.0 and npm are available.
- The appendix should describe global npm package locations and the way PATH affects command discovery.
- Teams may want to decide whether to pin OpenSpec 1.13.1 or allow controlled upgrades.

### **A.1.3. Install OpenSpec 1.13.1**

- Windows 11 and macOS should have separate terminal instructions.
- After installation, readers can verify the executable and its version.
- The tested baseline should include the expected command output.

### **A.1.4. Inspect the installed version**

- The appendix can show the OpenSpec version command.
- It can also show how npm reports local and global package versions.
- The CLI package version should be distinguished from repository-generated integration files.

### **A.1.5. Upgrade OpenSpec**

- Before changing the team baseline, readers should check the release notes.
- The global npm package can then be upgraded.
- The new version should be verified before any repositories are changed.

### **A.1.6. Update an existing repository**

- An existing repository can be updated with `openspec\update`, using the intended profile and GitHub Copilot integration.
- Before committing, readers should review changed instructions, prompt files, and Agent Skills.
- Slash-command discovery should be retested in VS Code and IntelliJ IDEA.
- The appendix should explain when `-V-force` may be appropriate and what needs review afterward.

### **A.1.7. Verification checklist**

- From a new terminal, the expected CLI version should run.
- Both repositories should report healthy OpenSpec roots.
- Generated GitHub Copilot files should match the selected profile.
- Existing project-owned configuration and artifacts should remain intact.

### **A.1.8. Online references**

- OpenSpec installation
- OpenSpec CLI reference
- OpenSpec supported tools
- npm documentation

## **A.2. Lab Setup**

### **A.2.1. Required tools**

Plan to have these available for each participant:

- Windows 11 or macOS;
- Git and access to the two prepared Bitbucket repositories;
- VS Code or IntelliJ IDEA with GitHub Copilot Chat access;
- OpenSpec 1.13.1;
- Node.js 24.19.0 and npm;
- Java 21 and Maven for the Spring Boot 4.1.1 project;
- Angular 21, Vitest, Playwright, and OpenAPI Generator for the frontend project;
- Quarto only if participants will render the book locally.

Before the class, check the current OpenSpec installation instructions at <https://openspec.dev/docs/installation>. Tool versions and package commands can change.

### **A.2.2. Prepare the exercise repositories**

Start by cloning the prepared backend and frontend repositories. It may help to confirm that each repository builds and passes its tests before initializing OpenSpec. The backend owns the OpenAPI contract, and OpenAPI Generator produces the TypeScript Angular client used by the frontend.

```
cd commercial-truck-search-api
openspec init

cd ../commercial-truck-search-ui
openspec init
```

When initialization asks which AI tool to configure, select GitHub Copilot. Give each repository its own context instead of copying one configuration into both repositories.

The backend configuration records Java 21, Spring Boot 4.1.1, Maven, Spring Data JPA, H2, test conventions, and ownership of the OpenAPI contract. The frontend configuration records Angular 21, generated-client boundaries, Vitest, Angular component tests, and Playwright.

Review the generated configuration. It can describe stable project facts, but it should not invent database products, deployment platforms, or business rules that the class has not selected.

### A.2.3. Preflight check

Before Morning 1, check that:

1. Both starter applications build and their existing tests pass.
2. openspec\init created the expected project files in both repositories.
3. GitHub Copilot Chat recognizes the expected OpenSpec commands in VS Code and IntelliJ IDEA.
4. Each participant knows which commands belong in the terminal and which belong in chat.
5. Each repository has a clean baseline commit suitable for reset between exercises.

You may want to keep prepared checkpoints for both repositories after each major exercise. That gives participants who miss a step a way to resume from the same accepted specs, contracts, and code state as the rest of the class.

## A.3. Troubleshooting OpenSpec

### A.3.1. A diagnostic method

For each problem, start with the symptom and a likely cause. Then run the cheapest check that could distinguish between possible causes before choosing a repair. Keep the command output from that check; it may be more useful than repeating a setup step that could change the evidence.

### A.3.2. Installation failures

You might see one of these symptoms:

- openspec cannot be found after npm installation.
- The terminal resolves a different global npm location than expected.
- Windows execution policy or macOS permissions prevent the command from running.
- The installed version differs from the book's 1.13.1 baseline.

### **A.3.3. Initialization and update failures**

Check whether:

- OpenSpec initialized a different directory than expected.
- Existing generated files conflict with the update.
- The selected GitHub Copilot integration or profile is unavailable.
- Project-owned configuration changed unexpectedly.

### **A.3.4. Command discovery failures**

You may be dealing with command discovery when:

- VS Code does not show the expected prompt command.
- IntelliJ IDEA does not discover the same workflow.
- A prompt-file command is being confused with an Agent Skill or terminal CLI command.
- The IDE appears to be using an earlier version of the generated integration files.

### **A.3.5. Artifact and validation failures**

The failure may be in artifact or validation handling when:

- A change cannot identify its schema.
- Artifact dependencies remain blocked.
- Requirements or scenarios fail strict validation.
- A modified requirement does not match the accepted source requirement.
- Archive or sync encounters overlapping deltas.

### **A.3.6. Two-repository failures**

Compare the repositories when:

- Frontend and backend changes use different business terms.
- The generated Angular client appears stale.
- The OpenAPI document does not match the backend behavior.
- A test passes against a different backend or port than the one you intended.

### **A.3.7. Recovery boundaries**

Before regeneration, preserve project-owned specs and active changes. Use version control to inspect generated-file changes. If an OpenSpec defect remains reproducible, report the version, operating system, minimal repository state, command, and complete output.

### **A.3.8. Online references**

- [OpenSpec troubleshooting](#)
- [OpenSpec CLI reference](#)
- [GitHub Copilot troubleshooting](#)



## B. OpenSpec Reference

### B.1. OpenSpec CLI Command Reference

This section offers a quick reference for the openspec CLI commands used throughout the book. For OpenSpec slash commands, see the next section. The CLI command names and examples are current as of OpenSpec CLI 1.13.1. Since your installed version may differ, you may want to run `openspec\ -V-help` or `openspec\TU\textless{ }command\TU\textgreater{ } -V-help` to check the syntax available to you.

The CLI helps manage the files and relationships that make a change reviewable. Decisions about a change's scope and the quality of its requirements remain with the people reviewing the work.

#### B.1.1. Typical workflow

1. You might run `openspec\init` once in a project, then use `openspec\update` when the generated instruction files need to catch up.
2. To begin a change, use `openspec\new\change\TU\textless{ }name\TU\textgreater{ }`, then inspect its artifact status with `openspec\status -V-change\TU\textless{ }name\TU\textgreater{ }`.
3. You can use `openspec\list`, `openspec\show`, `openspec\context`, and `openspec\instructions` to inspect the repository and work out what an agent needs next.
4. Before treating the change as ready, consider running `openspec\validate\TU\textless{ }name\TU\textgreater{ }`. In CI, `openspec\validate -V-all` may be a better fit when the repository is the unit of validation.
5. After implementation and verification, you can run `openspec\archive\TU\textless{ }name\TU\textgreater{ }`. Archiving updates the main specifications and moves the completed change out of the active work area.

OpenSpec reports failures through the process exit status, which makes commands such as `validate` suitable for scripts and CI. The CLI help does not promise a numbered list of exit codes, so this reference does not invent one. In a pipeline, treat a nonzero status as a failed check and preserve the command output as the diagnostic.

#### B.1.2. Core commands

The commands below cover the operations most projects use regularly. The `-V-json` options can help when another tool needs to consume a result, but reading the Markdown artifacts still matters during review.

| Command                               | Purpose                                                                         | Useful options                                                                                                                                                                                                                 |
|---------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>openspec\init\{ }path{ }</code> | Initialize OpenSpec in a project and write the selected tool instruction files. | <code>-V-tools\all</code> , <code>-V-tools\github-copilot</code> , <code>-V-language\TU\textless{ }language\TU\textgreater{ }</code> , <code>-V-profile\TU\textless{ }profile\TU\textgreater{ }</code> , <code>-V-force</code> |

| Command                                                                    | Purpose                                                                        | Useful options                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>openspec\update\{[]path{[]}</code>                                   | Update OpenSpec instruction files after the CLI or tool configuration changes. | <code>-V-force</code>                                                                                                                                                                                                                                                |
| <code>openspec\new\change\TU\textless{ }name\TU\textgreater{ }</code>      | Create a new change directory using the selected workflow schema.              | <code>-V-description\TU\textless{ }text\TU\textgreater{ },</code><br><code>-V-goal\TU\textless{ }text\TU\textgreater{ },</code><br><code>-V-schema\TU\textless{ }name\TU\textgreater{ },</code><br><code>-V-store\TU\textless{ }id\TU\textgreater{ }, -V-json</code> |
| <code>openspec\list</code>                                                 | List active changes.                                                           | <code>-V-changes, -V-specs,</code><br><code>-V-sort\recent\TU\textbar{ }name, -V-json,</code><br><code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>                                                                                                            |
| <code>openspec\show\{[]item-name{[]}</code>                                | Display a change or specification.                                             | <code>-V-type\change\TU\textbar{ }spec, -V-json, -V-diff,</code><br><code>-V-deltas-only,</code><br><code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>                                                                                                         |
| <code>openspec\status\ -V-change\TU\textless{ }id\TU\textgreater{ }</code> | Show which artifacts are complete for a change.                                | <code>-V-all,</code><br><code>-V-schema\TU\textless{ }name\TU\textgreater{ },</code><br><code>-V-json, -V-store\TU\textless{ }id\TU\textgreater{ }</code>                                                                                                            |
| <code>openspec\validate\{[]item-name{[]}</code>                            | Validate a change or specification.                                            | <code>-V-all, -V-changes, -V-specs, -V-strict, -V-json,</code><br><code>-V-type\change\TU\textbar{ }spec,</code><br><code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>                                                                                         |
| <code>openspec\archive\{[]change-name{[]}</code>                           | Archive a completed change and update the main specifications.                 | <code>-V-yes, -V-skip-specs, -V-no-validate, -V-json,</code><br><code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>                                                                                                                                             |
| <code>openspec\doctor</code>                                               | Report relationship health for the resolved OpenSpec root.                     | <code>-V-json, -V-store\TU\textless{ }id\TU\textgreater{ }</code>                                                                                                                                                                                                    |

The command names reflect two related interfaces. `openspec\new\change` creates a change. The older grouped forms, `openspec\change\show`, `openspec\change\list`, and `openspec\change\validate`, can inspect and validate changes. Since the help output marks `openspec\change\list` as deprecated, new scripts are better served by the top-level `list`, `show`, and `validate` commands.

For a quick inspection of a repository, you could start with:

```
openspec list
openspec status --all
openspec doctor
```

For a CI (continuous integration) check that covers both active and archived material, choose the narrowest validation that matches the pipeline's purpose. `openspec\validate\ -V-all` validates all changes and specifications, while `openspec\validate\ -V-archived` checks that archived changes have completed tasks.

### B.1.3. Store and workset commands

Stores and worksets address different needs. You can think of a store as a standalone OpenSpec repository registered on the local machine, giving several related projects a shared place for specifications and changes.

A workset is a personal, local view that composes folders for a working session. Removing a workset does not remove its member folders.

| Command                                                                    | Purpose                                                   | Useful options                                                                                                                                                                                                 |
|----------------------------------------------------------------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>openspec\store\setup\{[]id{}}</code>                                 | Create and register a local store.                        | <code>-V-path\\TU\textless{ }path\TU\textgreater{ }</code> ,<br><code>-V-remote\\TU\textless{ }url\TU\textgreater{ }</code> ,<br><code>-V-init-git</code> , <code>-V-no-init-git</code> , <code>-V-json</code> |
| <code>openspec\store\register\{[]path{}}</code>                            | Register an existing local store.                         | <code>-V-id\\TU\textless{ }id\TU\textgreater{ }</code> , <code>-V-yes</code> ,<br><code>-V-json</code>                                                                                                         |
| <code>openspec\store\list</code>                                           | List stores registered on this machine.                   | <code>-V-json</code>                                                                                                                                                                                           |
| <code>openspec\store\doctor\{[]id{}}</code>                                | Check a store's registration and metadata.                | <code>-V-json</code>                                                                                                                                                                                           |
| <code>openspec\store\unregister\\TU\textless{ }id\TU\textgreater{ }</code> | Forget a registration without deleting the files.         | <code>-V-json</code>                                                                                                                                                                                           |
| <code>openspec\store\remove\\TU\textless{ }id\TU\textgreater{ }</code>     | Forget a registration and delete its local folder.        | <code>-V-yes</code> , <code>-V-json</code>                                                                                                                                                                     |
| <code>openspec\context</code>                                              | Print the working context for the resolved OpenSpec root. | <code>-V-json</code> ,<br><code>-V-store\\TU\textless{ }id\TU\textgreater{ }</code> ,<br><code>-V-code-workspace\\TU\textless{ }path\TU\textgreater{ }</code> , <code>-V-force</code>                          |
| <code>openspec\workset\create\{[]name{}}</code>                            | Save a named view of project folders.                     | Repeat <code>-V-member\\TU\textless{ }path\TU\textgreater{ }</code> ;<br>optionally use<br><code>-V-tool\\TU\textless{ }id\TU\textgreater{ }</code> and<br><code>-V-json</code>                                |
| <code>openspec\workset\list</code>                                         | Show saved worksets and their members.                    | <code>-V-json</code>                                                                                                                                                                                           |
| <code>openspec\workset\open\\TU\textless{ }name\TU\textgreater{ }</code>   | Open a saved workset in its preferred tool.               | <code>-V-json</code>                                                                                                                                                                                           |
| <code>openspec\workset\remove\\TU\textless{ }name\TU\textgreater{ }</code> | Delete a saved workset without touching member folders.   | <code>-V-json</code>                                                                                                                                                                                           |

When the working root should come from a registered store rather than the current project, you can add `-V-store\\TU\textless{ }id\TU\textgreater{ }` to commands that support it. If shared context is missing or unexpectedly empty, it may help to run `openspec\store\doctor` first. That checks the registration boundary before you inspect individual artifacts.

#### OpenSpec Stores are still in BETA

Please keep in mind the OpenSpec Stores feature is still in BETA release and may change before they are released as a production feature in OpenSpec.

## B.2. GitHub Copilot Commands and OpenSpec Skills

OpenSpec names each workflow by its Agent Skill, while the command a developer types depends on the agent harness running it. This section covers GitHub Copilot inside VS Code, IntelliJ IDEA, and GitHub Copilot CLI. VS Code and IntelliJ IDEA use GitHub Copilot's prompt-file integration. Because GitHub Copilot CLI does not read `.github/prompts/`, the installed skills are the appropriate option there.<sup>1</sup>

The appropriate integration can install the relevant files when you initialize the repository, for example with `openspec\init\ -V-tools\github-copilot`.<sup>2</sup> The table uses the current OPSX command form for the Copilot extensions and the Agent Skill form for Copilot CLI. OpenSpec documents the command pattern using the apply workflow; the other workflows appear to follow the same host-specific naming pattern.<sup>3</sup>

| OpenSpec action | When to use it                                                                                                                                          | VS Code                                                                | IntelliJ IDEA                                                          | GitHub Copilot CLI                                                              |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| explore         | Investigate an idea, existing behavior, or an unclear requirement before creating a change proposal. It is a thinking step and does not implement code. | <code>/opsx-explore\TU\textless{ }Topic\ TU\textgreater{ }</code>      | <code>/opsx-explore\TU\textless{ }Topic\ TU\textgreater{ }</code>      | <code>/openspec-explore\TU\textless{ }Topic\ TU\textgreater{ }</code>           |
| propose         | Turn the agreed problem into a change proposal and generate its planning artifacts.                                                                     | <code>/opsx-propose\TU\textless{ }change-name\TU\textgreater{ }</code> | <code>/opsx-propose\TU\textless{ }change-name\TU\textgreater{ }</code> | <code>/openspec-propose\TU\textless{ }change-name\TU\textgreater{ }</code>      |
| apply           | Implement the tasks in an existing proposal after reviewing its requirements and design.                                                                | <code>/opsx-apply\TU\textless{ }change-name\TU\textgreater{ }</code>   | <code>/opsx-apply\TU\textless{ }change-name\TU\textgreater{ }</code>   | <code>/openspec-apply-change\TU\textless{ }change-name\TU\textgreater{ }</code> |

<sup>1</sup>OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

<sup>2</sup>OpenSpec CLI reference. The `init` command accepts a `-V-tools` option for selecting integrations.

<sup>3</sup>OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

| OpenSpec action | When to use it                                                                                                                                                                                                       | VS Code                                                                 | IntelliJ IDEA                                                           | GitHub Copilot CLI                                                                     |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| sync            | Merge implemented delta requirements into openspec/specs/ while leaving the change proposal open. Use this when the specification is ready to become the current baseline but the change still needs follow-up work. | /opsx-sync\\TU\<br>textless{ }change-<br>name\\TU\<br>textgreater{ }    | /opsx-sync\\TU\<br>textless{ }change-<br>name\\TU\<br>textgreater{ }    | /openspec-sync-<br>specs\\TU\<br>textless{ }change-<br>name\\TU\<br>textgreater{ }     |
| archive         | Complete a finished change. This merges its deltas into the main specs and moves the change folder into openspec/changes/archive/.                                                                                   | /opsx-archive\\TU\<br>textless{ }change-<br>name\\TU\<br>textgreater{ } | /opsx-archive\\TU\<br>textless{ }change-<br>name\\TU\<br>textgreater{ } | /openspec-archive-<br>change\\TU\<br>textless{ }change-<br>name\\TU\<br>textgreater{ } |

### B.2.1. Using the matrix

The VS Code and IntelliJ IDEA entries match because both use the GitHub Copilot IDE integration. OpenSpec generates prompt files under `.github/prompts/`, which the Copilot extension can expose as slash commands in both editors.<sup>4</sup>

GitHub Copilot CLI uses the skill spelling. For example, the CLI equivalent of `/opsx-apply\pets-search` would be `/openspec-apply-change\pets-search`. The IDE command is not expected to resolve in the CLI because the CLI does not read `.github/prompts/`.<sup>5</sup>

You might use the rows as a working loop rather than as mandatory ceremony for every small edit. Explore can help when the problem or existing behavior is unclear. Propose fits once the intended change is understood well enough to record. Apply is available when the proposal is ready to execute. Sync can update the main specification while the change remains open. Archive generally follows verification and completion, when the change is ready to leave the active-work area and its deltas can become part of the lasting specs.<sup>6</sup>

The terminal CLI is a separate interface. Commands such as `openspec\init` and `openspec\validate` can manage the project and its artifacts from a shell. Slash commands and skills ask an AI coding tool to carry out

<sup>4</sup>OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

<sup>5</sup>OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

<sup>6</sup>OpenSpec skills reference and OpenSpec quickstart. These pages define the explore, propose, apply, sync, and archive workflows and their skill names.

a workflow using those artifacts. If a workflow is unavailable, it may help to check that the repository is initialized and valid and that the selected host can see the files it expects.

## B.3. OpenSpec schema.yaml and Template Reference

OpenSpec uses Schemas to configure how an OpenSpec workflow executes. This section can help when you need to read or change an OpenSpec workflow. It covers the fields in schema.yaml, the dependency rules between artifacts, and the way templates become generated change files.

### B.3.1. Schema fields

An OpenSpec schema defines a planning workflow in YAML. It names the workflow, lists the artifacts it produces, and describes the handoff from planning to implementation. In the built-in spec-driven schema, the default sequence is explicit: proposal leads to specs and design, and tasks depends on both before implementation can begin.

The top-level fields are:

| Field       | Type             | Required? | Effect                                                                                                       |
|-------------|------------------|-----------|--------------------------------------------------------------------------------------------------------------|
| name        | string           | Yes       | Names the schema. OpenSpec uses the schema directory as the lookup key, even if this value differs.          |
| version     | positive integer | Yes       | Records the schema revision. It does not change OpenSpec behavior by itself.                                 |
| description | string           | No        | Describes the workflow when schemas are listed.                                                              |
| artifacts   | non-empty list   | Yes       | Declares the planning artifacts and their dependencies.                                                      |
| apply       | mapping          | No        | Defines what must be complete before implementation starts and, optionally, which task file tracks progress. |

Each entry in artifacts defines one document or one set of documents:

| Field       | Type                  | Required? | Effect                                                                                                                                  |
|-------------|-----------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| id          | unique string         | Yes       | Identifies the artifact. Other artifacts and apply use this value in requires.                                                          |
| generates   | relative path or glob | Yes       | Tells OpenSpec where the artifact is written inside the change folder. For example, specs/**/*.*md can produce one spec per capability. |
| description | string                | Yes       | Labels the artifact in the guidance given to the agent.                                                                                 |
| template    | relative path         | Yes       | Selects the Markdown template under the schema's templates/ directory.                                                                  |
| instruction | string                | No        | Gives the agent artifact-specific guidance, such as whether the document explains why, what, or how.                                    |
| requires    | list of artifact IDs  | No        | Makes this artifact wait until the named artifacts are complete. The default is an empty list.                                          |

It is useful to treat requires as a dependency graph rather than a suggested reading order. OpenSpec marks an artifact ready only after every dependency is complete. It rejects missing IDs, duplicate IDs, and dependency cycles. When several artifacts are ready at once, their order in artifacts determines which one OpenSpec returns first.

The generates value is relative to the active change directory. proposal.md therefore produces openspec/changes/TU\textless{ }change-name\TU\textgreater{ }/proposal.md, while specs/\*\*/\*.\*md matches Markdown files below that change's specs/ directory. Completion is structural: OpenSpec checks that the expected file exists, or that a glob matches at least one file. The check does not evaluate the document's content.

The optional apply block controls the planning-to-implementation handoff. Its requires field names the artifacts that must exist before apply is ready. Its tracks field points to a relative Markdown file in the change directory, normally tasks.md. OpenSpec uses task checkboxes in that file to report whether work is blocked, ready, or complete. An optional instruction replaces the built-in guidance sent to the implementation agent.

The default workflow has this essential shape:

```

name: spec-driven
version: 1
description: Default OpenSpec workflow - proposal -> specs -> design -> tasks
artifacts:
 - id: proposal
 generates: proposal.md
 description: Initial proposal document outlining the change
 template: proposal.md
 requires: []
 - id: specs
 generates: specs/**/*.*md
 description: Detailed specifications for the change
 template: spec.md
 requires:
 - proposal
 - id: design
 generates: design.md
 description: Technical design document with implementation details
 template: design.md
 requires:
 - proposal
 - id: tasks
 generates: tasks.md
 description: Implementation checklist with trackable tasks
 template: tasks.md
 requires:
 - specs
 - design
apply:
 requires:
 - tasks
 tracks: tasks.md

```

The example omits the longer agent instructions. The fields serve different purposes: description labels the artifact, template supplies its output shape, and instruction explains what the agent should produce. A changed dependency changes the workflow; a changed template changes the starting structure of every new artifact created from it.

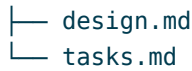
### B.3.2. Template structure

A schema directory contains schema.yaml and a templates/ directory. OpenSpec resolves the template value in each artifact entry relative to that directory:

```

openspec/schemas/spec-driven/
├─ schema.yaml
├─ templates/
│ ├─ proposal.md
│ └─ spec.md

```



Templates are Markdown skeletons rather than files copied into the change directory. OpenSpec gives the selected template to the agent as its expected output format. The agent fills that structure and writes the result at the path named by generates.

In the default schema, the mapping is:

| Artifact | Template    | Generated path                                                | Purpose                                                                                  |
|----------|-------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------|
| proposal | proposal.md | proposal.md                                                   | Explains why the change is needed, what it changes, its capabilities, and its impact.    |
| specs    | spec.md     | specs/TU\textless{ }capability-path\TU\textgreater{ }/spec.md | Defines observable behavior through requirements and scenarios.                          |
| design   | design.md   | design.md                                                     | Explains how the change will be implemented, including decisions, risks, and trade-offs. |
| tasks    | tasks.md    | tasks.md                                                      | Breaks the implementation into ordered, checkable work items.                            |

The specs entry is the exception to the one-file pattern. It allows a change to produce one delta specification for each capability named by the proposal. The proposal establishes the reason and scope, the spec files define behavior, the design records the technical approach, and the task list turns the plan into work the apply phase can track. Keeping these roles separate helps prevent implementation details from replacing requirements.

When a team needs different documents or a different sequence, it can create a project schema under openspec/schemas/TU\textless{ }name\TU\textgreater{ }/, edit schema.yaml and the matching templates, and run openspec\schema\validate\TU\textless{ }name\TU\textgreater{ } before using it. Forking the built-in spec-driven schema copies its schema definition, templates, and instructions into a project-owned directory. A custom template changes the structure of future artifacts; a schema dependency changes which artifacts can be drafted and when. Built-in schemas ship with OpenSpec, so forking one before editing it is preferable to modifying the package copy.

## B.4. Further Reading and Community Resources

OpenSpec is open-source software available through its GitHub project and repository. Those resources include an issue queue and discussions, which can provide information about OpenSpec and Spec-Driven Development more generally.

### **B.4.1. Official documentation**

The OpenSpec repository and its documentation are useful starting points. The OPSX workflow describes the workflow used throughout this book. The installation guide covers setup, while supported tools and aliases lists the available integrations.

These sources cover the mechanics of using OpenSpec. Your repository's proposal, requirements, design, tasks, and verification evidence still provide the context for a specific change. The documentation can explain how a command works, but it cannot decide whether the change is understood well enough to implement or whether the resulting behavior is correct. Your project team remains responsible for the specs, and OpenSpec is more useful when the team understands them fully.

### **B.4.2. Community schemas and extensions**

Community-built schemas and extensions may help when the official OpenSpec workflow does not cover a particular business domain or team practice. They can also provide a working example. It is useful to treat each community-built schema as a proposal to evaluate: read its source and examples, check its maintenance activity and compatibility with your OpenSpec version, and identify the assumptions it makes about your repository and development process.

A small, representative change can provide a useful trial before the schema becomes a team convention. It helps to ask whether its artifacts make requirements easier to review, give Copilot useful context, and produce evidence that the implementation matches the intended behavior. If it adds more SDD ceremony without improving those checks, you might cherry-pick the useful ideas from the community-built schema and leave the rest behind. Recording adopted conventions in the repository gives the next developer a way to understand them.

## C. Workshop Resources

### C.1. Prompt Workbook

Use these prompts as starting points. Replace bracketed decisions with the class's agreed behavior rather than asking the agent to invent them.

#### C.1.1. Explore unit-number search

`/opsx:explore` We are considering an initial commercial truck search capability. Could users search by a 15-character unit number? Please ask about exact versus partial matching, case handling, invalid input, no-match behavior, and the response shown in the UI. Hold off on creating change artifacts until those decisions are clear.

#### C.1.2. Propose unit-number search

`/opsx:propose\add-unit-number-search` Based on the exploration decisions, could you draft the proposal? Please keep implementation choices out of the behavioral requirements and include scenarios for valid input, invalid input, a match, and no match.

#### C.1.3. Add VIN search

`/opsx:propose\add-vin-search-option` Could we add a secondary search mode for a 17-character VIN while preserving unit-number search? Please ask how users would select the mode and how the two modes should share response behavior. Represent the new capability as an additive delta.

#### C.1.4. Modify unit-number validation

`/opsx:propose\update-unit-number-length` We are considering replacing the current unit-number length rule with a maximum of 10 characters. Could you identify every scenario that would change and every existing behavior that should remain intact? Represent the replacement as a modified requirement.

#### C.1.5. Update an active filter proposal

`/opsx:update` Could you revise the active origin-state filter change to include Active Status with values Active and Decommissioned? Please update the proposal, requirements and scenarios, design, and tasks, and flag any unanswered question about how the filters should combine.

### **C.1.6. Review generated artifacts**

Could you review this OpenSpec change without editing it? Please report contradictions, unsupported assumptions, missing edge cases, tasks that do not trace to requirements, and requirements that lack verification tasks.

### **C.1.7. Verify before archive**

Could you compare the implemented change with the active delta and task list before archiving? Please identify unmet scenarios, code outside the approved scope, stale tasks, and tests that do not prove the specified behavior. Hold off on archiving the change.

## **C.2. Artifact Review Checklists**

### **C.2.1. Proposal**

- Does it make the business problem and intended outcome clear?
- Is the scope narrow enough to review and implement with confidence?
- Does it call out the exclusions that matter for this change?
- Does it leave out technical assumptions that have not been approved?
- Could a reviewer tell whether this adds behavior or modifies what is already accepted?

### **C.2.2. Requirements and scenarios**

- Does each requirement describe behavior that someone could observe?
- Are terms such as “maximum,” “exact,” and “optional” clear enough to avoid ambiguity?
- Do the scenarios cover successful, invalid, and empty outcomes where those cases matter?
- Would an additive delta leave existing behavior unchanged?
- Does a modified delta include the full replacement requirement?
- Could each scenario become an automated or repeatable test?

### **C.2.3. Design**

- Does it focus on decisions that have implementation consequences?
- Does it explain the API, UI, validation, and persistence boundaries that may be affected?
- Does it identify compatibility or migration concerns that reviewers may need to consider?
- Does it record tradeoffs instead of presenting one choice as inevitable?
- Does it keep from duplicating behavioral requirements?

### **C.2.4. Tasks**

- Would each task produce a visible code, test, or documentation result?
- Could each requirement be traced to implementation and verification work?
- Are the tasks ordered in a way that reflects their dependencies?
- Are the review and test steps clear enough to follow?
- Would a new agent session be able to tell what remains incomplete?

### C.2.5. Apply review

- Does the implementation appear to match the latest artifacts rather than an earlier chat message?
- Are the frontend and backend validation rules consistent with one another?
- Do the tests demonstrate the scenarios rather than only executing lines?
- Has the agent changed code outside the approved scope?
- Does the task list reflect what has actually been completed?

### C.2.6. Archive readiness

- Has the team accepted the final behavior?
- Do the affected tests pass?
- Would the main spec remain coherent after the delta merges?
- Have overlapping active changes been reviewed against the new baseline?
- Is any unfinished work represented by a separate tracked change?

## C.3. Instructor Guide

### C.3.1. Course shape

Each morning is planned for 90 minutes and follows the same rhythm: establish one concept, demonstrate one artifact transition, let participants perform it, and review the resulting evidence.

| Time      | Morning 1                    | Morning 2                    | Morning 3                        |
|-----------|------------------------------|------------------------------|----------------------------------|
| 0-15 min  | Context continuity and SDD   | Brownfield baseline          | Parallel-change risks            |
| 15-30 min | OpenSpec structure and setup | Additive vs. modified deltas | Worktrees and isolation          |
| 30-55 min | Explore and propose          | Add VIN search               | Propose and update filters       |
| 55-75 min | Apply and test               | Modify unit-number length    | Apply, sync, and resolve overlap |
| 75-90 min | Archive and debrief          | Archive and compare specs    | Governance review and wrap-up    |

### C.3.2. Teaching checkpoints

It may help to measure progress by more than whether the agent generated files. Pause at these points:

1. After exploration, ask which ambiguities a person resolved.
2. After proposal, invite participants to separate behavior from implementation.
3. After apply, consider tracing one scenario through UI, API, persistence, and test code.
4. After archive, read the main spec without referring to the completed delta.

### C.3.3. Expected friction

Participants may type slash commands into the terminal, accept generated assumptions, or treat task checkmarks as test results. Keeping the terminal/chat distinction visible can help, as can asking for an artifact review before each apply or archive step.

Tool output may differ across OpenSpec releases or coding-agent models. Teach the invariant workflow and review criteria rather than requiring identical generated prose.

### C.3.4. Recovery points

Maintain three instructor baselines:

- **Baseline A:** Initialized project with no truck-search spec.
- **Baseline B:** Archived unit-number search.
- **Baseline C:** Archived VIN search and 10-character unit-number rule.

These baselines can keep a setup problem from consuming the next morning. They also let the instructor demonstrate a delta against a known source of truth.

### C.3.5. Closing discussion

Ask participants to name one change in their own codebase where conflicting interpretations would cost more than writing a short spec. Then ask for one change too small or reversible to justify the workflow. The contrast can help keep SDD grounded as an engineering tradeoff rather than a universal ceremony.